

The Open Group Guide

A Guide to Modeling SABSA® with the ArchiMate® Specification



Copyright © 2024, The Open Group and The SABSA Institute C.I.C. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior permission of the copyright owners unless it is presented in its current form as published by The Open Group and The SABSA Institute.

If you did not obtain this copy through The Open Group, it may not be the latest version. For your convenience, the latest version of this publication may be downloaded at www.opengroup.org/library.

The Open Group Guide

A Guide to Modeling SABSA® with the ArchiMate® Specification

ISBN: 1-957866-55-0

Document Number: G24B

Published by The Open Group, December 2024.

Comments relating to the material contained in this document may be submitted to:

The Open Group, Apex Plaza, Forbury Road, Reading, Berkshire, RG1 1AX, United Kingdom

or by electronic mail to:

ogspecs@opengroup.org

Contents

1	Introduction.....	1
1.1	Background.....	1
1.2	The ArchiMate Specification.....	2
1.3	Purpose	4
2	Rationale for the Alignment of SABSA and the ArchiMate Modeling Language ...	5
2.1	Benefits of Modeling	5
2.2	The Case for an ArchiMate Security Perspective	6
2.3	Benefits that Modeling Can Bring to SABSA	7
2.4	Benefits of Modeling to the Practitioner.....	8
2.5	Vendor Neutrality	8
3	An Introduction to the ArchiMate Language	10
3.1	Core Elements.....	10
3.2	Core Relationships.....	10
3.2.1	Definition and Notation of Relationships.....	11
3.2.2	Derived Relationships	12
3.2.3	The Core Layers and their Elements	12
3.3	Extension Layers and Elements	16
3.3.1	Strategy Layer	16
3.3.2	Motivation Elements	16
3.3.3	Implementation and Migration Layer.....	17
3.3.4	Composite Elements.....	18
3.4	ArchiMate Language Customization	18
3.4.1	User-Defined Attributes	18
3.4.2	Specializations and “Stereotypes”	18
3.4.3	Overloaded Relationships.....	19
3.5	The ArchiMate Full Framework	20
4	Aligning the SABSA and ArchiMate Frameworks.....	22
4.1	Introduction to the Security Overlay.....	22
4.2	An Overview of the Task.....	23
4.3	Risk & Security Modeling in the ArchiMate Specification.....	24
4.4	The Basic Element and Relationships.....	28
5	The Motivation Aspect.....	29
5.1	Value & Loss	29
5.2	Value Chain	31
5.3	SABSA Business Attributes	32
5.3.1	Structural Placement of Business Attributes	34
5.3.2	Traceability of Business Attributes	37
5.4	Meaning	39
5.5	Impact, Threat, Vulnerability, and Risk.....	40
5.6	Controls: Objectives, Requirements, and Measures	41
5.7	Multi-Tiered Security	42
5.8	Regulations and Standards.....	43
5.9	Articles, Mandates, and Compliance Objectives	44

5.10	Control Mechanisms	47
5.11	Trust.....	48
6	Modeling the Contextual Security Architecture	49
6.1	Business Assets.....	51
6.1.1	Capability and Value Stream.....	51
6.1.2	Business Object	52
6.1.3	Business Service, Interface, and Service Level Agreements.....	54
6.2	Business Risk.....	55
6.3	Business Process/Function/Interaction	55
6.4	Business Roles and Actors.....	57
6.4.1	Governance.....	59
6.4.2	Threat Actors.....	61
6.5	Business Geography	63
6.6	Business Time Dependencies	63
7	Modeling the Conceptual Security Architecture.....	64
7.1	Attribute Profiling.....	65
7.2	Risk Management & Strategy.....	68
7.2.1	Risk Management.....	68
7.2.2	Policy Architecture.....	69
7.2.3	Multi-Regulatory Compliance.....	71
7.3	Conceptual Security Services	73
7.4	Identity and Trust.....	74
7.4.1	Identity and Access Rights	74
7.4.2	Roles and Responsibilities.....	77
7.4.3	Trust	78
7.5	Domain Framework Model.....	82
7.6	Security Events	83
8	Modeling the Logical Security Architecture	84
8.1	Information Assets.....	85
8.1.1	Application Components.....	85
8.1.2	Security Configuration	87
8.1.3	Software Defects and Malware	87
8.1.4	Data Assets	87
8.2	Risk Modeling	89
8.2.1	Risk Modeling.....	89
8.2.2	The Scenario.....	89
8.3	Application Functionality and Services	92
8.4	Logical Access Management	94
8.4.1	Account	95
8.4.2	Application Role.....	95
8.4.3	Application Service	95
8.4.4	Application Process and Function.....	96
8.4.5	Application Interface.....	96
8.5	Logical Domains.....	98
8.6	Timing and Events.....	98
8.6.1	Application Security Events.....	98

9	Modeling the Physical Security Architecture.....	99
9.1	Data and Technology Assets.....	101
9.1.1	Artifact	101
9.1.2	Device and Node	103
9.2	Risk Management Practices.....	103
9.2.1	Defect	103
9.3	Process Mechanisms	104
9.3.1	Technology Functions and Services.....	105
9.3.2	System Software.....	106
9.4	Human-Machine Interfaces.....	106
9.4.1	Technology Interface.....	106
9.5	Physical Environment	106
9.6	Timing and Interrupts	106
9.6.1	Technology Security Events.....	106
10	Conclusion	110

Preface

The Open Group

The Open Group is a global consortium that enables the achievement of business objectives through technology standards and open source initiatives by fostering a culture of collaboration, inclusivity, and mutual respect among our diverse group of 900+ memberships. Our membership includes customers, systems and solutions suppliers, tool vendors, integrators, academics, and consultants across multiple industries.

The mission of The Open Group is to drive the creation of Boundaryless Information Flow™ achieved by:

- Working with customers to capture, understand, and address current and emerging requirements, establish policies, and share best practices
- Working with suppliers, consortia, and standards bodies to develop consensus and facilitate interoperability, to evolve and integrate specifications and open source technologies
- Offering a comprehensive set of services to enhance the operational efficiency of consortia
- Developing and operating the industry's premier certification service and encouraging procurement of certified products

Further information on The Open Group is available at www.opengroup.org.

The Open Group publishes a wide range of technical documentation, most of which is focused on development of Standards and Guides, but which also includes white papers, technical studies, certification and testing documentation, and business titles. Full details are available at www.opengroup.org/library.

This Document

This document is The Open Group Guide to Modeling SABSA® with the ArchiMate® Specification. It is a revision to the 2019 SABSA White Paper¹ that set out a proposal for presenting security concepts using the ArchiMate modeling language.

Encouraged by feedback from Subject Matter Experts (SMEs) in the SABSA and Enterprise Architecture modeling communities, a Joint Working Group was established with The Open Group to develop a settled consensus of core security elements, relationships, and properties – referred to collectively as the “Security Overlay”. Subsequent updates of the content reflect this Working Group’s conclusions and evolution of the ArchiMate language specification.

By proposing a reference of vocabulary, patterns, and conventions in this domain, the authors hope to progress toward standards, to further enterprise security as a practice, and encourage the wider appreciation of security concepts in general and the SABSA approach in particular.

A comprehensive and well-considered Security Overlay should relieve each member of the emerging security modeling community of the need to resolve these issues for themselves,

¹ Refer to: <https://sabsacourses.com/wp-content/uploads/2021/02/TSI-T100-Modelling-SABSA-with-ArchiMate.pdf>.

enabling them to concentrate instead on modeling the security perspective for their systems of interest. Consolidation around such a standard would also incentivize toolmakers to facilitate support for security within their products. While this effort may or may not become that definitive standard, it is hopefully a significant contribution toward it.

This latest revision of the Security Overlay has been updated by The Open Group to become a Guide for inclusion in The Open Group Library. It aligns the document with the ArchiMate 3.2 Specification [C226] and updates cited references to their latest versions. Any comments, feedback, or suggestions are welcomed via the Institute site² or to The Open Group.

² Refer to: <https://sabsa.org/contact>.

Trademarks

ArchiMate, FACE, FACE logo, Future Airborne Capability Environment, Making Standards Work, Open Footprint, Open O logo, Open O and Check certification logo, Open Subsurface Data Universe, OSDU, The Open Group, TOGAF, UNIX, UNIXWARE, and X logo are registered trademarks and Boundaryless Information Flow, Build with Integrity Buy with Confidence, Commercial Aviation Reference Architecture, Dependability Through Assuredness, Digital Practitioner Body of Knowledge, DPBoK, EMMM, FHIM Profile Builder, FHIM logo, FPB, IT4IT, IT4IT logo, O-AA, O-DA, O-DEF, O-HERA, O-PAS, O-TTPS, O-VBA, Open Agile Architecture, Open FAIR, Open Process Automation, Open Trusted Technology Provider, Sensor Integration Simplified, Sensor Open Systems Architecture, SOSA, and SOSA logo are trademarks of The Open Group.

Archi is a registered trademark of Phillip Beauvoir.

BOT is a trademark of Amazon Technologies, Inc.

IT Infrastructure Library and ITIL are registered trademarks of Axelos Limited.

JavaScript is a trademark of Oracle Corporation.

NIST is a trademark of National Institute of Standards and Technology.

SABSA is a registered trademark of The SABSA Institute.

STIX and CVE are trademarks of The MITRE Corporation.

UML is a registered trademark and Business Process Modeling Language, BPML, and Unified Modeling Language are trademarks of the Object Management Group.

XML is a registered trademark of W3C[®], World Wide Web Consortium.

All other brands, company, and product names are used for identification purposes only and may be trademarks that are the sole property of their respective owners.

Acknowledgements

(Please note affiliations were current at the time of approval.)

The Open Group gratefully acknowledges the contribution of the following people in the development of this document:

- Steven Bradley, Cyber Enterprise Modelling; The SABSA Institute
- John Czaplewski, The SABSA Institute
- John Sherwood, The SABSA Institute
- Maurice Smit, The SABSA Institute

The Open Group gratefully acknowledges the following reviewers who participated in the review of this document:

- Steven Bradley, Cyber Enterprise Modelling; The SABSA Institute
- Rolf Knoll, NovaTec Consulting GmbH
- John Linford, The Open Group
- Jean-Baptiste Sarrodie, BNP PARIBAS S.A.

Referenced Documents

The following documents are referenced in this guide.

(Please note that the links below are good at the time of writing but cannot be guaranteed for the future.)

[Archi] The Archi ArchiMate Modeling Toolkit; refer to: <https://www.archimatetool.com/>

[Broy et al., 2011]

What is the Benefit of a Model-Based Design of Embedded Software Systems in the Car Industry, by Manfred Broy, Sascha Kirstan, Helmut Krcmar, and Bernhard Schätz, published in *Emerging Technologies for the Evolution and Maintenance of Software Models*, Eds. Jörg Rech, Christian Bunse, by IGI Global, January 2011

[C19C] ArchiMate® Model Exchange File Format for the ArchiMate Modeling Language, version 3.1, a standard of The Open Group (C19C), published by The Open Group, November 2019; refer to: www.opengroup.org/library/c19c

[C20B] Risk Taxonomy (O-RT), Version 3.0.1, a standard of The Open Group (C20B), published by The Open Group, November 2021; refer to: www.opengroup.org/library/c20b

[C220] TOGAF® Standard, 10th Edition, a standard of The Open Group (C220), published by The Open Group, April 2022; refer to: www.opengroup.org/library/c220

[C226] ArchiMate® 3.2 Specification, a standard of The Open Group (C226), published by The Open Group, October 2022; refer to: www.opengroup.org/library/c226

[Fowler, 2023]

Who Needs an Architect?, by Martin Fowler; refer to: <http://files.catwell.info/misc/mirror/2003-martin-fowler-who-needs-an-architect.pdf>

[G112] Open Enterprise Security Architecture (O-ESA): A Framework and Template for Policy-Driven Security, The Open Group Guide (G112), published by The Open Group, April 2011; refer to: www.opengroup.org/library/g112

[G152] TOGAF® Series Guide: Integrating Risk and Security within a TOGAF® Enterprise Architecture (G152), part of The Open Group TOGAF® Standard, 10th Edition, published by The Open Group, April 2022; refer to: www.opengroup.org/library/g152

[G204] A Guide to the ArchiMate® Model Exchange File Format, The Open Group Guide (G204), published by The Open Group, March 2020; refer to: www.opengroup.org/library/g204

[G21A] Open FAIR™ Risk Analysis Example Guide, The Open Group Guide (G21A), published by The Open Group, July 2021; refer to: www.opengroup.org/library/g21a

[GDPR] Regulation (EU) 2016/679 On the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data, April 2016, updated May 2018; refer to: <https://eur-lex.europa.eu/legal-content/EN/AUTO/?uri=CELEX:02016R0679-20160504&qid=1532348683434>

[Goldstein & Goldstein, 1997]

Feynman's Lost Lecture: The Motion of Planets Around the Sun, by David L. Goldstein and Judith R. Goldstein, published by Vintage, May 1997

[ISO/IEC 27001]

ISO/IEC 27001:2022 Information Security, Cybersecurity and Privacy Protection – Information

- Security Management Systems – Requirements, published by ISO, October 2022; refer to: <https://www.iso.org/standard/27001>
- [ISO/IEC 29115]
Information Technology – Security Techniques – Entity Authentication Assurance Framework, published April 2013; refer to: <https://www.iso.org/standard/45138.html>
- [ISO/IEC/IEEE 42010]
ISO/IEC/IEEE 42010:2022 Software, Systems and Enterprise – Architecture Description, published by ISO, November 2022; refer to: <https://www.iso.org/standard/74393.html>
- [ISO/IEC/IEEE 42020]
ISO/IEC/IEEE 42020:2019 Software, Systems and Enterprise – Architecture Processes, published by ISO, July 2019; refer to: <https://www.iso.org/standard/68982.html>
- [ISO/IEC/IEEE 42030]
ISO/IEC/IEEE 42030:2019 Software, Systems and Enterprise – Architecture Evaluation Framework, published by ISO, July 2019; refer to: <https://www.iso.org/standard/73436.html>
- [NIST CSF]
The NIST Cybersecurity Framework 2.0; refer to: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>
- [NIST SP 1500]
Big Data Interoperability Framework, NIST Special Publication 1500; refer to: <https://www.nist.gov/itl/big-data-nist/big-data-nist-documents/nbdif-version-30-final>
- [NIST SP 800-53]
NIST SP 800-53 Rev.5 Security and Privacy Controls for Information Systems and Organizations, published by NIST, September 2020; refer to: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- [NIST SP 800-63-3]
Digital Identity Guidelines, NIST Special Publication 800-63, Revision 3, published by NIST, June 2017; refer to: <https://pages.nist.gov/800-63-3/sp800-63-3.html>
- [SABSA R101]
R101, SABSA Matrices 2018, A Report published by The SABSA Press, an Imprint of The SABSA Institute, June 2018; refer to: <https://sabsacourses.com/wp-content/uploads/2021/02/TSI-R101-SABSA-Matrices-2018-Release-Notes.pdf>
- [Sherwood et al., 2005]
Enterprise Security Architecture: A Business-Driven Approach, by John Sherwood, Andrew Clark, and David Lynas, published by CRC Press, November 2005
- [UCF] The Unified Compliance Framework®; refer to: <https://www.unifiedcompliance.com/home>
- [W117] TOGAF® and SABSA® Integration, The Open Group White Paper (W117), published by The Open Group, October 2011; refer to: www.opengroup.org/library/w117
- [W172] How to Model Enterprise Risk Management and Security with the ArchiMate® Language, The Open Group White Paper (W172), published by The Open Group, March 2017, updated November 2019; refer to: www.opengroup.org/library/w172
- [Wierda, 2021]
Mastering ArchiMate Edition 3.1: A Serious Introduction to the ArchiMate® Enterprise Architecture Modeling Language, by Gerben Wierda, published by R&A, April 2021

1 Introduction

This document discusses how security architecture concepts can be expressed using The Open Group ArchiMate® modeling language [C226]. It describes a model-based approach for creating SABSA® artifacts using ArchiMate notation and tooling.

The integration of security into Enterprise Architecture methodologies, through the alignment of SABSA concepts with the Zachman Framework™, the TOGAF® Standard [C220], and other popular frameworks, has been established for several years. In practice, however, the lack of native support for security concepts in Enterprise Architecture modeling notations (and therefore tools and processes) has placed security architects at a disadvantage to their architectural peers.

A means of expressing the full range of security perspectives in the ArchiMate modeling language addresses a deficiency that has been impeding the realization of a truly holistic architecture methodology that serves the needs of all architecture practitioners with a concern for security; not just security architects.

1.1 Background

In the not-too-distant past, information security was considered a collection of disparate disciplines (people, processes, and technologies) that were detached from the main body of Enterprise Architecture. Yet it is self-evident that security architecture is heavily context-dependent – it cannot and does not exist in isolation. Its purpose is to protect the organization’s information assets and capabilities. It produces artifacts that guide the Enterprise Architecture development.

Thankfully, this demarcation is no longer the norm, having been challenged, both conceptually by the philosophy developed in the “SABSA Blue Book” [Sherwood et al., 2005] and also by the subsequent integration and alignment of the SABSA approach with other Enterprise Architecture frameworks,³ including the TOGAF Standard, IT Infrastructure Library® (ITIL®), Zachman Framework, and the Department of Defense Architecture Framework (DODAF). An approach to integrate the TOGAF Standard with the SABSA security architecture was published in the 2011 TOGAF® and SABSA® Integration White Paper [W117] and is also included in The TOGAF Series Guide: Integrating Risk and Security within a TOGAF Enterprise Architecture [G152].

The Open Group and SABSA collaboration aligns and integrates essential concepts of the two frameworks. It recognizes the need for a truly holistic approach. Nevertheless, there are practical impediments to adopting an integrated methodology: the lack of security support in popular Enterprise Architecture modeling languages and the consequent underdevelopment of the topic in the general body of knowledge and its implementation in modeling tools and development environments.

A model-based approach is as essential to security as it is to any other design and engineering discipline. Models amplify the efficiency and effectiveness of practitioners, and provide a valuable

³ The SABSA Institute: SABSA Executive Summary; refer to: <https://sabsa.org/sabsa-executive-summary/>.

means of establishing a common understanding with stakeholders – a pre-requisite for quality. And yet the analyst’s toolkit (Threat Models, Trust Models, Risk Analysis, Control Traceability, etc.) are too often a collection of disparate artifacts, detached from core Enterprise Architecture models. Consequently, security artifacts tend to be costly to create and maintain, desynchronized and inconsistent with the main body of system documentation and therefore poorly integrated with related domains such as Enterprise Risk Management, Governance, or Compliance.

Ultimately, real-world systems recognize a single, de facto architecture: what ISO 42010 calls the “Entity of Interest”⁴ (EoI) [ISO/IEC/IEEE 42010]. It makes sense to strive towards a single holistic model⁵ of that EoI, capable of describing, validating, querying and analyzing all its pertinent functional and non-functional aspects, including the security perspective.

1.2 The ArchiMate Specification

The ArchiMate Specification [C226] is an open and independent modeling language for Enterprise Architecture published by The Open Group. It supports the description, analysis, and development of enterprise business and IT systems in the same way that the Unified Modeling Language™ (UML®) describes software design or the Business Process Modeling Language™ (BPML™) describes business processes.

The ArchiMate Core Framework resolves Enterprise Architecture into three core layers: Business, Application (Software & Data), and Technology Infrastructure.

In each layer, three aspects are considered: active structure elements (actors, roles, applications) perform behaviors (services realized by processes and functions) on passive structural elements (information, data) via a constrained set of relations. Figure 1 shows a simple example.

⁴ This second edition changed the subject of an architecture description from what was previously referred to as a “System of Interest” to an “Entity of Interest”. This aligns with ISO/IEC/IEEE 42020 and ISO/IEC/IEEE 42030 and other non-system architecture contexts.

⁵ Or a series of interconnected models forming a complete Architectural Description.

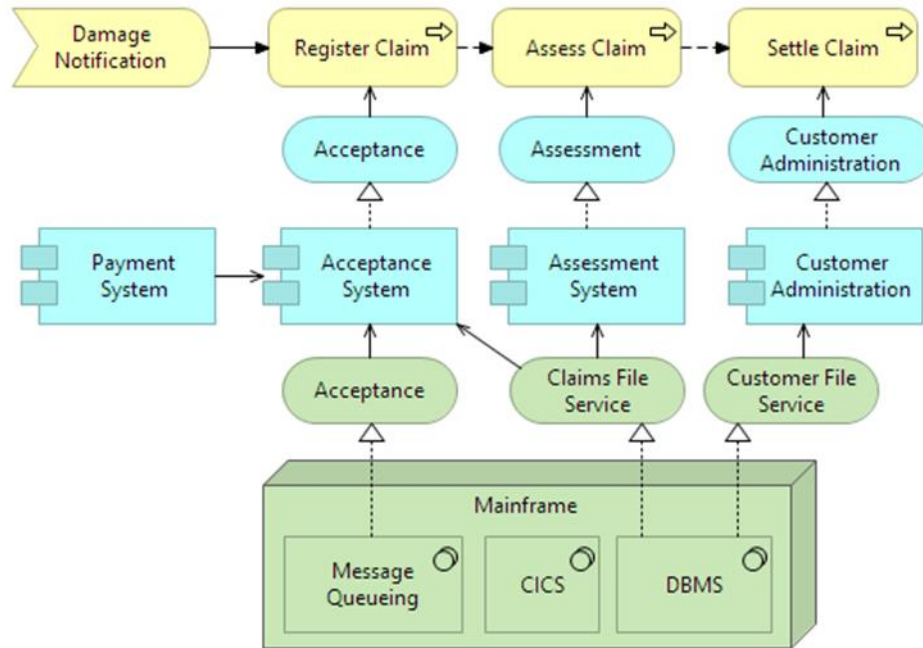


Figure 1: A Simple ArchiMate Diagram

The ArchiMate Core Framework is supplemented with additional layers and perspectives including Strategy, Motivation, Physical, and Implementation & Migration, to form an extended framework known as the ArchiMate Full Framework. A brief primer on the notation is presented in Section 3: “An Introduction to the ArchiMate Language”.

The ArchiMate language has existed for over twenty years.⁶ Its popularity is the result of its essential usefulness (developed through practitioner feedback from earlier versions), its concise but expressive notation (easy to learn, yet capable of communicating complex patterns with precision and clarity), a wealth of literature covering everything from language semantics to good modeling style, the availability of certified, compliant modeling tools to suit all budgets (including excellent freeware), and the portability of models (via a standardized ArchiMate Model Exchange File Format).

Currently, there are over 15,500 certifications in the ArchiMate People Certification program worldwide,⁷ and tens of thousands of free ArchiMate resources have been downloaded from The Open Group website.⁸

Despite this progress, perhaps the least developed aspect of the notation remains its lack of specific support for security, although the specification does illustrate how language customizations could be used to partially address the need. As a result, a detailed approach to addressing security has been a gap.

⁶ The ArchiMate Specification originated from the Telematica Instituut in the Netherlands in cooperation with several Dutch partners from government, industry, and academia. In 2008, stewardship was transferred to the ArchiMate Forum within The Open Group. The ArchiMate 1.0 Specification was first published as a formal technical standard in 2009.

⁷ ArchiMate Certification: Directory of Certified People; refer to: <https://archimate-cert.opengroup.org/certified-individuals>.

⁸ Refer to: The Open Group ArchiMate Library: <https://publications.opengroup.org/archimate-library>.

1.3 Purpose

This document explores how to express security concepts in the ArchiMate Specification using only the in-built extensibility capabilities of the existing notation. It aims to be as complete as reasonably possible, using the ability to create the artifacts identified in the SABSA framework as a benchmark. Its goal is to describe a unified Enterprise Architecture/Enterprise Security Architecture modeling environment capable of supporting in practice what *TOGAF® and SABSA® Integration*, the Joint White Paper [W117] and the *TOGAF® Series Guide: Integrating Risk and Security within a TOGAF® Enterprise Architecture* [G152] have already established in theory.

The analysis explores the following questions:

- What is the rationale for using the ArchiMate language to model security concerns in general and SABSA in particular?
 - What motivates the goal of expressing security in a modeling language?
 - What does a model-driven approach offer to SABSA practitioners and other security stakeholders?
- Which kinds of security artifact are required from an ArchiMate model?
- What are the gaps and limitations of the current language specification?
- Which new elements, relationships, properties and viewpoints are needed to create these artifacts?

The analysis leads to two main focus areas:

- What can currently be achieved within the constraints of ArchiMate language and certified tools?
- What might further be achieved via a security-specific extension to the core language, and what would such an extension look like?

2 Rationale for the Alignment of SABSA and the ArchiMate Modeling Language

Before engaging in a discussion of the “what” and “how” of security modeling, it is worth stepping back to reflect on the “why”. What can we hope to achieve by augmenting the ArchiMate security modeling capability?

2.1 Benefits of Modeling

Models are used in virtually all science and engineering disciplines to represent complex ideas or systems. They serve in many ways: to document, query, analyze, explain, explore, validate, plan, teach, or predict a system’s structure or behavior without the need for interaction with the system itself.

The ICT discipline’s drive to digitalization for agility and competitiveness demands that organizations actively manage their Enterprise Architecture. The scale and complexity of these systems present a significant challenge to managing that change and demands an effective Enterprise Architecture modeling capability – practicing Enterprise Architecture without specialist Enterprise Architecture tools is analogous to maintaining documentation without a word processor.

The ability to interact with models has many advantages over doing so with real systems. Generally, they are available earlier and are, by definition, faster, cheaper, more flexible and accessible, safer, and less disruptive.

A study [Broy et al., 2011] of the benefits of model-based engineering estimated an average 27% cost saving and a 36% time saving (rising to 40% and 50% respectively when the model includes testing). It also found a tendency to produce “*better-architected solutions*” – the evaluation of a model promotes understanding, reveals problems that can be addressed early, and allows the selection of Solution Building Blocks (SBBs) (products and components) to be deferred until a more complete and well-defined set of requirements has been elicited. The characteristics of modeling that produce these benefits include:

- Resolving complexity

Models allow modelers to highlight focus points by abstracting out irrelevant or inconsequential details. They support navigation capabilities that provide links to related views, or zoom in or out to show elements in context or resolve their finer detail. Views can be tailored to reflect specific stakeholder concerns, eliminating unnecessary complexity and presenting what remains with clarity.

- Quality of documentation

Documentation quality is likely to be enhanced in two aspects:

- Concise, expressive, comprehensible notation enables a more efficient, effective review process

This encourages greater involvement and feedback from stakeholders who otherwise may not devote enough time or cognitive effort to the (recurrent) review of a lengthy textual document.

— By generating the documentation set from the model, artifacts are referentially consistent, more easily kept up to date, better targeted to specific stakeholders, and available in multiple formats/media types (traditional document, intranet, etc.) at a fraction of the effort of manual authoring

- Efficiency

Human visual processing is our most powerful faculty for making sense of the surrounding world. Numerous studies⁹ have shown that visual information is absorbed and analyzed from text and pictures with far less cognitive effort than from text alone. And, from an author's perspective, an ability to generate documentation from a model produces artifacts that are consistent, easily refreshed, up-to-date, and generally maintainable at higher quality for less effort.

- Better collaboration

A good visual notation is concise, expressive, and yet, being semi-formal, conveys precise meaning. It provides a more effective communication medium than a verbose textual description. This advantage is especially beneficial across international projects with multilingual teams.

- Reuse

Experienced modelers express recurring ideas using established notational patterns. Consistent use of such design patterns facilitates comprehension through recognition: implicitly conveying contextual information such as problem analysis, design trade-offs, or emphasis on resulting system characteristics such as flexibility, extensibility, separation of concerns, policy compliance, etc.

- Stakeholder buy-in

The ability to create and maintain viewpoints addressing specific stakeholder concerns and perspectives (with minimal additional effort) helps provide assurance for stakeholders about the system or project, manage their expectations, and help maintain their buy-in.

- Standardization

Standards promote sharing beyond the immediate project team. Standards make it possible to benefit from a wide pool of literature, blogs, tools, trainings, certifications, etc., and facilitate the exchange of artifacts with partners, suppliers, customers, and communities of practice.

2.2 The Case for an ArchiMate Security Perspective

The ArchiMate Specification delivers the benefits of modeling to architects and designers, and is designed to support the TOGAF Standard. However, the ArchiMate notation provides only basic support for the representation of security concerns. Consequently, security artifacts must be produced largely outside the Enterprise Architecture modeling environment, using separate models, notations, tools, and processes. This results in parallel representations of the EoI that Enterprise Architecture and Enterprise Security Architecture teams must strive to reconcile and synchronize, to the detriment of the organization's overall security posture.

⁹ Including: Levie & Lentz, 1982; Levin, Anglin, & Carney, 1987

With a Security Overlay extending the language with a means of expressing security perspectives, the ArchiMate modeling language has the potential to unify these distinct but intrinsically interrelated domains.

2.3 Benefits that Modeling Can Bring to SABSA

The SABSA methodology focuses on risk-driven enterprise information security and information assurance. Its goal is to deliver security solutions and architectures designed to support critical business initiatives. Perhaps SABSA's most striking achievement is its completeness: every conceivable security service and activity practiced has its logical place and sequence in the SABSA Matrices. Strategic goals in the upper layers can be traced to requirements and control measures in the layers below, and back again.

As elegant and as comprehensive as SABSA method is, performing it in earnest generates volumes of complex, interdependent artifacts – creating a significant documentation management challenge. Because each cell of the SABSA Matrix is related to those in the same row and column, the artifacts produced from any cell must be consistently maintained with peers in the same row, be aligned with the requirements and assumptions of the cell above, and be traceable to consequences in the cell below, while remaining aligned with the activities of the corresponding cell in the Management Matrix.

In practice, most artifacts require several review cycles. Review feedback can introduce changes that may impact several related documents. Applying these additional changes can drive additional revisions, and trigger further review cycles, resulting in complex, interconnected documents under perpetual change.

At the same time, IT Divisions are adopting Agile (code sprints, rapid review cycles, and direct stakeholder feedback), creating pressure to eliminate all but the most essential documentation. If the traditional approach to security struggles to keep pace, what might be the consequences for security?

Performing SABSA well requires a formidable information management capability in order to keep artifacts current and consistent at the required velocity. A traditional approach, based on maintaining multiple independent artifacts (documents, spreadsheets, diagrams, etc.) in referential integrity, requires effort that scales exponentially with the number of items and rate of change. The demands seem irreconcilable.

A model-driven approach has the potential to resolve this information management problem. By generating artifacts as views of a single underlying model, any change to an item is immediately effective globally. Qualities such as referential integrity can be revalidated automatically. The review cycle is streamlined (based on views reflecting stakeholder perspectives) and, when all is correct, the documentation set can be autogenerated with the assurance that it is consistent with a single source of truth.

Security is such an essential aspect of modern enterprises that the benefits of modeling are worth pursuing. The SABSA methodology is recognized as the gold standard in Enterprise Security Architecture. Yet arguably, the resources required to perform SABSA properly have been a significant barrier to adoption for some organizations. The development of sophisticated security capabilities that are compatible with existing ArchiMate tooling should significantly lower this barrier and enable SABSA's rigor to become economically viable for mainstream commercial organizations.

Complete standardization of an ArchiMate Security Overlay is still in its infancy, and incorporation into the core language only a distant prospect. In the meantime, this document represents a set of ideas that explore what can be achieved within the constraints of the modeling language as defined in the ArchiMate 3.2 Specification. Hopefully, the ideas presented here will continue to be refined, developed, and extended into a settled consensus of the security community.

2.4 Benefits of Modeling to the Practitioner

The scientist Richard Feynman famously concluded that, because he could not explain one of the finer aspects of particle physics at freshman level, he did not really understand the mechanism himself [Goldstein & Goldstein, 1997] (and in his case, by extension then, that the scientific community did not really understand either!).

It is not necessary to be a Nobel Prize winning physicist to have had similar experiences – to have observed that the act of committing an idea to writing, of explaining a concept in layman’s terms or of implementing some intuitive functionality in software will often uncover gaps, nuances, false assumptions, special cases, and other conundrums that ultimately lead to a deeper and more complete understanding.

Modeling is just such an activity: the factorization of an EoI into structural/behavioral dimensions, the focus on one aspect at a time, the careful construction of viewpoints within notational constraints. The act of modeling often reveals key insights that might otherwise be glossed over in an informal diagram, lost in the data of a spreadsheet or lie inconspicuously in a lengthy, serialized, text-based description. In short, the discipline of modeling is likely to train the practitioner into being a sharper, more effective, security analyst.

2.5 Vendor Neutrality

The SABSA Institute is committed to preserving the vendor neutrality of the SABSA methodology and its related artifacts. In keeping with this principle, every aspect introduced by the Security Overlay and described in this document is fully compatible with the ArchiMate Specification and any product that supports The Open Group ArchiMate® Model Exchange File Format [C19C].

With that made clear, the ArchiMate diagrams featured in this document and related models¹⁰ have been created using the free Archi® modeling tool [Archi]. In using Archi, the intention was to minimize the entry barriers for any reader new to modeling with the ArchiMate notation, but who wishes to try these techniques for themselves.

This document leverages the tool support for custom icons¹¹ for expressiveness: a visual shorthand for properties that would otherwise require inspection of underlying properties. However, because custom iconography is a non-standard feature, the Security Overlay neither relies on it nor defines an extended icon set.

The Security Overlay leverages the ArchiMate specialization relationship, which enables the ability to customize the language with new element / relationship types by extending existing ones. Although in the specification, the UML-style <<stereotype>> notation is used as an alternative

¹⁰ Many of which are made available from The SABSA Institute site; refer to: <https://sabsa.org/>.

¹¹ Featured in Archi v4.9 and later.

expression of specialization, this document, for reasons explained in Section 3.4.2, makes a firmer distinction between specializations, created using the specialization relationship, and those marked by double chevrons.

As a result of this distinction, the Security Overlay adopts a more formal mechanism of denoting <<stereotypes>> than reliance on a naming convention alone: a “stereotype” property on the element being extended. This approach is fully interoperable because no information is lost in export in the Model Exchange File format, though the rendering of stereotype names may look slightly different in the target environment.

3 An Introduction to the ArchiMate Language

This document assumes no familiarity on the part of the reader with the ArchiMate language. This section presents a brief introduction to the structure and grammar of the notation to help familiarize readers to the level needed to evaluate the proposals that will be put forward.

3.1 Core Elements

The ArchiMate core elements belong to one of three categories:

- **Active Structure Elements:** Entities capable of performing behavior, such as applications or human / organizational actors and their roles relating to particular activities
- **Behavior Elements:** Units of activity performed by Active Structure Elements (events are specific types of behavior that denote a state change)
- **Passive Structure Elements:** Upon which activity is performed such as business information, data (and their physical realization in files or databases) or other consumable resources

This Active->Behavior->Passive syntax provides the basic grammar of the core language, rather like the *subject-verb-object* construct of a natural language. The Active and Behavior types are refined further to support service-oriented viewpoints:

- **External Behavior Elements (Services):** Units of functionality exposed to the external environment. Services provide a “Service Definition” while concealing the details of their internal implementation
- **Internal Behavior Elements (Processes & Functions):** Define internal processing behind services
- **External Active Structure Elements (Interfaces):** Access points to services
- **Internal Active Structure Elements (Actors, Roles, Applications, Devices):** Active entities capable of offering interfaces and performing behavior

3.2 Core Relationships

ArchiMate elements relate to each other through a set of core relationships, grouped into four categories:

- **Structural** relationships: Model the static construction or composition of elements: composition, aggregation, realization, and assignment
- **Dependency** relationships: How elements support each other: serving, access, influence, and association
- **Dynamic** relationships: Model information and control flows between elements: triggering and flow
- The specialization relationship stands apart as it is not classified into any of the above categories

The language does not allow elements and relationships to be combined arbitrarily – each relationship is constrained to connect a predefined set of valid source and target elements, usually in a specific direction.

Relationships are directional and connect exactly two endpoints, though this latter constraint can be extended by using “junction connectors” that allow, for example, the splitting and joining of data flows.

3.2.1 Definition and Notation of Relationships

The definition and notation for each of these relationships are shown in Table 1.

Table 1: ArchiMate Relationships

Structural Relationships		Notation
Composition	An element consists of one or more other concepts.	
Aggregation	An element combines one or more other concepts.	
Assignment	The allocation of responsibility, performance of behavior, storage, or execution.	
Realization	An element plays a critical role in the creation, achievement, sustenance, or operation of a more abstract element.	
Dependency Relationships		Notation
Serving	An element provides its functionality to another element.	
Access	The ability of behavior and active structure elements to observe or act upon passive structure elements.	
Influence	An element affects the implementation or achievement of some motivation element.	
Association	An unspecified relationship, or one not represented by another ArchiMate relationship. Basic association is non-directional, ¹² but specializations may show directionality.	
Dynamic Relationships		Notation
Triggering	A temporal or causal relationship between elements.	
Flow	A transfer (e.g., of information, goods, money, ...) from one element to another.	
Other Relationships		Notation
Specialization	An element is a particular kind of another element.	
Junction	Used to connect relationships of the same type.	● And ○ Or

¹² Although association is defined as non-directional, the underlying model always identifies a source and target element. In effect, although association is visually and semantically bi-directional, in the underlying model, it is not.

To constrain the number of relationships in the vocabulary, some are overloaded; i.e., although their usage is conceptually consistent, their precise meaning adapts to the context of the source and destination elements. This philosophy will be extended when adapting relationships for security contexts.

3.2.2 Derived Relationships

Relationships have a transitive property that allows them to be traversed as chains in the forward direction. The general premise is that if an Element A relates to an Element B ($A \rightarrow B$), and Element B relates to an Element C via another relationship ($B \rightarrow C$), then a derived relationship between Element A and Element C ($A \rightarrow C$) may be said to exist, subject to rules that consider the relationships' type compatibility. The details of these rules have been revised with successive versions of the specification and are now too elaborate to be reproduced here. In simple terms though, each relationship is assigned a relative strength, as shown in Table 2. If a derived relationship exists, its type is determined by the weakest constituent relationship in the chain.

Table 2: Relative Strength in Derived Relationships

(← stronger) Relationship Strength (weaker →)							
composition	aggregation	assignment	realization	serving	access	influence	association

3.2.3 The Core Layers and their Elements

The ArchiMate core language has three core layers into which elements are placed:

- The Business Layer depicts business services offered to customers, which are realized in the organization by business processes performed by business actors
- The Application Layer depicts application services that support the business, and the applications that realize them
- The Technology Layer comprises both information and operational technology

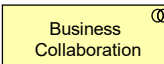
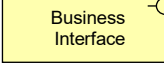
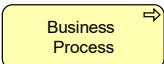
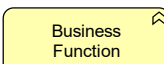
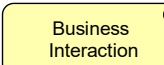
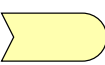
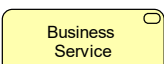
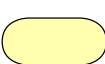
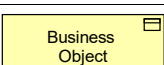
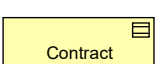
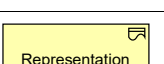
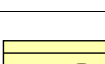
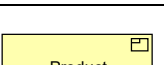
You can model, for example, processing, storage, and communication technology in support of the application world and Business Layers, and model operational or physical technology with facilities, physical equipment, and distribution networks

It is important to note that ArchiMate elements represent classes rather than instances; i.e., they provide type definitions for actors, processes, data, etc., not individual persons, process executions, or documents.

The definition and notation for Business Layer elements are shown in Table 3.

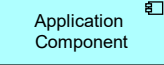
Table 3: Business Layer Elements

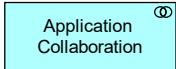
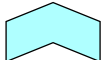
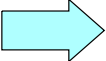
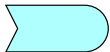
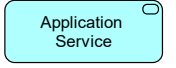
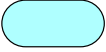
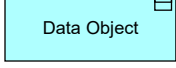
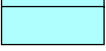
Element	Definition	Notation
Business Actor	A business entity that is capable of performing behavior.	Business Actor  

Element	Definition	Notation
Business Role	The responsibility for performing specific behavior, to which an actor can be assigned, or the part an actor plays in a particular action or event.	 
Business Collaboration	An aggregate of two or more business internal active structure elements that work together to perform collective behavior.	 
Business Interface	A point of access where business services are made available to the environment.	 
Business Process	A sequence of business behaviors that achieves a specific result such as a defined set of products or business services.	 
Business Function	A collection of business behaviors based on a chosen set of criteria such as required business resources and/or competencies, managed or performed as a whole.	 
Business Interaction	A unit of collective business behaviors performed by (a collaboration of) two or more business actors, business roles, or business collaborations.	 
Business Event	A business-related state change.	 
Business Service	An explicitly defined behavior that a business role, business actor, or business collaboration exposes to its environment.	 
Business Object	A concept used within a particular business domain.	 
Contract	A formal or informal specification of an agreement between a provider and a consumer that specifies the rights and obligations associated with a product and establishes functional and non-functional parameters for interaction.	 
Representation	A perceptible form of the information carried by a business object.	 
Product	A coherent collection of services and/or passive structure elements, accompanied by a contract, which is offered as a whole to (internal or external) customers.	 

The definition and notation for Application Layer elements are shown in Table 4.

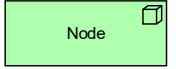
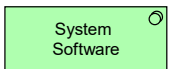
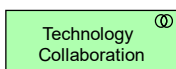
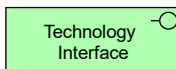
Table 4: Application Layer Elements

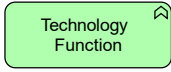
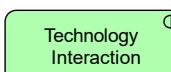
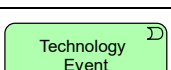
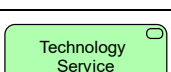
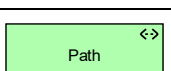
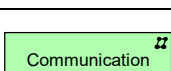
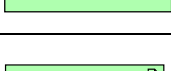
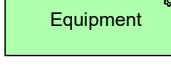
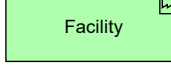
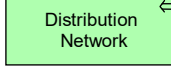
Element	Definition	Notation
Application Component	An encapsulation of application functionality aligned to implementation structure, which is modular and replaceable.	 

Element	Definition	Notation
Application Collaboration	An aggregate of two or more application internal active structure elements that work together to perform collective application behavior.	 
Application Interface	A point of access where application services are made available to a user, another application component, or a node.	 
Application Function	An automated behavior that can be performed by an application component.	 
Application Process	A sequence of application behaviors that achieves a specific result.	 
Application Interaction	A unit of collective application behaviors performed by (a collaboration of) two or more application components.	 
Application Event	An application state change.	 
Application Service	An explicitly defined exposed application behavior.	 
Data Object	Data structured for automated processing.	 

Similarly, the definition and notation for Technology Layer elements are shown in Table 5.

Table 5: Technology Layer Elements

Element	Definition	Notation
Node	A computational or physical resource that hosts, manipulates, or interacts with other computational or physical resources.	 
Device	A physical IT resource upon which system software and artifacts may be stored or deployed for execution.	 
System Software	Software that provides or contributes to an environment for storing, executing, and using software or data deployed within it.	 
Technology Collaboration	An aggregate of two or more technology internal active structure elements that work together to perform collective technology behavior.	 
Technology Interface	A point of access where technology services offered by a technology internal active structure can be accessed.	 

Element	Definition	Notation
Technology Function	A collection of technology behavior that can be performed by a node.	 
Technology Process	A sequence of technology behaviors that achieves a specific result.	 
Technology Interaction	A unit of collective technology behavior performed by (a collaboration of) two or more nodes.	 
Technology Event	A technology state change.	 
Technology Service	An explicitly defined exposed technology behavior.	 
Path	A link between two or more nodes, through which these nodes can exchange data, energy, or material.	 
Communication Network	A set of structures that connects nodes for transmission, routing, and reception of data.	 
Artifact	A piece of data that is used or produced in a software development process, or by deployment and operation of an IT system.	 
Equipment	One or more physical machines, tools, or instruments that can create, use, store, move, or transform materials.	 
Facility	A physical structure or environment.	 
Distribution Network	A physical network used to transport materials or energy.	 
Material	Tangible physical matter or energy.	 

3.2.3.1 Use of Color

An observant reader may notice that the Business, Application, and Technology and Physical elements are shown in different colors (yellow, blue, and green, respectively). However, it is important to note that while a number of color schemes have become popularized, the ArchiMate Specification itself is color-agnostic. Color is defined as carrying no semantics, leaving the modeler free to apply color for visual emphasis.

The colors adopted in this section reflect those used in the ArchiMate Specification but generally, the colors used in model fragments elsewhere in the document follow the 9-color scheme recommended in *Mastering ArchiMate* [Wierda, 2021].

3.3 Extension Layers and Elements

Beyond the core elements, the specification includes other elements relevant to Enterprise Architecture practice.

3.3.1 Strategy Layer

The Strategy elements (Capability, Resource, Course of Action, and Value Stream) are used to model business strategy and capability-based planning. The notation is shown in Table 6.

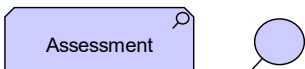
Table 6: Strategy Elements

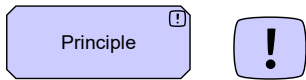
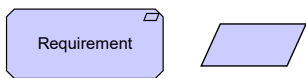
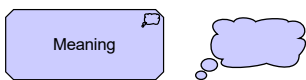
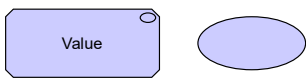
Element	Definition	Notation
Resource	An asset owned or controlled by an individual or organization.	
Capability	An ability that an active structure element, such as an organization, person, or system, possesses.	
Course of Action	An approach or plan for configuring some capabilities and resources of the enterprise, undertaken to achieve a goal.	
Value Stream	A sequence of activities that create an overall result for a customer, stakeholder, or end user.	

3.3.2 Motivation Elements

Motivation elements form an independent aspect that models the intent and purpose of the architectural design, and can be placed in any layer. The set of Motivation elements is shown in Table 7.

Table 7: Motivation Elements

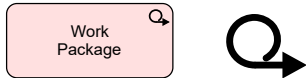
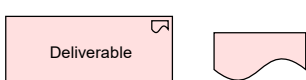
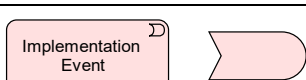
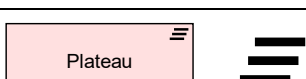
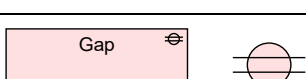
Element	Definition	Notation
Stakeholder	The role of an individual, team, or organization (or classes thereof) represents their interests in the effects of the architecture.	
Driver	An external or internal condition that motivates an organization to define its goals and implement the changes necessary to achieve them.	
Assessment	The result of an analysis of the state of affairs of the enterprise with respect to some driver.	
Goal	A high-level statement of intent, direction, or desired end state for an organization and its stakeholders.	

Outcome	An end result, effect, or consequence of a certain state of affairs.	
Principle	A statement of intent defining a general property that applies to any system in a certain context in the architecture.	
Requirement	A statement of need defining a property that applies to a specific system as described by the architecture.	
Constraint	A limitation on aspects of the architecture, its implementation process, or its realization.	
Meaning	The knowledge or expertise present in, or the interpretation given to, a concept in a particular context.	
Value	The relative worth, utility, or importance of a concept.	

3.3.3 Implementation and Migration Layer

The Implementation and Migration elements support the implementation and migration of architectures. This includes modeling implementation programs and projects to support program, portfolio, and project management. It also includes support for migration planning. This includes representations of Work Package, Deliverable, Plateau, and Gap, as shown in Table 8.

Table 8: Implementation and Migration Elements

Element	Definition	Notation
Work Package	A series of actions identified and designed to achieve specific results within specified time and resource constraints.	
Deliverable	A precisely defined result of a work package.	
Implementation Event	A state change related to implementation or migration.	
Plateau	A relatively stable state of the architecture that exists during a limited period of time.	
Gap	A statement of difference between two plateaux.	

3.3.4 Composite Elements

Composite elements consist of other concepts, possibly from multiple aspects or layers of language. Grouping and location are generic composite elements. The elements¹³ shown in Table 9 can be used to annotate any layer.

Table 9: Composite Elements

Element	Definition	Notation
Location	A conceptual or physical place or position where concepts are located (e.g., structure elements) or performed (e.g., behavior elements).	
Grouping	An element that aggregates or composes concepts that belong together based on some common characteristic.	

3.4 ArchiMate Language Customization

The ArchiMate Specification offers several mechanisms to customize the language to users' contextual needs.

3.4.1 User-Defined Attributes

All elements and relationships can be assigned user-defined attributes in the form of key-value pairs. Attributes can be grouped to create a “profile” that can be applied to these concepts and their specializations. While the ArchiMate Specification defines remarkably few attributes, the Security Overlay uses them extensively to annotate security-relevant information. In effect, the Security Overlay is a pre-defined profile of the type described in §14.1 of the ArchiMate Specification that “standardizes” security documentation and analysis.

Both the ArchiMate Specification and the Model Exchange File Format support basic property types (string, integer, real, boolean, currency, date, time, url, etc.).¹⁴ In order to support compound properties, the Security Overlay occasionally uses text strings in JavaScript™ Object Notation (JSON)¹⁵ format.

3.4.2 Specializations and “Stereotypes”

The ArchiMate Specification offers two methods of extending the vocabulary with user-defined elements and relationships:

- **“In-model”** specialization uses the specialization relationship introduced in Section 3.2.1 to show that one element is a particular kind of another (i.e., a subtype); e.g., a “privileged user” might be modeled as a specialization of a “user”

¹³ **Note:** ArchiMate tools generally support additional visual elements to annotate diagrams (e.g., comment boxes, visual groupings) that are not part of the formal specification.

¹⁴ In practice, many tools do not support data types, treating all values as text.

¹⁵ JavaScript Object Notation is an open standard data interchange format that uses character text to store and transmit data objects consisting of key–value pairs (and other serializable records). It is a commonly used data format with diverse uses in data interchange, including that of web applications with servers.

Specialization echoes UML’s “generalization” relationship that similarly conveys a progressive refinement of definition (parent to child) coupled with inheritance.

- “**Out-of-model**” specialization is denoted by double chevrons in a type-qualified name (resembling a UML stereotype)

It is used to introduce a new concept without refining an existing one. Because the specialization relationship is not used, this form implies no inheritance and is constrained only by the grammatical rules of the base type. Unlike in-model specialization, this form can be applied to relationships as well as elements. To differentiate this kind of specialization from the previous one, this document occasionally use “<<stereotype>>” as shorthand for out-of-model specialization.

The Security Overlay adopts another UML convention when using inheritance: an abstract base class (i.e., one that cannot be instantiated without further specialization) is denoted with its name in *Italics*. Again, in order to avoid dependency on visual formats, this is also expressed formally using an “*isAbstract*” property.

Figure 2 shows these conventions in use. An abstract Business Actor, *Staff Member*, is explicitly specialized into Developer and Lead Developer. Specialization implies that both concrete subtypes inherit *Staff Member*’s properties and that Lead Developer inherits the assignment to the Development process.

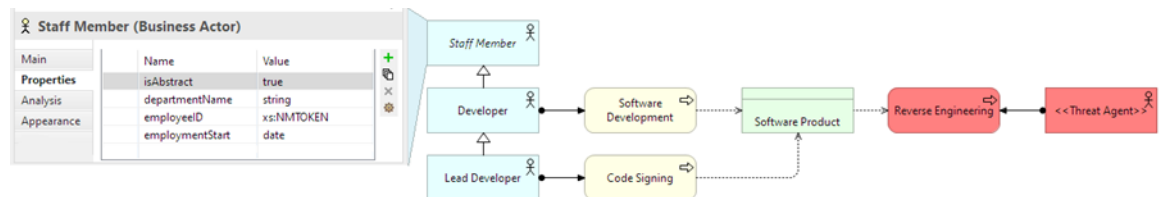


Figure 2: In-Model versus Out-of-Model Specialization

Conversely, Threat Agent is modeled using <<stereotype>> notation, emphasizing that it is somewhat untypical of “a *business entity capable of performing behavior*”. It is not derived from another Actor via inheritance. Although it is bound by the same grammatical rules as its base class (e.g., it can be assigned to a role or a business behavior), the model is emphasizing a distinction from the native concept of “business entity”. The use of a stereotype enshrines the absence of a conventional business relationship and the malevolent intent into the element definition (using something more than color), and exempts Threat Agent from any attribute profile that may be defined for Business Actors.

3.4.3 Overloaded Relationships

The ArchiMate language uses relationship overloading: the same symbol represents different but semantically compatible concepts in different contexts. For example, assignment is used both to bind Actors to Roles, and to deploy Artifacts onto Nodes. Realization depicts both the implementation of services and the persistence of logical data on physical storage. Overloading trades minimization of the vocabulary for a minor loss of precision. The Security Overlay will further overload relationships in keeping with this general principle.

3.5 The ArchiMate Full Framework

The ArchiMate Full Framework, as shown in Figure 3 arranges the layers (rows) and aspects (columns) described previously to show the overall structure of the language. Generally speaking, elements in any layer only connect to elements in the same layer or those of same aspect in the layers directly above or below.^{16 17}

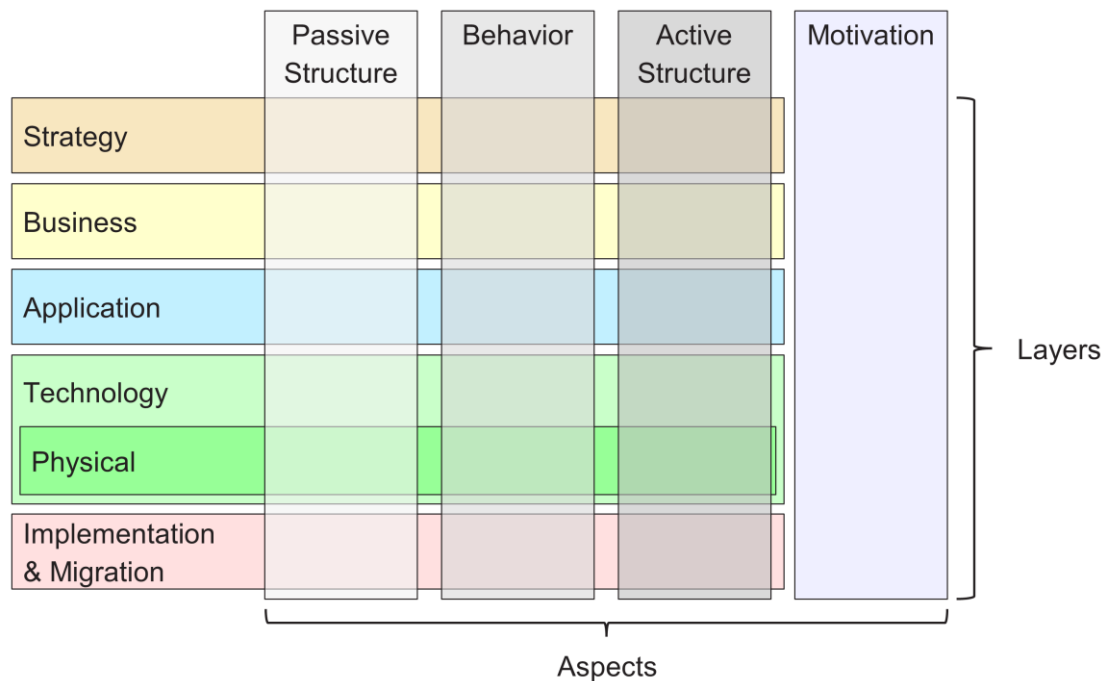


Figure 3: The ArchiMate Full Framework

To complete this introduction to the ArchiMate language,, Figure 4 shows the mapping of the ArchiMate Specification to the TOGAF Architecture Development Method (ADM). Since the mapping between the TOGAF ADM and SABSA has also been defined [W117; G152], any mapping between the ArchiMate Specification and SABSA must be consistent with this alignment.

¹⁶ This statement refers to best practice when creating the underlying ArchiMate model. In diagrams, visual shortcuts are possible and valid when implemented through the use of derived relations.

¹⁷ Motivational elements being the notable exception: they may connect with elements of any layer or aspect.

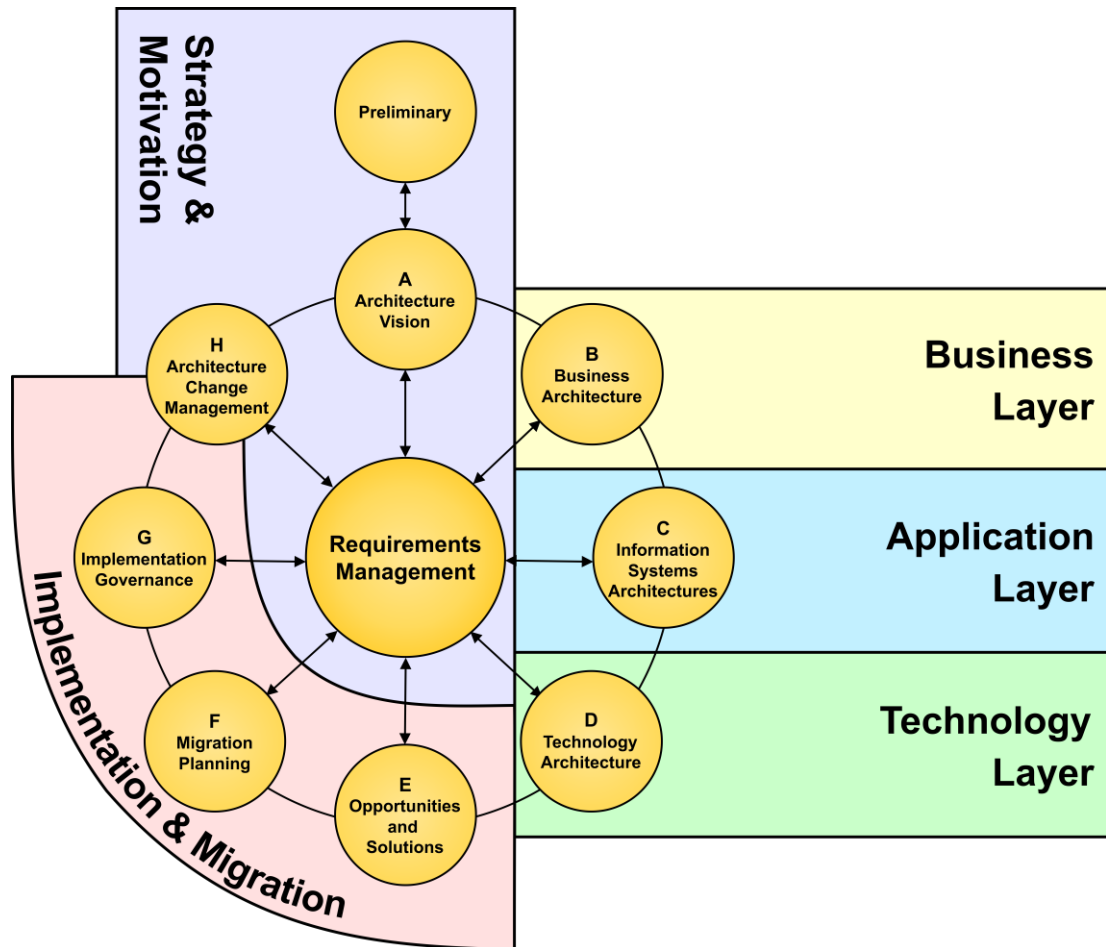


Figure 4: Correspondence between the ArchiMate Language and the TOGAF ADM

4 Aligning the SABSA and ArchiMate Frameworks

4.1 Introduction to the Security Overlay

The rest of this document describes the Security Overlay for the ArchiMate language. It focuses on extending the architectural layers and Motivation aspect to create a model-based framework for SABSA. It uses the inbuilt customization capabilities of the ArchiMate Specification to define a Security Overlay: a set of security profiles for existing ArchiMate elements, supplemented by a set of new stereotyped elements, to express security concepts.

This section first addresses the ArchiMate Motivation aspect and then works through the layers of the SABSA Matrix, top-down, left to right, referencing elements from the corresponding layers of the ArchiMate framework.

Each element is presented in terms of its purpose, design rationale, description, and examples of intended usage. The documentation here is intended to be informative rather than normative.

For a more formal definition of the Security Overlay, an accompanying XML[®] Schema is provided via the MSA Working Group¹⁸ page on The SABSA Institute website.

¹⁸ Refer to: <https://sabsa.org/category/working-groups/modelling-sabsa-archimate>.

4.2 An Overview of the Task

Figure 5 illustrates, from a high-level perspective, the task of aligning the SABSA and ArchiMate frameworks.

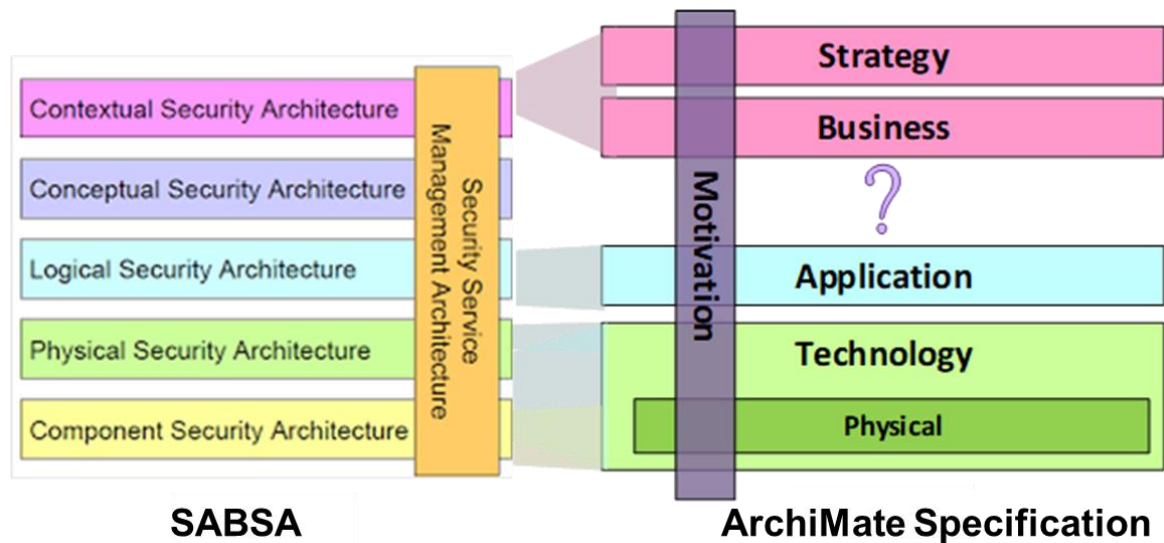


Figure 5: Aligning the SABSA and ArchiMate Frameworks

As a first approximation, and allowing for different interpretations of the label “Physical”, the layers of the SABSA and ArchiMate frameworks broadly correspond, with the exception of an equivalent of SABSA’s Conceptual Security Architecture in which to model security concepts (except for its intersection with the Motivation (Why) column). The aim of this document is to find practical solutions that address this gap within the constraints of the ArchiMate Specification and with an eye on tool capabilities.

A more subtle deficiency is that, even though the ArchiMate core layers have SABSA equivalents, the elements in these layers lack a profile that defines their “essential” security attributes to adequately protect a “Business Object”. For example, it is necessary to know its data classification, or to adequately authenticate a User acting in a “Business Role” requires an indication of privileged or non-privileged access. Much of this initiative is concerned with identifying appropriate attributes for security profiles.

The ArchiMate Physical elements that model material world entities (equipment, materials, facilities, distribution networks) align with SABSA’s Component layer.

Although SABSA's Security Service Management architecture has no equivalent layer in the ArchiMate Specification, it can be expressed in practical models. Wierda [Wierda, 2021] recommended to partition the Architectural Description into three planes:

1. A primary architecture showing how the Enterprise Architecture supports business processes at run-time
2. A secondary management plane that shows how the primary Enterprise Architecture is created, deployed, and operated
3. A tertiary plane that deals with ownership, administration and governance

SABSA Management processes can be modeled using orthodox notation in the secondary architecture with governance structures (Collaboration) and responsibilities (Responsible/Accountable/Consulted/Informed (RACI) roles) modeled in the tertiary architecture.

4.3 Risk & Security Modeling in the ArchiMate Specification

Although the ArchiMate Specification provides no special provision for security, neither does it exclude it from the “*many concerns*” of Enterprise Architecture. So, although the specification of the language does not address security in its formal definition, the specification does contain several “informative” examples with a security theme:

- Information domains are cited as an appropriate use of the Grouping element, described as “*a set of users, their information objects and a security policy*” (§4.5.1)
- Driver is defined as “an external or internal condition that motivates an organization to define its goals and implement the changes necessary to achieve them”

Security concerns are cited as an example (§6.2.2). Note that Driver expresses both positive and negative motivation, mirroring SABSA's view of risk as an enabler (exploiting opportunity) as well as a threat to be countered (mitigation).¹⁹

- In illustrating the capacity of the ArchiMate Specification for language extension through specialization (§14.2), several examples of specialized elements are proposed for a Risk & Security Overlay:
 - **Assessment** is specialized into Risk and Vulnerability
 - **Business Event** is specialized into Threat Event and Loss Event
 - **Business Actor** is specialized into Threat Agent
 - **Goal** is specialized into Control Objective
 - **Principle** is specialized into Business Policy
 - **Requirement** is specialized into Control Measure
 - **Grouping** is specialized into Risk Domain

¹⁹ Note: The terminology in this section compares that of SABSA and the ArchiMate Specification. It differs in places from that of the Open FAIR Standard which for example, defines risk as resulting solely in loss ([C20B], §4.2). The modeling of the Open FAIR taxonomy will be discussed in Section 8.2.

- Example 36 of the ArchiMate Specification (reproduced here as Figure 6) illustrates the use of specialization to derive Control Measures from Threat & Loss Events, Vulnerability & Risk Assessments, and Control Objectives

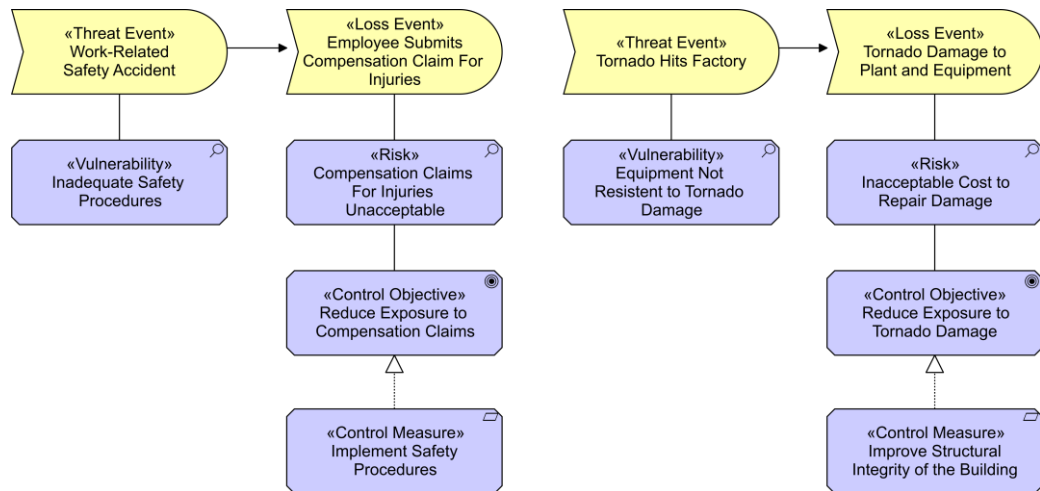


Figure 6: The Risk-Modeling Example in the ArchiMate 3.2 Specification

The ArchiMate Specification examples are intended to be informative rather than normative or complete: they illustrate only how the language *could* be used rather than how it *should* be used. Nevertheless, they provide insight into how the authors of the ArchiMate Specification envisaged how risk and security concerns could be expressed.

They confirm that security modeling should stem from the specialization of Motivation elements by providing clear guidance on four elements (Assessment, Principle, Goal, and Requirement) and a recommended use for one other (Driver). The semantics of these elements are a natural fit for their application to security. It also proposes Grouping for Trust Domains, and specialized Events for the materialization of threats and losses.

Security events may occur in all core layers and so should not be limited to the specialization of Business Event. Application Layer events (suitable for modeling input injection threats, time-outs), and Technical layer events (malware detection, equipment malfunction, SIEM events, backup failure, etc.) are also possible.

Example 36 uses a non-directional association between Assessment and Event to convey the ideas that a Threat Event can exploit a Vulnerability and a Risk can materialize into a Loss Event. In the direction Assessment → Event, the specification permits only the association relationship. In the opposite direction (Event → Assessment), it allows both association and influence. Let us examine these options.

The history of events is a relevant factor in making an Assessment, so depicting an Event influencing an Assessment can make sense if that is what is intended; e.g., in the context of modeling a threat or risk analysis process even if it is not the choice in this example.

What is shown is an unnamed bi-directional association. How should we interpret this? In the direction Event to Assessment, we could say that a Threat Agent/Event “exploits” a Vulnerability; a Loss Event *materializes* a Risk. In the opposite direction (Assessment to Event), it seems intuitive to read the association as simply binding the analysis with the object of that analysis.

Apart from the ambiguity, all these interpretations carry an inherent directionality and, in the absence of a better bi-directional interpretation, could be improved by using the directed association relationship. Thus, the interpretation (Threat Agent/Event <<exploits>> Vulnerability; Loss Event <<materializes>> Risk), is expressed explicitly in Figure 7.

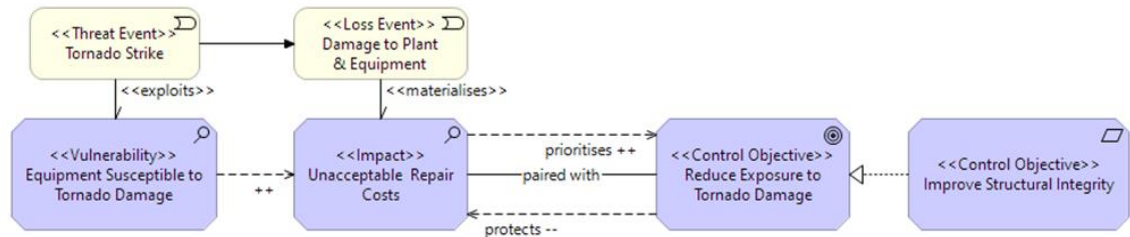


Figure 7: Risk Modeling Relationships

Figure 6 also uses a non-directional association between Goal (Control Objective) and Assessment (Risk). Although the specification allows an influence relationship in either direction, the association relationship feels like a natural fit to express a correlation between an Assessment and a Control Objective that model analysis may need to navigate in either direction. The influence relationship could be used to express more nuance; for example, discovering a new security flaw would raise both the criticality and prioritization of the associated Control Objective, and deployment of mitigating controls would correspondingly reduce exposure to the vulnerability. These augmentations have also been redrawn in Figure 7.

Turning to the representation of Control Objective and Control Measure, both modeled as specialized Motivational elements – Goal and Requirement, respectively. While the first of these (Goal – Control Objective) is intuitive, the latter (Requirement – Control Measure) is worthy of comment. By using Requirement as the base element, a Control Measure is being defined here as a requirement for a control rather than the implementation of a control that the term <<Control Measure>> might suggest. We might ponder what type of element might represent the control implementation – something concrete and deployable that provides effective protection?

This issue has been addressed in The Open Group White Paper: *How to Model Enterprise Risk Management and Security with the ArchiMate® Language* [W172], leading to the expanded example, reproduced in Figure 8.

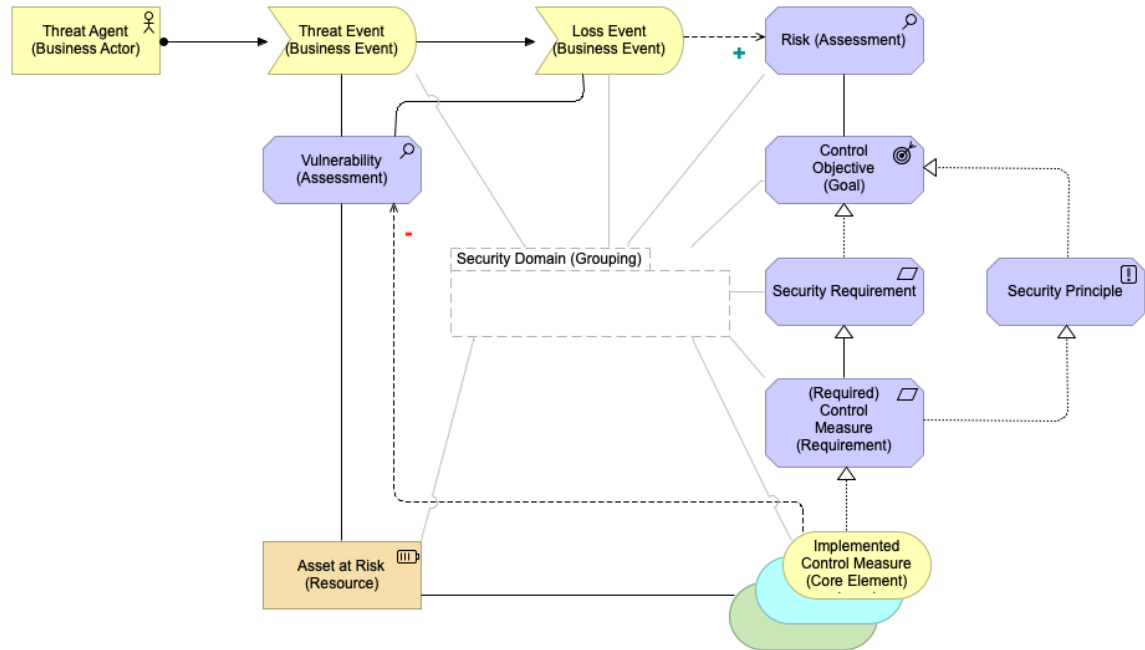


Figure 8: Mapping of Risk and Security Elements

Here, a Security Requirement is specialized by a Control Measure (no longer stereotyped), which is then realized by a control implemented by core Services. Though the diagram is non-normative, the definition of Control Requirement [W172, p25] describes the general case: “A *control requirement is realized by core entities used for mitigation, treatment, and control*”.

Although the ArchiMate Specification lacks a specific way to marking an entity as a “Control Implementation”, they become de facto Controls through the realization of Control Measures. This topic is discussed in depth in Section 5.10.

Both references confirm the use of a realization chain from Control Objective to implementation. This relationship is structural (strong) and a good semantic fit where a single Control Measure suffices. Complications arise where multiple measures (or a pattern of elements) are necessary to achieve the required objective.

The default semantics of realization imply that each measure is an OR’ed alternative (e.g., multiple software implementations of the same service definition). Security realizations often need AND semantics: a set of requirements, all of which are necessary to achieve an objective of end-to-end protection or defense in depth. The specification allows two other relationships: influence and association. Let us consider whether either of these would be a better fit.

Association is the ArchiMate Specification’s “weakest relationship”, used for linking elements without any precise meaning beyond “an unspecified relationship or one that is not represented by another ArchiMate relationship”. If we accept that realization is a good semantic fit, there is no need to use the “association relationship”. If not, it would be better to specialize realization (strong and inherently directional) than use a form of association.

Influence, though stronger than association, is much weaker than realization. It implies that implementing a set of measures will improve the attainment of the Control Objective rather than delivering it assuredly. Control Measures and Control Objectives can be in conflict, undermining

some Objectives while reinforcing others positively. The influence relationship is the only one capable of expressing positive and negative effects and therefore merits a place.

Realization, then, is the best choice for the structural relationship between Control Objectives, Control Requirements, and actual Control Measures, with influence reserved for “cross-coupling”. A means of expressing AND/OR semantics is discussed in Section 5.6.

Finally, the specification suggests Driver as the appropriate element to model security concerns but offers no example of how this should be related to other elements. For guidance, the ArchiMate Motivation metamodel is reproduced in Figure 9, where Driver (as a form of the abstract Motivation element) may associate with, influence or be influenced by any other Motivation element.

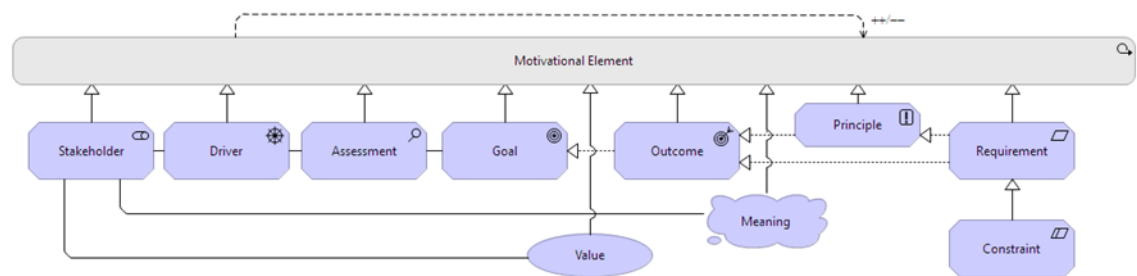


Figure 9: ArchiMate Motivation Metamodel

4.4 The Basic Element and Relationships

The Security Overlay defines basic properties that can be applied to every element and relationship type. They are described here, but to avoid repetition, they are not necessarily referenced in every place where they would typically appear. They include:

- An **identifier**: Used to provide a globally unique reference for the element within the model
- An **isAbstract** flag, set to false by default, indicating that an element cannot be instantiated; it must have a non-abstract specialization (visually, an abstract element name is displayed in *Italics*)
- **Stereotype/stereotypeOf** used to declare out-of-model specialization structurally, rather than having to parse the structure of the (<<stereotype>>) name format
- **Cardinality** indicators at the source and target ends of relationships are expressed in UML notation; i.e., n (exactly n), n..m (range n to m), * (any) (the meaning of the cardinality varies with context)

5 The Motivation Aspect

Building on the previous section, Figure 10 shows the full Security Overlay for the Motivation metamodel that formalizes the non-normative examples into a specification that better supports the security perspective.

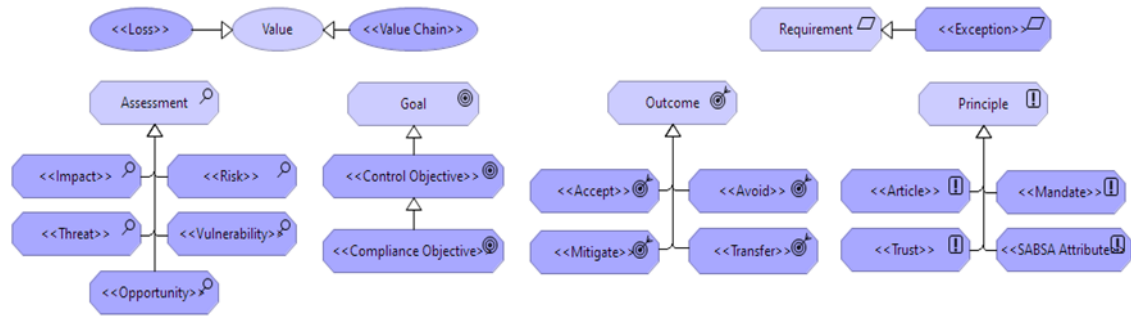


Figure 10: Security Enhanced Motivation Metamodel

This section describes how these elements can be used to express security motivations: the identification and valuation of assets, analysis of threats and vulnerabilities, evaluation and treatment of risks, and the identification of Control Objectives and Control Requirements.

5.1 Value & Loss

The “What” column of the SABSA Architecture Matrix [SABSA R101] acknowledges the business value of assets. Business Value may be expressed in several forms (financial, legal, brand, social, economic, health & safety) in combination or with others. This evolution brings it closer to the ArchiMate Specification, which has no element to represent an abstract Asset but instead offers the ability to associate a **Value** element with any model element.

Models containing many Values will encounter the issue of providing sufficient meaningful and unique names. This can be overcome through adoption of a naming convention that reflects the associated Element name. Figure 11(a) shows the simple association of a Value with a generic element, thereby marking it as an asset.

To indicate the Stakeholder to whom the Value is important, a tertiary relationship (Figure 11(b) lower) has been found to be better than the in-line association (Figure 11(b) upper). This pattern avoids complications that arise when a Value is appreciated by several Stakeholders or associated with multiple assets. The in-line form leads to ambiguity and unintended coupling in the underlying model even when not explicitly drawn as such, as in Figure 11(c).

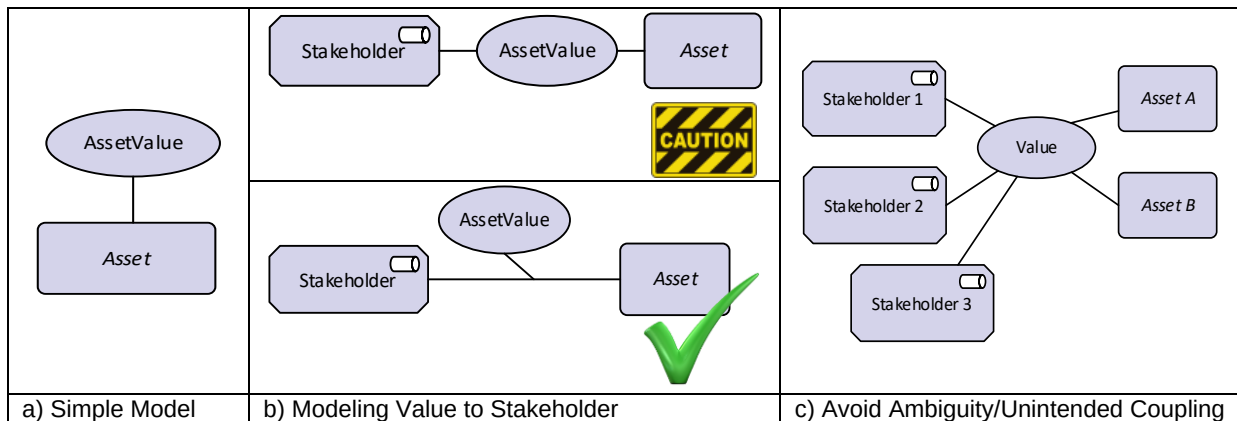


Figure 11: Modeling Assets using Value

Because security is concerned with enhancing value and protecting Assets, the Value element (and a “Loss” specialization introduced by the Security Overlay) provide a basis for modeling risk.²⁰ A good textual description of the nature and scale of Value and Loss is essential for a complete model. Enabling automated analysis of Value and Loss requires capturing key characteristics of these elements in a structured way. This can be achieved using the custom profile proposed in Table 10. Financial values, to support quantitative analyses, are expressed as a three-point range consisting of a lower and upper bound (typically the 90% confidence interval) and a mode (most likely) value.

Table 10: Proposed Value Property Overlay

Element	Properties	Schema
Value	The properties of the Value and Loss elements include the following key-value pairs: valueType classifies the asset in terms of a set of enumerated values: FINANCIAL, REGULATORY, REPUTATION - The set of valueTypes can be drawn from a standardized set or customized as required. currencyUnits and currencyCode describe the scale and denomination, respectively rating supports a qualitative scale (e.g., from VLOW to VHIGH) for non-financial valuations valuation denotes the value, either as a single point value or a confidence interval	
<<Loss>>		

²⁰ More on this topic in Section 7.2.

5.2 Value Chain

The Security Overlay also specializes Value into <<Value Chain>>, providing a specific profile for the Strategy elements, Capability and Value Stream. This is discussed briefly here and further in Section 6.1.

The key measures of meta-processes are rooted in cost-accounting: production cost, sales price, and margin. The organization seeks to identify which activities generate value and which are inefficient or uneconomic. The security perspective is concerned with enhancing and protecting the value of the value generators while containing the costs of the inefficient process. The ValueChain stereotype models the breakdown of costs and is, therefore, principally a financial measure. The Security Overlay can be used to map the composition of a Value Chain to any behavioral element in the Enterprise Architecture model, as shown in Figure 12.

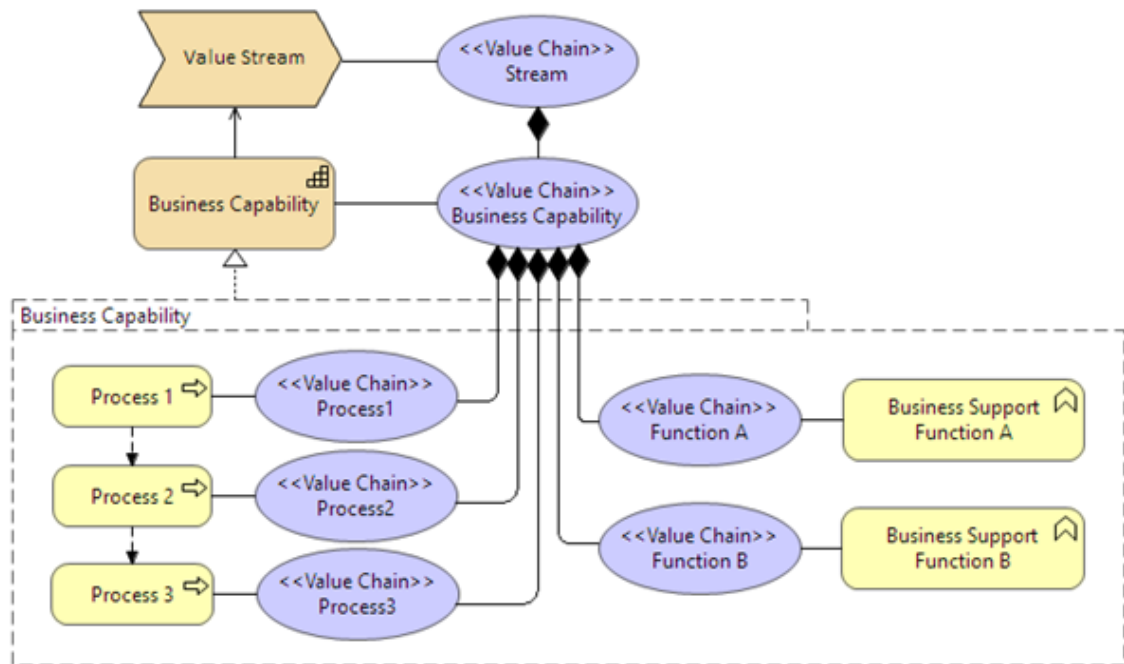
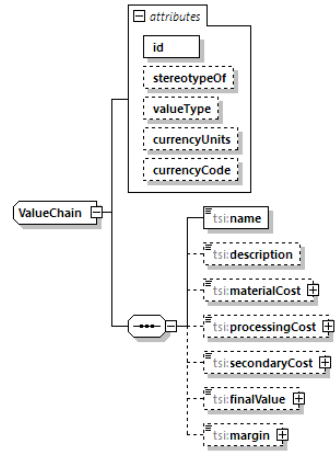


Figure 12: Composition of Value Chains

The attributes proposed for the Value Chain profile enable the margin to be calculated as the sum of its constituent sub-chains. The ArchiMate Specification allows the Value element to be decomposed into compositions and aggregations of the same type. Composition seems the safer option here because, unlike aggregation, it should prevent cost accounting figures from being counted multiple times.

Table 11: Value Chain Properties

Element	Properties	Schema
	The Value Chain element extends the Value element with properties to express value in currency units. The definition supports the expression of production costs (input materials, process handling, and consumption of support services) and the manifest worth in the final value. The margin can also be expressed as a derived value, calculated as the difference between total costs and the final delivered value.	

5.3 SABSA Business Attributes²¹

Business Attributes represent the essential qualities of the Stakeholders’ ideal system, to be promoted, protected, and enhanced in the Target Architecture if the enterprise is to deliver its strategy. They are atomic, universal Singletons: conveying an indivisible characteristic that can be understood regardless of context. Business Attributes are essential to the SABSA approach but have no equivalent concept in the ArchiMate Specification.

On close examination however, the definition of the ArchiMate Principle element as “*an intended property of a system ... a general property that applies to any system in a certain context ... motivated by some goal or driver*” is semantically highly compatible for representing SABSA Business Attributes in the Security Overlay. Figure 13 recreates the Business Attributes catalog as out-of-model specializations of Principle.

²¹ To disambiguate SABSA Business Atttributes from ArchiMate profile atttributes, the former will be capitalized.

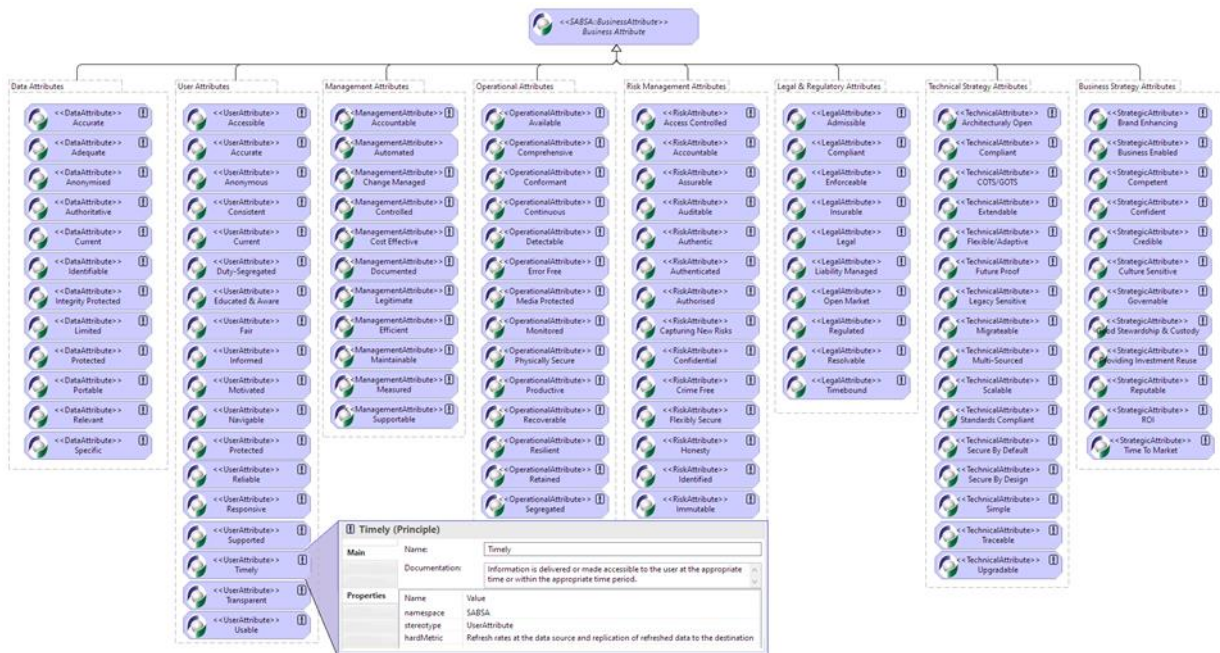


Figure 13: SABSA Business Attributes Represented in the ArchiMate Language

A few points to note about the Business Attributes representation shown in Figure 13:

1. Business Attribute is modeled as a specialization of Principle, distinguishing it from conventional uses (such as “Cloud First”) and enabling it to define a distinct security profile
2. The taxonomy is structured as an abstract base *Business Attribute*, specialized into domains using Groupings

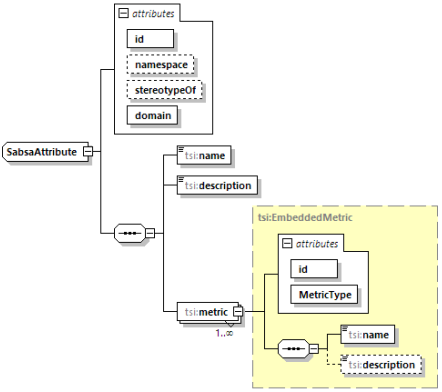
This approach offers benefits in practical modeling and various analysis scenarios where:

- Domains act as namespaces that support the construction of qualified names enabling Attributes to be overloaded with an appropriate definition and metrics for different contexts²²
- Grouped attributes promote consistency and correctness in modeling; e.g., Stakeholders with legal concerns should only be offered associations with Legal and Regulatory attributes
- It enables Attributes with the same non-qualified name to appear on the same diagram

The SABSA Attribute has been modeled with a profile that reflects that published in *Enterprise Security Architecture: A Business-Driven Approach* [Sherwood et al., 2005].

²² For example, Accountable is defined in both Management (assignment of accountability) and Operational (being held to account for actions taken) contexts.

Table 12: SABSA Attribute Properties

Element	Properties	Schema
	The SABSA Attribute defines profile attributes that express: - a name: an adjective by which it is commonly recognized - a domain: a grouping that qualifies the name by acting a specific namespace - a description defining of the attribute - a sequence of metrics, each with a name and description and categorized as either hard or soft	

5.3.1 Structural Placement of Business Attributes

In the SABSA method, business-layer Attributes are refined and resolved at lower layers into contributing Attributes with sub-system scope and ultimately addressed by Control Objectives (**Goals**) achieved through the realization of Control Requirements and Constraints.

A dictionary definition of “principle” is: “*a fundamental, primary, or general law or truth from which others are derived*”.²³ They are more abstract than policy statements, goals, and objectives and intended to provide focus; e.g., the principles of “equality”, “diversity”, and “inclusion” might be considered the primary principles of an organizations’ Human Resources (HR) policy.

In the hierarchy of Figure 14, principles are placed at the apex, supported by policies, control objectives, and requirements in lower layers.

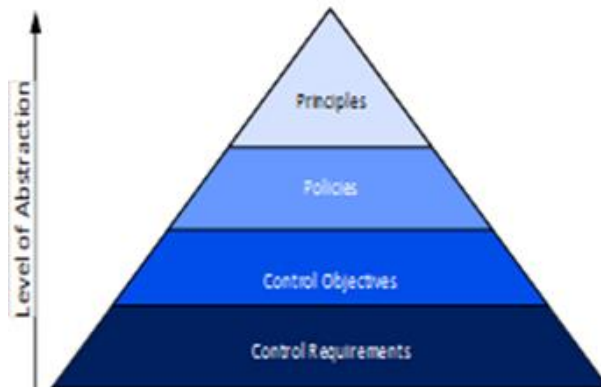


Figure 14: Hierarchy of Abstraction

²³ Refer to: www.dictionary.com/browse/principle.

This concept of a principle is not, however, universal: the original 2002 COSO Enterprise Risk Management framework²⁴ placed “Principles” between Objectives and Controls and this structure has been adopted by many affiliated organizations such as SWIFT (see Figure 15).

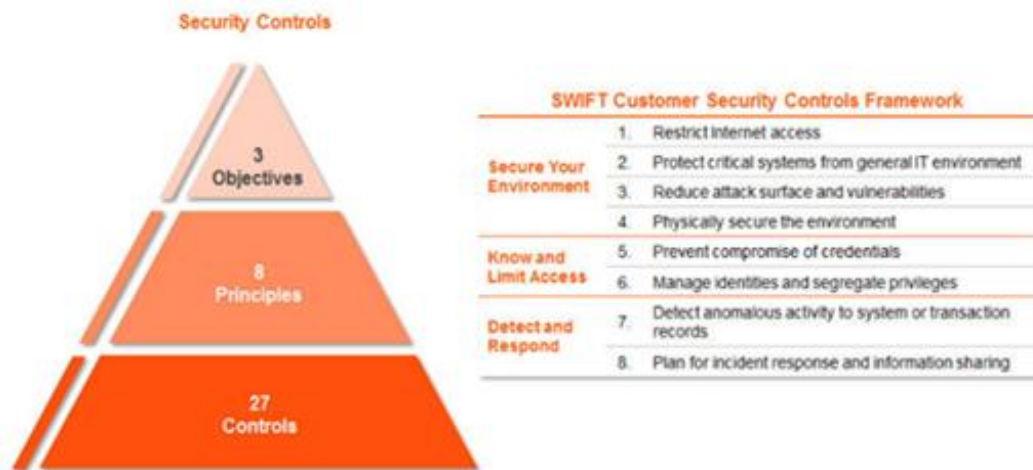


Figure 15: The SWIFT/COSO Model

The ArchiMate Specification follows the COSO paradigm;²⁵ i.e., Principles are used as maxims to guide Designers *toward* Outcomes and Goals (Figure 16). SABSA Attributes align with the dictionary definition, as aspirational ideals toward which the architecture should align through the achievement of costed, concrete, contextualized goals, and requirements, but which may never be fully attained.

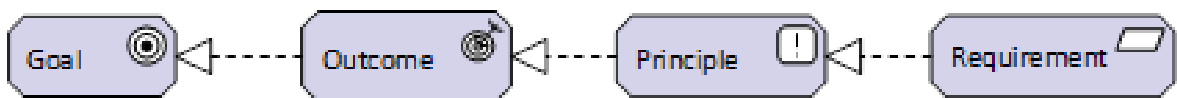
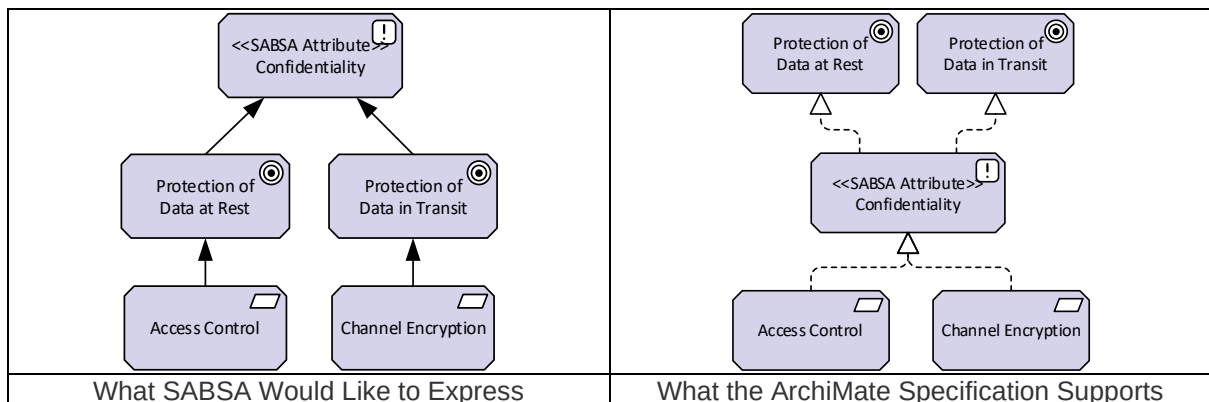


Figure 16: Principle in the ArchiMate Motivation Hierarchy

This presents a few problems for SABSA’s intended usage. Take the example shown in Figure 17.



²⁴ Refer to the Committee of Sponsoring Organizations: <https://www.coso.org/enterprise-risk-management>.

²⁵ Note the example from the ArchiMate Specification which proposes the Principle element to represent policy.

Figure 17: Highlighting the Control Hierarchy Mismatch (i)

The SABSA view (left) expresses the Attribute (*Confidentiality*) as being satisfied by specific Goals (“*Protection of Data at Rest*” and “*Protection of Data at in Transit*”) at various points in the model. These are to be achieved by distinct, verifiable Requirements (“*Access Control*” and “*Channel Encryption*”), respectively.

The ArchiMate metamodel (right) places *Confidentiality* in the center of the structure and, by doing so, forces the realization paths to merge. The resulting model suggests “*Protection of Data at Rest*” might be realized through “*Channel Encryption*” and “*Protection of Data in Transit*” through “*Access Control*”.

The result is both unintentional and incorrect. It is also difficult to disentangle because, even if the structures are *drawn* as distinct views, they remain merged in the underlying model, causing a problem for automated analysis which sees both paths as legitimate and is consequently unable to resolve the modeler’s intent.

A workaround might be parallel stacks (Figure 18), each with its own instance of “*Confidentiality*”. Note that these need to be distinct elements (with distinct names) to avoid the entanglement of the previous proposal.

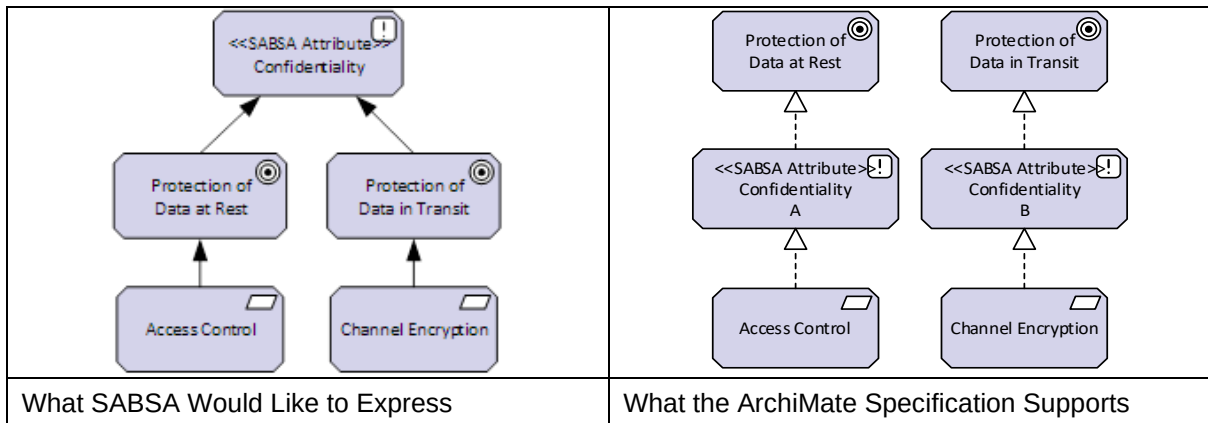


Figure 18: Highlighting the Control Hierarchy Mismatch (ii)

Although this approach achieves the required segregation and is entirely within the specification, creating distinct *Confidentiality* elements for each stack is inelegant. Conceptually, Attributes are singletons, so any duplication creates redundancy and maintenance issues. If a good model is meant to reflect reality, this is a poor style: the concept of different “*flavors*” of confidentiality is not a natural way to model the real world.

A better solution (Figure 19) eschews the standard structure, but remains legal and reflects the original intent.

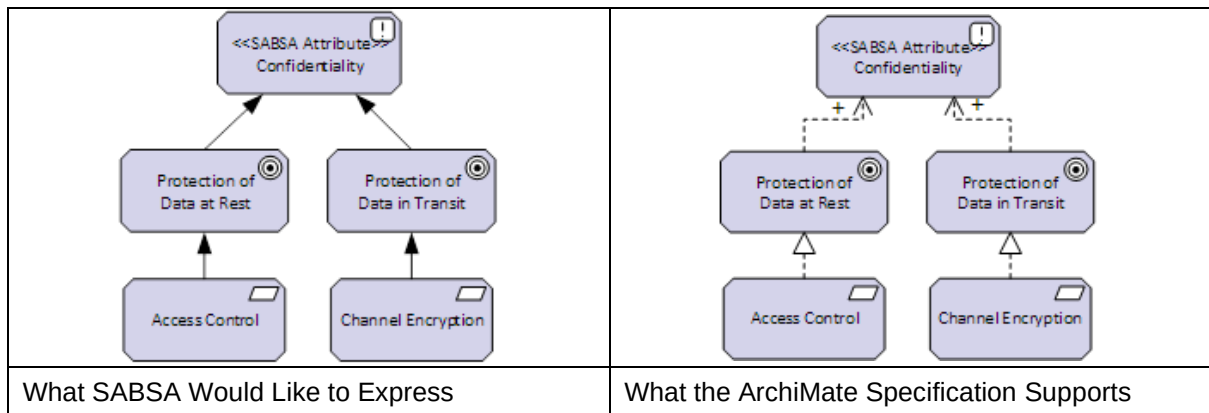


Figure 19: Achieving the Desired Hierarchy

Via a convention in which SABSA Attributes are only ever **influenced** by Control Objectives, the desired structure can be achieved within legal grammar. This choice aligns with the ArchiMate definition of **influence** as “a traceable motivational path”, where the “motivation element is achieved to a certain degree”. That is to say, SABSA Attributes are *strengthened* by the achievement of Control Objectives rather than attained absolutely.

5.3.2 Traceability of Business Attributes

Another essential requirement of SABSA Attribute modeling is the ability to trace their refinement through the architectural layers, also implemented using influence relationships, as shown in Figure 20. Element reuse can be an issue when creating moderately-sized Attribute hierarchies.

Consider a situation where “Compliant” is a desired SABSA Attribute that occurs twice in the model: once to indicate General Data Protection Regulation (GDPR) compliance on personal data in the Business Layer, and again in the context of Federal Information Processing Standards (FIPS) compliance for a Hardware Security Module (HSM) in the Technology layer.

The Modeler encounters the following problems:

- Because Attributes are singletons, they should not appear more than once in the same view
- Using the Attribute once, associating it to both targets, is both possible and “legal”, but:
 - It is more difficult to find definitions and metrics that are compatible with multiple contexts
 - The reused Attribute becomes a hub of merged relationships, warping the structure to such an extent that it can become a “circular” hierarchy
- Artificial “flavors” of global, abstract concepts such as Attributes are an inelegant design practice

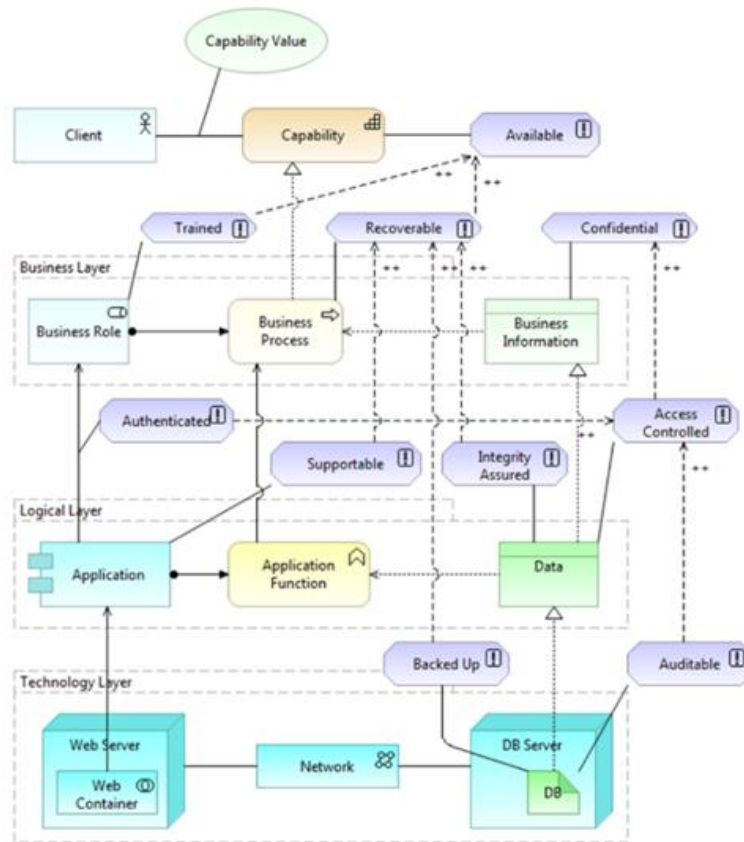


Figure 20: Attribute Traceability Across Layers

Because SABSA Attributes are global singletons, reuse within a model runs the risk of unintended coupling. In cases where an Attribute’s definition is compatible, and reuse does not warp the layer hierarchy, it is possible to accommodate different metrics using an externalized “Meaning” element (discussed later in Section 5.4).

In all other cases, the best solution is to use distinct instances from the Attribute taxonomy (Figure 13). In this example, this would mean using the “Compliant” Attribute from the Technical Strategy category for use with the HSM, and then one from the “Legal & Regulatory” category for GDPR. As distinct elements (same short name, different qualified names), both Attributes can legitimately appear on the same diagram without creating the hub/warping effects associated with what we might refer to as a “Singularity”: a black hole at the heart of our model.

The influence relationship conveys the same sense of intangibility that justified its use with Control Objectives. There is a subtle difference, however. Attributes are pure abstract singletons in a taxonomy that is reused across models, projects, and enterprises. When *directly* connected to each other by “concrete” relationships in a model, they also become connected in the abstract – universally and forever.

It is perhaps an esoteric point, but it is important to note that in reality, SABSA Attributes are just abstract concepts with no intrinsic correlation. The relationship only arises as an “echo” of a real correlation in the context of the specific real-world system being modeled.

5.4 Meaning

The ArchiMate Meaning element is not extended by the Security Overlay other than by the addition of an optional profile. The element plays a useful role in representing concepts such as Identity and in the application/adaptation of SABSA Attributes to multiple contexts.

As described in the previous section, the SABSA Attribute taxonomy incorporates a catalog of elements, definitions and hard/soft metrics via the security profile. These attributes can and should be adapted for each new context, because Business Attributes are modeled as singletons, any definition and metric, modeled within the profile, will apply to everything that the Attribute touches; i.e., in every context where the Attribute is referenced.

As we have seen, this causes problems where an Attribute is applied to multiple points in the same model. Consider, for example, whether there is a single *Confidentiality* Attribute enhanced by the two Control Objectives and four Controls of Figure 21. How can we express context-sensitive definitions and metrics?

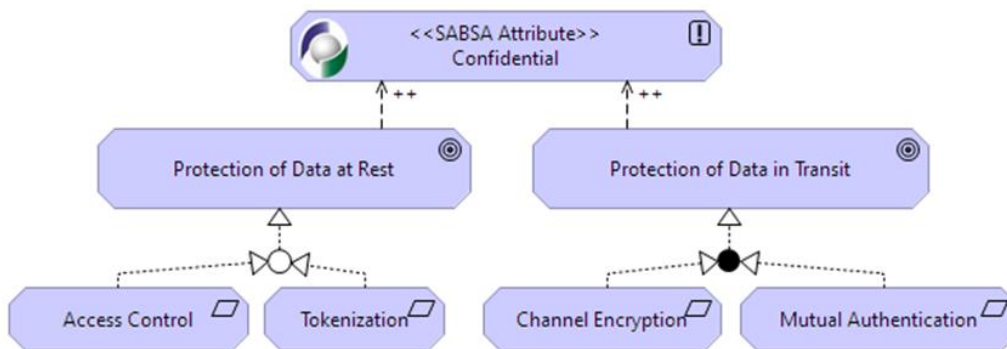


Figure 21: Applying Attribute Metrics to Multiple Controls

The Security Overlay recommends the use of **Meaning** elements to re-interpret the Attribute for each context. Figure 22 shows how the meaning of *Confidentiality* (i.e., its definition and metrics) can be externalized and then adapted to *Tokenization*, *Access Control*, and the other requirements using Meaning elements.

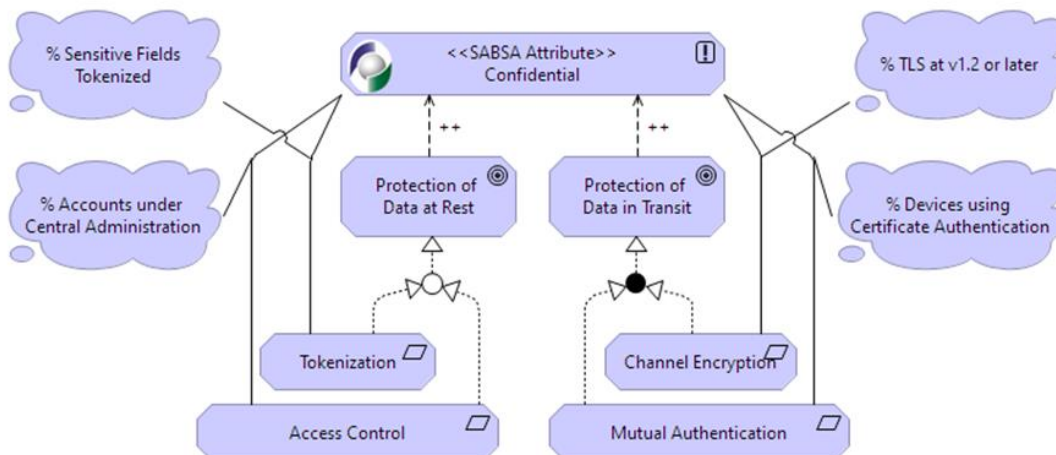


Figure 22: Use of Meaning to Externalize Context-Sensitive Metrics

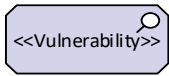
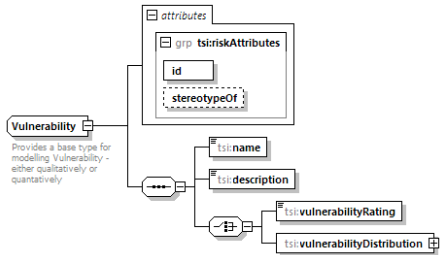
5.5 Impact, Threat, Vulnerability, and Risk

Guided by the ArchiMate Specification (see Section 4.3), impacts, threats, vulnerabilities, and risks are modeled as stereotyped **Assessment** elements, providing a good semantic fit that reflects the subjective uncertainty inherent in assessing these unknowns.

Neither SABSA nor the majority of Information Security Management System (ISMS) standards prescribe a specific risk assessment methodology. The Security Overlay attempts to accommodate any approach commonly practiced: qualitative or quantitative, by providing support for methods based on enumerated labels (e.g., HIGH, MEDIUM, LOW) and numerical calculation (e.g., Bayesian methods or Monte Carlo simulation). Usage will be discussed in Section “Risk Management & Strategy”. The property profiles of the Security Overlay elements are shown in Table 13.

Table 13: Risk Element Properties

Element	Properties	Schema
	Risk expresses annual loss expectancy as a qualitative rating or a quantitative probability distribution: currencyCode & currencyUnits qualify the scale and value of the expected loss	
	Impact expresses impact (loss magnitude) as a qualitative rating or a quantitative probability distribution: currencyCode & currencyUnits qualify the scale and value of the expected loss	
	Threat expresses vulnerability as a qualitative rating or a quantitative probability distribution.	

Element	Properties	Schema
	Vulnerability expresses vulnerability as a qualitative rating or a quantitative probability distribution.	

5.6 Controls: Objectives, Requirements, and Measures

This section models the SABSA’s Conceptual/Motivation cell that deals with Control Objectives. As usual, the starting point is the ArchiMate Specification, discussed in Section 4.3, in which Goal is specialized as Control Objective, and Requirement is specialized as Control Measure. We have already encountered some semantic questions about this model. The Security Overlay retains the <<Control Objective>> stereotype of Goal, retains Requirement for security requirements without specialization, and introduces a new <<Control>> stereotype to represent the control mechanism, discussion of which is deferred to Section 5.10.

Modeling a Control Objective realized by a control mechanism via a requirement is a good semantic fit. In practice, though, a single Requirement is often insufficient. Figure 23 uses the ArchiMate AND and OR Junctions to express that the “*Protection of Data at Rest*” objective can be achieved through “*Access Control*” **or** “*Tokenization*” and that the “*Protection of Data in Transit*” objective requires both “*Channel Encryption*” **and** “*Mutual Authentication*”.

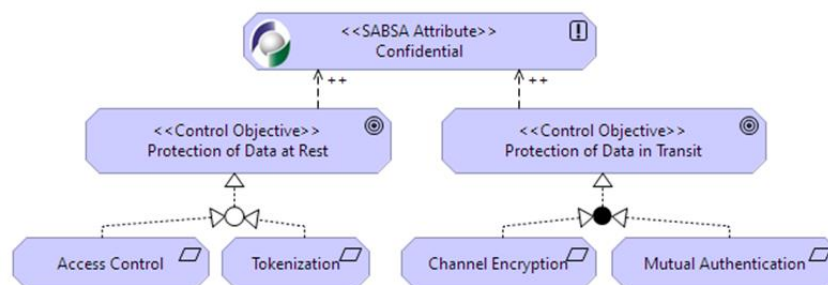


Figure 23: Expressing Composite Requirements

Sometimes a requirement is realized through a combination of elements and relationships, acting in concert. In such cases, the realization of the requirement can be expressed as a pattern using a grouping marked as a control stereotype. As an example, Figure 24 proposes a way to model the requirement that Asset Owners should perform an annual user access rights recertification.

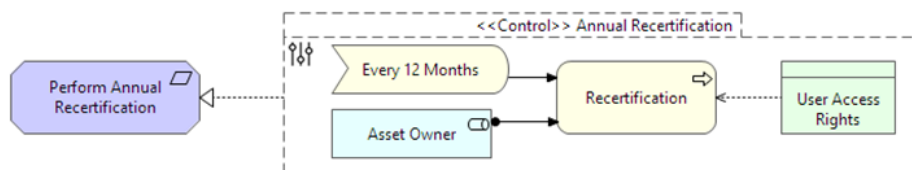


Figure 24: Example of a Control Pattern

5.7 Multi-Tiered Security

SABSA’s Conceptual/Process cell discusses the combination and layering of Controls for “Defense-in-Depth” or “End-to-End Protection”. The Blue Book [Sherwood et al., 2005] describes a 4-tier model: Prevent, Contain, Detect & Notification, and Recovery & Restoration. The NIST™ Cybersecurity Framework [NIST CSF] identifies six control functions: Govern, Identify, Protect, Detect, Respond, and Recover. The Security Overlay adds the category “Deter” to cover sanctions expressed in policies & contractual clauses. The *Identify* set (including Asset Management, Business Environment, Governance, Risk Assessment, and Risk Management Strategy) are modeled in the secondary and tertiary architectures rather than the primary, run-time architecture. The profile for the Control elements has been derived from analysis and use of several commonly used control frameworks. The resulting schema is shown in Table 14.

Table 14: Control Element Properties

Element	Properties	Schema
<<Control Objective>> Requirement Constraint	The Control set (Objectives, Requirements and Constraints) define a similar set of properties. The namespace and refCode properties provide a means to link the element to an external reference document such as a remediation plan. The status property is used to track the element’s lifecycle using enumerated types: MANDATORY, ADVISORY, WITHDRAWN, and PENDING. The applicability property can be used to indicate the element’s scope: COMPONENT, SYSTEM, ORGANIZATION, or GLOBAL. The baseline property is used to indicate the element’s level in the baselining tiers defined by some standards and maturity models; e.g., STANDARD, ENHANCED, or ASSURED. The protection profile expresses a control strength for each layer of the multi-tiered defense.	

Figure 25 shows how this works in practice. Profile attributes are declared in the Control Objective element to define its protection requirement and in each Requirement element to describe its protection capability.

We see, in this example, that the Control Objective “*Prevent Unauthorized Modification*” defines an assurance requirement of “*Moderate*” across all tiers. While there is no single Requirement that can satisfy this profile in full, the aggregate set of Requirements meet and exceed the profile: an Acceptable Use Policy that sanctions unauthorized access satisfies the “*deter*” aspect, Tamper Detection mechanisms delivers the “*detect*” aspect, Regular Back-Up, fulfilled by a Business Continuity Plan (BCP) infrastructure service, provides the “*recover*” capability, etc.

By standardizing this approach within the Security Overlay, an analysis tool could verify that the protection profile required by the Control Objective is met or exceeded by the set of Requirements proposed to realize it.

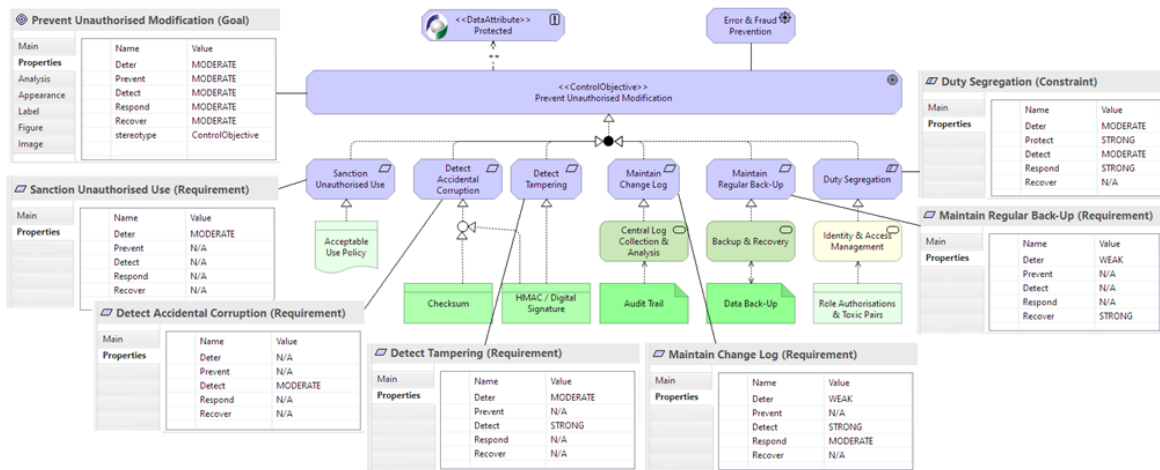


Figure 25: Example of Multi-Tiered Security

Compliance

Risk analysis is far from being the only source of controls. As the digital economy becomes increasingly critical to society's functioning and prosperity, organizations are being subjected to regulatory demands that are becoming ever more numerous, extensive, rigorous, frequent, and held to ever-higher levels of scrutiny.

The "Why" cells of Contextual and Conceptual cells address this important and demanding driver. The Security Overlay has been extended with additional stereotypes to support the representation of Compliance Objectives and guidance on managing the complexity created by the challenges of simultaneous multi-regulatory compliance.

Having been used to model several regulations, legislation, and industry or sector standards against which organizations must certify, the Security Overlay provides the following elements that enable these external references to be incorporated into the model as reusable architectural reference modules.

5.8 Regulations and Standards

Compliance controls are defined in public standards. The level of adherence demanded spans a broad spectrum: some are mandatory carrying the force of law, some are pre-requisite for a license to operate, while others may be important in demonstrating good governance, fulfilling market expectations or measuring capability maturity. As a container element to model these publications, the Security Overlay offers two stereotypes of Representation:

- Regulation: for compliance obligations that are enforced by law
- Standard: for compliance obligations that are driven by commercial or technical considerations

Both regulations and standards tend to follow a regular structure. Although substantial standards, such as NIST SP1500 (Big Data) [NIST SP 1500], consist of several volumes (modeled as a composite <<Standard>>), it is more typical for the document to be a single document partitioned into chapters or sections devoted to a particular theme. These top-level sub-divisions are modeled as regular Groupings, as shown in the examples of ISO/IEC 27001 [ISO/IEC 27001] and NIST SP800-53 [NIST SP 800-53] in Figure 26.

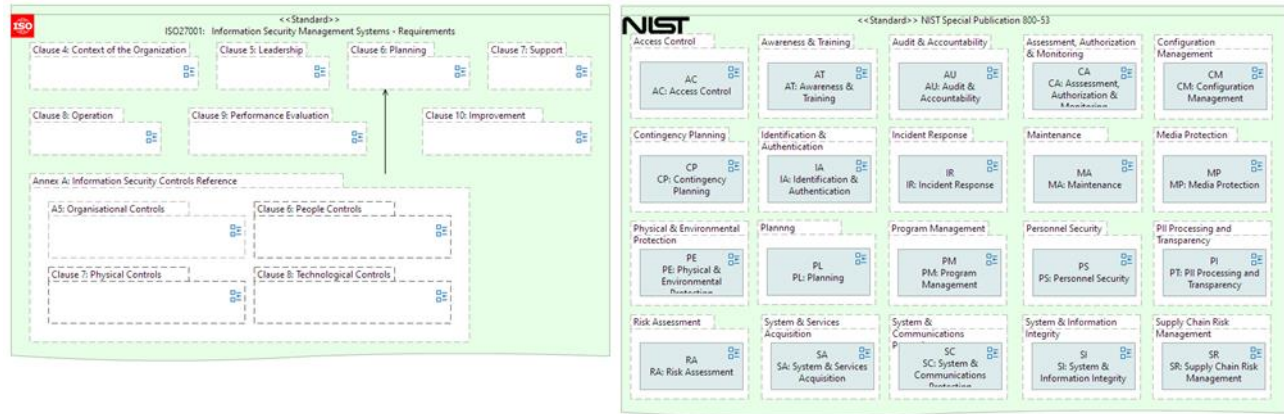


Figure 26: The Structure of Standards and Regulations

The schema definitions of Standard and Regulation are given in Table 15.

Table 15: Standard and Regulation Properties

Element	Properties	Schema
	Standard and Regulation are both modeled as specializations of Representation and define a similar security profile. The namespace, publisher, and sourceURL attributes provide a means to link the element to an external reference document. version and publishDate provide support for version control, while expiryDate can be used to show that the standard has been superseded or withdrawn. Regulation extends Standard with one additional attribute, effectiveDate, enabling legislation to be modeled and used ahead of the date it comes into legal force.	

5.9 Articles, Mandates, and Compliance Objectives

Legal texts tend to consist of articles (schedules), paragraphs, and clauses, numbered for cross-reference. Articles serve multiple purposes; for example, they can be used to define terminology, to assert scope and applicability, to cite earlier mandates that empower the issuing body with authority to act in the given domain. Only after considerable preamble do the Articles generally

turn to address what needs to be accomplished. Articles consist of paragraphs and sub-paragraphs that are eventually built from short, well-focused clauses.

To model this in ArchiMate language, the Security Overlay draws an equivalence between legal paragraphs and Compliance Objectives (an analog of the risk-based Control Objective) and between legal clauses and Requirements/Constraints. This structure fits reasonably well as clauses tend to be succinct and focused. By definition, adherence to the clauses is necessary and sufficient to satisfy the objective of the paragraph.

The question remains of how to model the articles themselves. As with SABSA Attributes, we need to represent an abstract concept, the law, an ideal to be pursued in letter and in spirit by meeting specific, achievable goals. Again, the Security Overlay turns to a stereotype of Principle, <<Article>>: and the convention of only referencing it through influence relationships.

In practical terms, legal articles affecting IT architecture are imported using the <<Article>> stereotype; the documentation field is used to fully reproduce the article text. <<Compliance Objective>> elements are then used to translate the formal style of legal paragraphs into a plain language goal description that is more suitable to a technical audience. Requirements, Constraints, and <<Exception>> (a stereotype of Requirement that de-scopes the stated objective) are then used to spell out fine-grained, actionable statements, again in plain language, that mirror the legal clauses. Figure 27 shows how this has been implemented for Article 7 of the GDPR [GDPR].

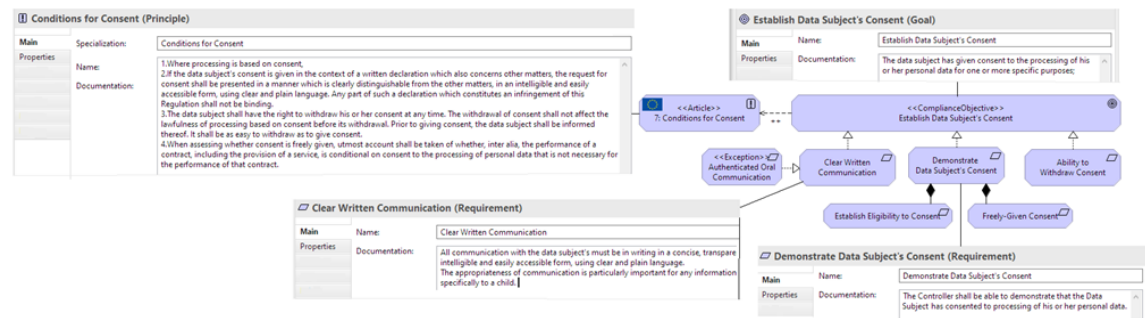


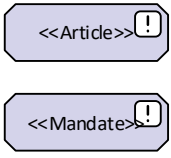
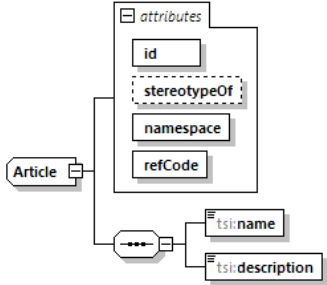
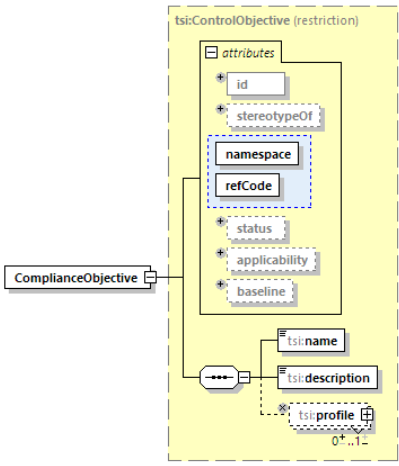
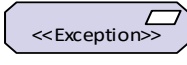
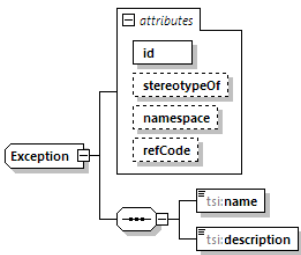
Figure 27: Articles and Compliance Objectives

This same three-tier pattern (Concept—Goal—Requirement) is also evident in sectoral and technical standards. While the stereotypes already identified are re-usable at the Goal and Requirement levels, neither the <<SABSA Attribute>> nor <<Article>> are intuitive names for the top-level concepts found in these standards.

Consensus on naming in the standards is absent: they are variously referred to as security principles, overarching objectives, strategic goals, or capabilities – all of which clash with ArchiMate vocabulary. The term currently adopted in the Security Overlay is <<Mandate>> (also used by UCF [UCF] to abstract over similar control objectives defined in disparate standards), but the final choice is still a work-in-progress.

The stereotypes Article and Mandate are, broadly speaking, synonyms with a minor tuning of the schema to reflect the more stringent regulatory context, as shown in Table 16.

Table 16: Compliance Conceptual Element Properties

Element	Properties	Schema
	Article and Mandate are declared as stereotypes of Principle and define attributes that express: The namespace and refCode properties provide a means to reference the Article/Mandate in an external reference document declared in the enclosing Regulation/Standard. The description reproduces the original text in full. The elements are synonyms of each other with no difference in the schema definition.	
	Compliance Objective is a restricted extension of Control Objective in which the namespace and refCode attributes are required. Compliance Objectives are generally stronger than Control Objectives. Being derived from regulatory or standards compliance, they tend to be mandatory and non-negotiable. By contrast, Control Objectives can be derived from risk analysis or maturity models and therefore governed by internal strategy, policy, and project timescales with more flexibility in prioritization, scheduling, and reporting. The description translates sections of the original text, often expressed in formal, legalistic style, into plain language for the intended audience.	
	Exception provides a visual means to de-scope a Compliance Objective, Requirement or Constraint; Without such an element, caveats and exceptions in the detailed text might be overlooked. The namespace and refCode attributes are optional.	

Models of commonly used regulations and standards are highly re-usable. We hope and intend that ArchiMate models of these references will be developed and then shared among the community. Please keep watch on The SABSA Institute website for such contributions, or better still, if you are the first to model a particular reference, please consider sharing it via the MSA Forum on The SABSA Institute website.²⁶

²⁶ Refer to: <https://sabsa.org/modelling-sabsa-with-archimate>.

5.10 Control Mechanisms

The previous sections have identified a need for three fundamental elements to model a control architecture:

- An abstract concept of an ideal that is being pursued, such as a SABSA Attribute, a statute of regulatory (Article) or standards (Mandate) compliance, modeled as a stereotype of Principle
- A specific, realizable, defined state of a system component that represents a sufficient realization of the ideal if achieved, maintained, and evidenced

These are modeled as Control Objectives and Compliance Objectives, stereotypes of Goal. The architecture will aim for this state by a given date. The emphasis is on bringing a risk within appetite or demonstrating compliance with the law at an acceptable cost and schedule. Objectives are not aiming for the achievement of perfection.

- Control Requirements, Constraints, and Exceptions specifying granular actions and restrictions that, when performed in the right way, in the right place, at the right time, and for the right cost, achieve the overall objective

Section 5.6 observed that Motivational elements are only capable of expressing intent. A complete model needs a means of representing the concrete mechanisms that implement the requirements in the Target Architecture. Although these by definition cannot be Motivational elements, the document deals with them here for completeness of the topic.

Services are often cited as the means of realizing security requirements and work well in many use cases; e.g., Identity & Access Management, Vulnerability Management, Security Incident Event Management (SIEM). In practice, however, there are also many controls for which a service-element implementation feels contrived. Figure 28 confirms that real-world Controls manifest in many forms: nominated accountability, a policy clause, a process design, a logical control, or a physical device.

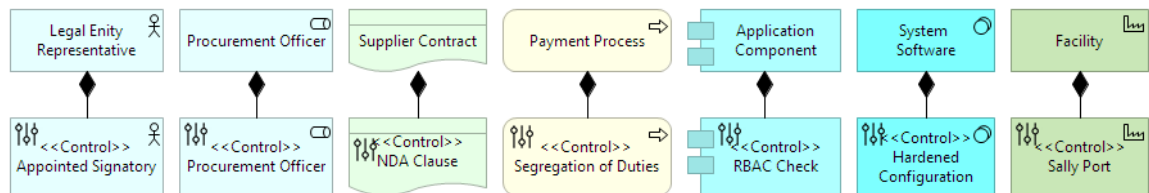
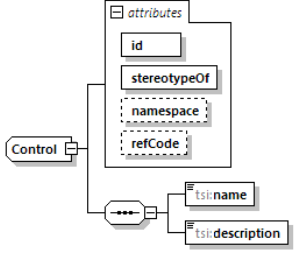


Figure 28: Use Cases and Iconography for Control

What can be concluded from this set of examples is:

- Controls are often a component of an element of the same type
- Control stereotypes cannot be formed from a single base type but require multiple base types

Table 17: Control Properties

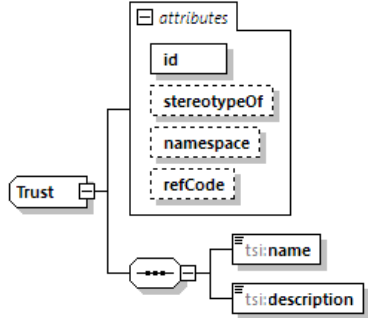
Element	Properties	Schema
	Control is unique in the Security Overlay in that it can extend any base type. It is used to emphasize a standalone element or a component of a composite element that addresses a specific requirement. Its profile expresses: An optional source and refCode link it to an external document Though Controls are often a component of an element of the same type, no rule requires this to be necessarily so.	

5.11 Trust

The final Motivational element introduced by the Security Overlay is Trust, another stereotype of Principle. The element enables the explicit modeling of any trust implicit in an interaction between Parties.

The definition is given here, but discussion of its usage is deferred until Section 6.4.1, “Governance”.

Table 18: Trust Profile

Element	Properties	Schema
	Trust enables the explicit modeling of any trust implicit in an interaction between two Parties. Its attributes express: An optional source and refCode link it to an external document	

6 Modeling the Contextual Security Architecture

The latest edition of the SABSA framework is defined in the 2018 revision of the SABSA Matrices [SABSA R101]. Rows corresponding to Contextual Architecture artifacts and activities are reproduced in Table 19.

Table 19: SABSA Contextual Architecture

		Assets (What)	Motivation (Why)	Process (How)	People (Who)	Location (Where)	Time (When)
Contextual	Artifacts	Business Goals & Decisions	Business Risk	Business Meta-Processes	Business Governance	Business Geography	Business Time Dependence
		Business Value; Business Asset Taxonomy	Opportunities & Threats Inventory	Business Value Chain; Business Capabilities, Services & Functions	Operational Structure & the Extended Enterprise	Inventory of Buildings, Sites, Territories, Jurisdictions, etc.	Time Dependencies of Business Goals & Value Creation
	Activities	Business Driver Development	Business Risk Analysis	Capability Management	Relationship Management	Supply Chain Management	Performance Management
		Business Benchmarking; Identification of New Drivers	Analysis of Internal & External Risk Factors	Managing Processes & Capabilities for Stakeholder Value	Managing Suppliers, Service Providers, Customers & Employees	Supply & Demand Management; Deployment & Consumption	Define & Monitor Business-Driven Performance Targets

This outline enables us to visualize the set of processes and activities necessary to create and maintain an Architectural Description of the Contextual Architecture, expressed in the ArchiMate Specification in Figure 29.

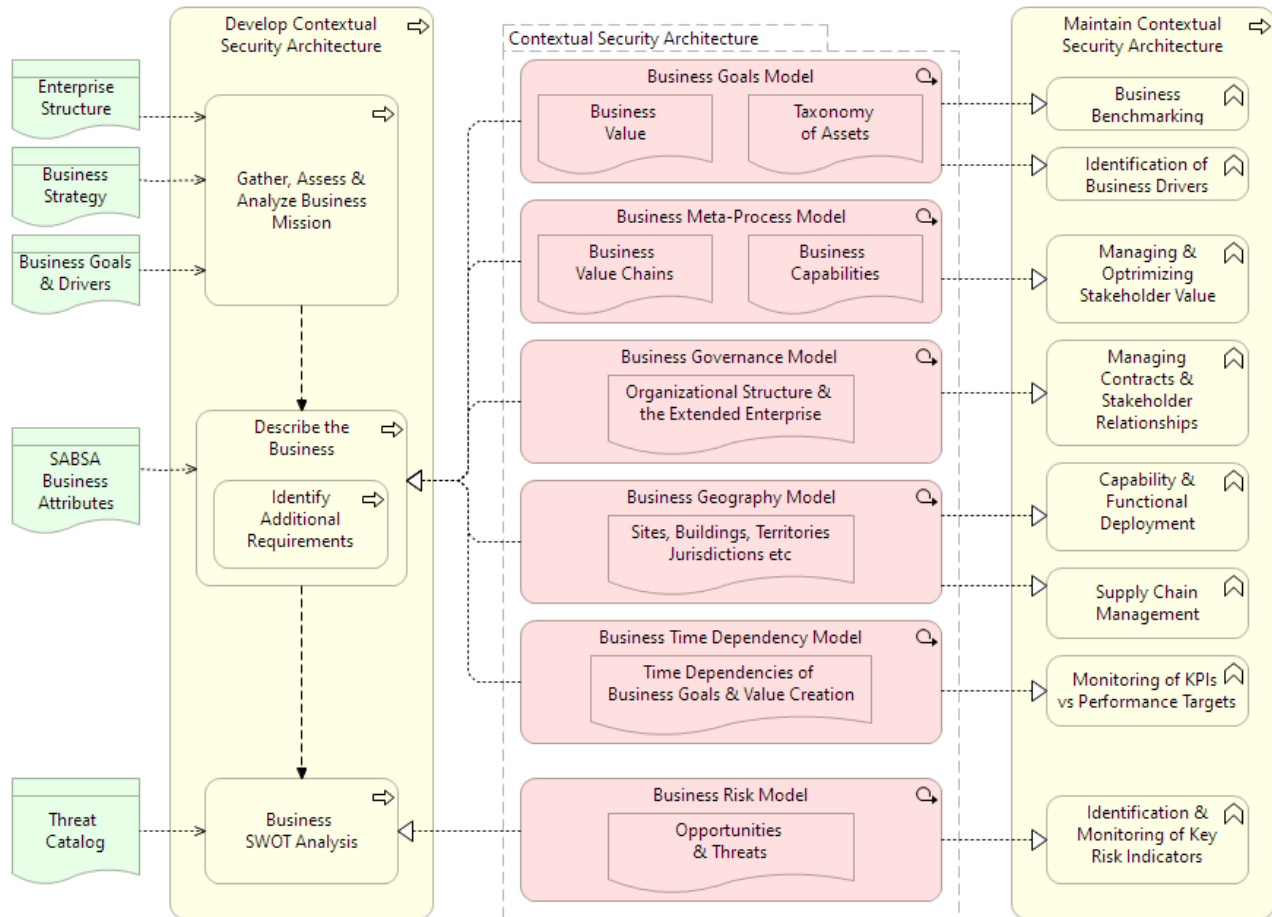
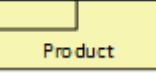
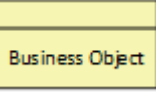


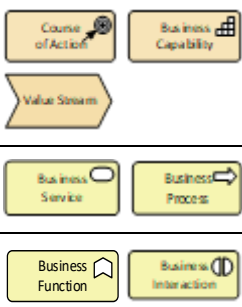
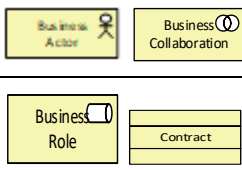
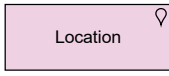
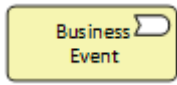
Figure 29: Developing and Maintaining the Contextual Security Architecture

This section discusses how elements from the ArchiMate Specification’s Strategy and Business Layers can be customized to express these Contextual artifacts as views of an ArchiMate model.

Table 20 summarizes the native ArchiMate elements that are available to model the contextual layer.

Table 20: Contextual Elements

	SABSA Matrix	ArchiMate Element	Description
Contextual Layer	Asset (What)	Resource 	The ArchiMate Specification does not offer a generic Asset element. As discussed in the previous section, Assets are denoted by associating a Value element to represent its value, worth, utility, or general importance to the organization. Business Value is typically associated with the external appreciation of goods, services, information, knowledge, or money, normally as part of a customer-provider relationship, but supported by internal capabilities and processes at all layers that should be traced in the model. Value is most often expressed in financial terms. Still, non-monetary value is also essential to business; e.g.,
		Product 	
		Business Object 	

	SABSA Matrix	ArchiMate Element	Description
			practical/functional value (including the right to use a resource or service, a certification or license to operate) and the value of information, knowledge, or intellectual property. Many Business and Strategic elements have intrinsic value, such as Resource (Physical Assets), Capability (the ability to harness resources to produce wealth) via Value Streams, the expected Outcome of some project or initiative, Business Information (Information Assets), Products, and Services that are bound to Contracts (Commercial Assets).
	Motivation (Why)		Motivation is expressed using elements from the ArchiMate Motivation aspect across all layers.
	Process (How)		Processes and meta-processes can be expressed using behavioral elements from the Business and Strategy Value layers, respectively. The value generated by activity is most often expressed in financial, cost accounting terms. The previous section introduced the <<ValueChain>> stereotype as a means of expressing this.
	People (Who)		The active structure elements of the Business Layer are used to represent organizational structure and relationships that can be formalized into contracts, or as RACI roles.
	Location (Where)		The Location element represents a conceptual or physical place or position where concepts are located.
	Time (When)		The Business Event element denotes the moment or moments at which the event happens.

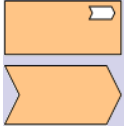
Many of these elements need to be elaborated with properties that bear on the system's security posture to perform effective security analysis. Others need to be stereotyped so that they can be assigned security properties when occurring in a security context. Achieving a consensus about the set of Element properties required to support security is a major goal of the Security Overlay initiative.

6.1 Business Assets

6.1.1 Capability and Value Stream

Value Stream is a behavioral element within the Strategy Layer. The definition and notation are shown in Table 21.

Table 21: The Value Stream Element

Element	Definition	Notation
Value Stream	A Value Stream represents a sequence of activities that create an overall result for a customer, stakeholder, or end-user. It can also be used to model a Value Chain, which according to the TOGAF definition, is a high-level view of an organization: how it works, how it delivers value, how its functions are organized within an operating model.	

A key principle of a Value Stream is that its value is always defined from the perspective of the Stakeholder, the consumer of the product, service, or deliverable, and not on its intrinsic value; i.e., the cost of production. The value of Value Stream can be modeled in the ArchiMate language using the Value element. An example of a Value Stream is provided by the ArchiMate Specification [C226] and reproduced in Figure 30.

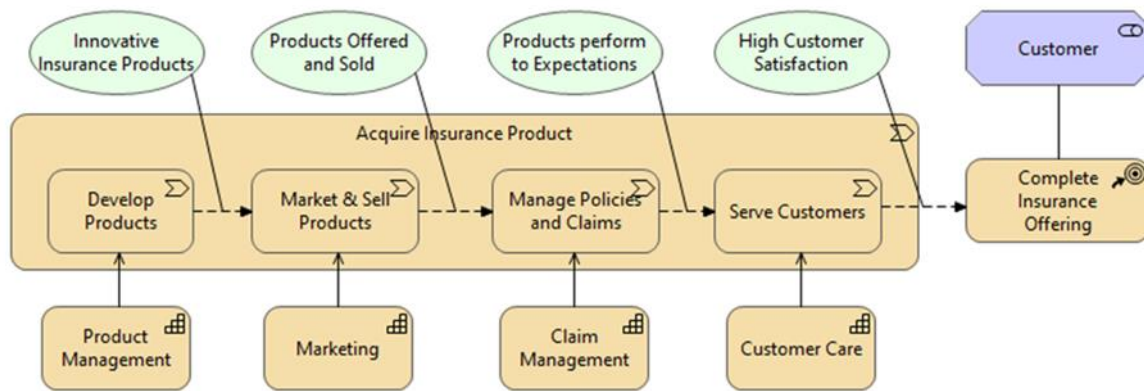


Figure 30: Value Stream Modeling

The Security Overlay proposes that the cost-accounting properties of processes and meta-processes should be externalized using a <<Value Chain>> stereotype to facilitate factoring and composition across all process elements in the Strategy and Business Layers.

6.1.2 Business Object

Business Objects represent Information Assets. Its Security Overlay properties are shown in Table 22.

Table 22: Business Object Security Properties

Elements	Properties	Schema
Business Object	The Security Overlay defines the following properties: - A confidentiality classification to declare sensitivity to unauthorized disclosure - An integrity classification to declare sensitivity to unauthorized modification - An authenticity classification as assurance of the information's authenticity/origin - A classification of privacy sensitivity in terms of Personally Identifiable Information (pii), particularly if that information pertains to a child (couldBeMinor) and the need to be maintained up to date (reviewPeriod) - Any minimum retention period required by policy or legislation Many of these properties use enumerated types that can be tailored to the organization.	

The example of Figure 31 shows the relation with SABSA Attributes: a Business Object (Medical Record) has confidentiality property of “CONFIDENTIAL” and is also tagged with the SABSA Attribute of the same name.

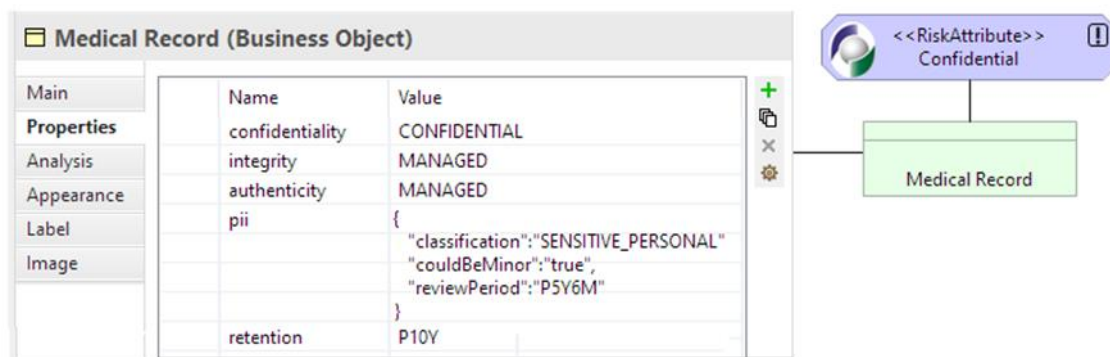


Figure 31: Element Properties and SABSA Attributes

This pattern begs the question: “Why is the confidentiality aspect of this information being modeled twice?”. Consider this scenario: classified documents have been published on the Internet. Are they still secret?

In one sense, they are – the classification endures. These are still highly sensitive documents, and their publication may cause tangible harm. In another sense, they are clearly not because they are irrevocably in the public domain. Herein lies the answer: the element property is a protective marking, assigned by the Owner and immutable over the classification lifecycle. The Attribute represents a precious and fragile status, highlighted by the Architect as a characteristic that the Enterprise Architecture must protect. The answer provides a useful insight into model validation. It would be curious if the information classified as CONFIDENTIAL was not linked to the Confidential Attribute as an outcome of the Attribute Profiling process. It would be similarly

remarkable if the Attribute were associated with information classified as PUBLIC. Modeling offers the possibility of SABSA Attributes being validated against such properties and patterns.

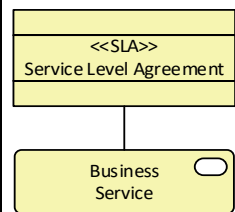
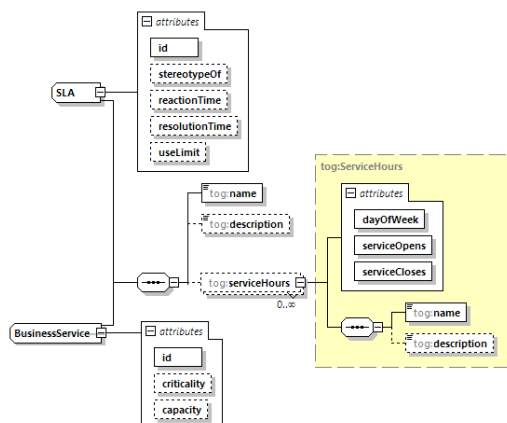
6.1.3 Business Service, Interface, and Service Level Agreements

In terms of asset analysis, the most important aspects of service elements are those committed in a Service Level Agreement (SLA)²⁷ that incurs penalties if not delivered. SLAs are enforceable contracts that set out expectations of service type, quality, and the resolution processes and remedies when these are not met. They may include a combination of technical services (applications or infrastructure), business (management) services (e.g., Support, Subscription, Accounting), data, or products.

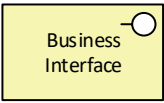
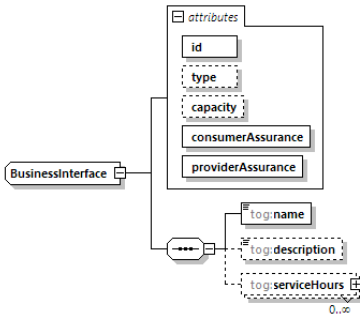
The SLA element's documentation should summarize the key contracted services, responsibilities, and service level expectations. These can then be modeled as SLA metrics using Security Overlay properties that reflect the most commonly occurring service parameters; i.e., availability, recovery time, and recovery point objectives. A bespoke Requirement/Constraint may also be used where a simple property is insufficient.

Business Layer services need a distinct property set when they are offered through human interfaces. They may include reaction and response times, service windows, or capacity limitations. Business services may be offered via several Interfaces (appointment, telephone, form submission, etc.), each of which may exhibit different delivery characteristics: cost, capacity, service hours, etc. From a security perspective, these interfaces may also offer different possibilities for the authentication of consumers: by telephone, for example, being generally less reliable than a physical meeting. Table 23 shows a proposed schema.

Table 23: Business Service Properties

Element	Properties	Schema
	Business Services are delivered via processes supported by human intervention. The SLA consolidates information on all contracted services and their delivery expectations as a combination of free text and user-defined properties. Services and SLAs exhibit similar properties from the perspective of Provider & Consumer respectively: • The service availability window • Committed reaction and response times on the handling of service requests • Limits on the number of requests submitted • The business criticality of the service • The maximum capacity of the service	

²⁷ Or, to a lesser extent, an Operational Level Agreement (OLA) between business functions in the same organization.

Element	Properties	Schema
 Business Interface	Business Services may be offered via multiple interfaces, each offering a different capacity, service window, or other quality of service parameters. The interface type is an enumeration of possible channels: TELEPHONE, MAIL, APPOINTMENT, etc. Interfaces also differ in their ability to identify or authenticate the Service Consumer and Provider: consumerAssurance & providerAssurance.	

In terms of validation, SLA metrics are strong candidates for Attributes Profiling during security analysis. Once tagged with an Attribute, model validation rules can ensure that it is supported by Control Objectives and Requirements that satisfy the protection profile. It would also be possible to detect a service that depends on other services that do not meet its minimum service levels requirements.

Interfaces and services exposed over a domain boundary merit analysis using a Trust.

6.2 Business Risk

Risk, threats, vulnerabilities, and opportunities can be modeled at the Business and Strategy level using the orthodox ArchiMate approach discussed in Section 4.3. The Security Overlay adds stereotypes of motivational layer elements for this purpose, introduced in Section 5.5.

6.3 Business Process/Function/Interaction

Just as Data Owners classify business information, business processes are often categorized in terms of their criticality to core business mission, capabilities, and value chains. Security analysis needs to be able to recognize such processes and protect them accordingly.

Behavioral elements may also be deemed sensitive due to the way they operate on information. Element properties should reflect the privilege level expected to perform the activity and those of the access relation to reflect the sensitivity of the information and the nature of the access: Behavior element properties include:

- A criticality indicator that denotes the importance of the activity to the business
- A “privilege level” indicator that denotes behavior requiring some degree of restriction, perhaps because it handles high-value information or has the potential to impact other users
- If the processing handles personal data, it should state the legal basis for such processing

and for the access relation through which behavior elements operate on Business Objects:

- The standard access modifiers (read, write, read-write & access) can be refined using a controlled vocabulary that includes publish, subscribe, delete, copy, rename, encrypt, set access profile, etc.
- Cardinality; i.e., whether the process operates on a single object or has access to multiple objects

Table 24: Business Behavior Properties

Element	Properties	Schema
Business Process Business Function Business Interaction	The business behavior elements that denote: • The criticality to the business • A “privilege” level that indicates whether the process is performed on behalf of the assigned Actor, another user, as a supervisor of other users or in general administration • The legal basis for any processing of personal data • The executionFrequency & period are used to express as a confidence range; i.e., how often the process is performed; e.g., 10 - 20 times per week In addition, Business Interaction enumerates any need for segregation of duties between the participants.	
Relationship		
..... > <..... <.....>	Modifiers for access relations include: • Read: SCAN, SEARCH, SUBSCRIBE, VERIFY • Write: CREATE, APPEND, PUBLISH; • Read/write: UPDATE, SIGN, ENCRYPT, DECRYPT; • Access: COPY, MOVE, DELETE, ERASE, RENAME, ARCHIVE, SET_READONLY, SET_ACCESS sourceCardinality expresses how many process executions may be occurring in parallel. targetCardinality indicates how many data records can be accessed per execution.	

Several validations become feasible with these properties:

- Critical behavior elements must be associated with at least one SABSA Attribute that characterizes the nature and degree of the criticality
- The privilege level of the behavior must be sufficient for:
 - Creation rights on non-repudiable information
 - Read access to confidential/personal information or modifying access to high integrity information
- Behavior elements declaring no processing of personal data should not be accessing personal data
- Behavior elements accessing personal data must declare the legal basis for processing
- Identification of sensitive processes associated with access to high volumes of sensitive information requiring high assurance security services; e.g., strong authentication

6.4 Business Roles and Actors

In the ArchiMate Specification, Actors represent human or organizational entities that can be assigned to Roles that describe:

- The extent of their responsibilities with respect to a given business process
- Their use of business and application services

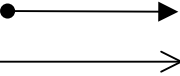
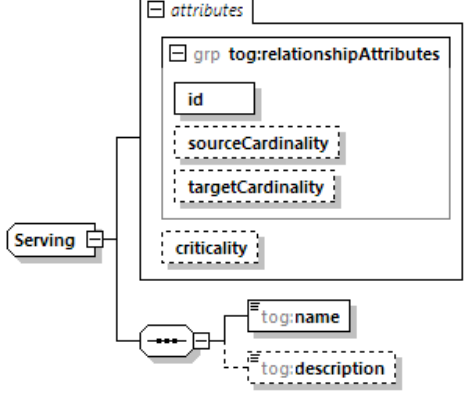
Security analysts should be able to describe the following relevant information from the model:

- Whether an Actor is human or organizational (e.g., a corporate legal entity, department, or team)²⁸
- If human, whether they qualify as a Data Subject and if so, the possibility of being a minor
- Whether any Role to which the Actor has been assigned is considered privileged
- Whether the Role is considered part of a “Toxic Pair” in a process requiring segregation of duties
- The expected number of Actors (individuals/entities) assigned to a Role (many to one)
- The expected number of service requests (normal and peak demand) generated by a constituency

²⁸ The ArchiMate Specification does not envisage a technical entity being modeled as an Actor. On one hand, it would not be appropriate for technical entities to appear in the Business Layer, but on the other, they are Principals that have identities, need to authenticate, are granted access rights to consume services, etc. More on this topic in the discussion of Principals in 7.4.

Table 25: Actor & Role Properties

Element	Properties	Schema
Business Actor	A Business Actor should declare: - Its population size expressed as a point value or an expected range - Its type (HUMAN or ORGANIZATION); - A flag, if also a Data Subject; e.g., a member of staff in the context of an HR system	
<<Data Subject>>	If the Actor is primarily a Data Subject; e.g., an external Actor such as a citizen, this can be emphasized by using the DataSubject stereotype. DataSubject offers additional properties: - A flag to signal the possible participation of minors - A review period defining how often any personal data records should be reviewed and updated	
Business Role	A Business Role should indicate: - The privilege level assigned to the role, using the same set of enumerations defined earlier toxicPairs to mark the role's sensitivity to any segregation of duty conflicts	
Business Collaboration	Business Collaboration is the ArchiMate element to which multiple roles can be assigned. It is the natural place model any process with a Separation of Duty requirement. The segregationOfDuty property indicates whether segregation should be enforced between participating Actors (i.e., individuals possibly in the same role) or by distinct Roles.	
Relationships		

Element	Properties	Schema
	Assignment relationships just support the basic cardinality properties: sourceCardinality expresses the number of Actors assigned to a given Role or the number of active Roles performing a given behavior simultaneously. targetCardinality is normally 1. Serving relationships extend the basic relationship schema with a criticality property that denotes the importance of the service to the Consumer. targetCardinality indicates the number of concurrent service consumers/sessions. sourceCardinality indicates the number of request/response exchanges generated for each consumer process/session execution.	

These properties support the following validations:

- Data Subjects, which can only be specialized from human Actors, must be associated with at least one element of Business Information with the Personally Identifiable Information (PII) property set
- With respect to Toxic Groups that are segregated by Role:²⁹
 - An Actor being assigned to multiple Roles belonging to the same Toxic Group
 - A technical account participating in Role requiring segregation of duties
- That the rate of service requests remains within the service’s capacity
- Identification of high-security risk roles: associated with large constituencies of Actors having execute critical processes or sensitive access services

6.4.1 Governance

Governance runs like a seam through the People column of the SABSA Matrix, exhibiting two main aspects:

- Organization structures representing decision-making bodies, review boards, steering committees, etc.
They can be adequately modeled as Collaborations to which all the participating Roles can be assigned.
- Individual involvement as might be expressed by a RACI matrix

RACI presents an interesting design consideration in the ArchiMate Specification. The language is predicated on a grammar in which Active Structure’s (Actor or Role) access to a Passive Structure (the Asset) is mediated by a Behavior element in a “Subject–Verb–Object” syntax.

²⁹ Rather than segregated on the basis of distinct individuals who may share a common organizational or business role.

Applying this to RACI requires us to consider what the “Process of Being Accountable” means and what it would look like. Instead of contriving some pattern that fits this grammar, the Security Overlay proposes to stereotype a directed association: <<RACI>> specifically for this purpose. Figure 32 shows how it can be used.

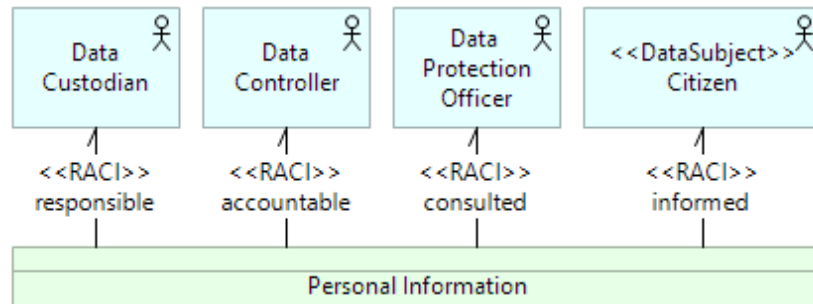


Figure 32: Representing RACI Relationships

A note of caution about using generic names for RACI targets: take care not to create Singularities.

Consider the example of Figure 33a: “Data Owner” is a poor choice of name for a RACI role. It is likely to be re-used in multiple contexts in multiple views. While it still may be possible to understand the modeler’s intent from each view in isolation, the entanglement in the underlying model, shown in Figure 33b, renders automated analysis impossible.

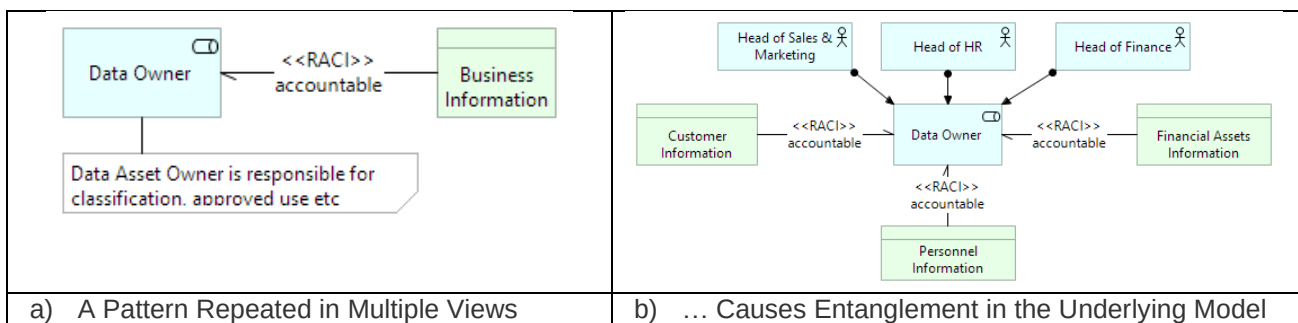


Figure 33: Avoid RACI Entanglement

Alternative patterns that avoid this issue are shown in Figure 34.

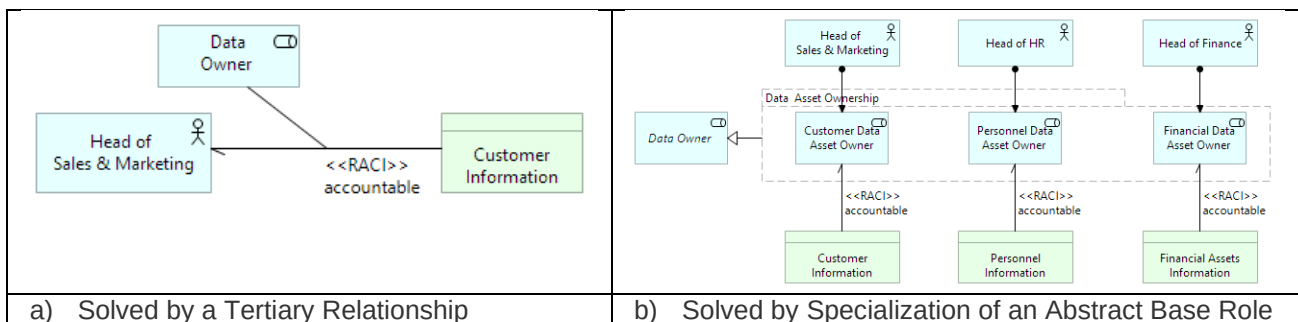


Figure 34: Recommended RACI Patterns

Table 26: RACI Properties

Relationship	Properties	Schema
<pre><<RACI>></pre>	RACI is a stereotype of Association in which the relationship name is restricted to the enumerated values: RESPONSIBLE, ACCOUNTABLE, CONSULTED, or INFORMED. Directionality is fixed to TRUE and directed towards the Actor or Role. sourceCardinality expresses the number of items that are within the scope of the RACI association. targetCardinality is normally 1.	

6.4.2 Threat Actors

Security models, by definition, have to consider the potential abuse of a system through malicious intent. Possible ways of modeling such threats are considered in Figure 35:

- As an Actor: A constituency that is known to pose an accidental or intentional threat
- As a Role: Representing a malicious intent, directed against the target system
- As an action (a behavior or event) that occurs by error, omission, or intent

All three raise concerns.

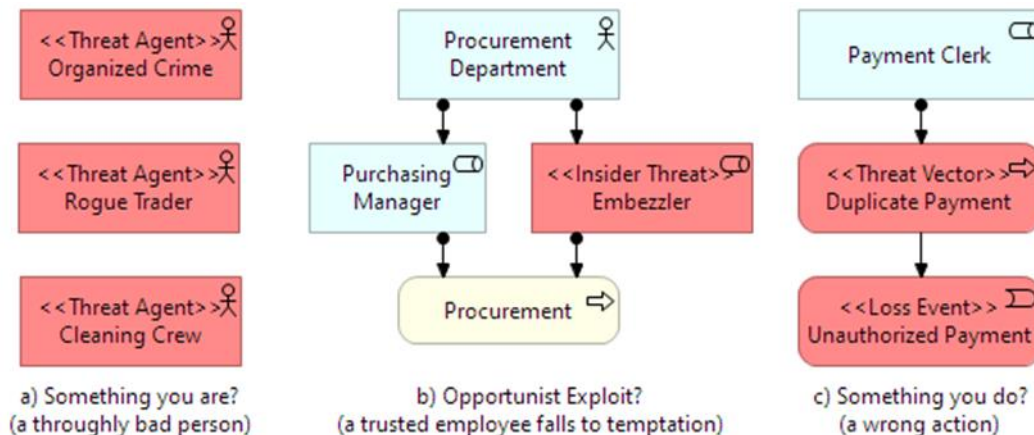


Figure 35: Possibilities for Modeling Threat Actors

Option (a) seems the most intuitive. It provides a useful representation of Threat Agent constituencies (population size, motivation, capability, determination, etc.), essential information to threat and risk modeling. It is fine for cases where the perpetrator is not explicitly identified (e.g., Organized Crime, Rogue Trader). It may even be reasonable to identify a particular Hacker Collective that poses a concern. Caution should be exercised if using <<Threat Agent>> to label specific persons or entities (competitors, states, and certainly not internals) as malicious, if these

groups may eventually have access to the model. Maligning individuals in this way is unnecessary, insensitive, and in many contexts, culturally unacceptable.

Option (b) exhibits the same issue: if we accept that personnel are generally honest and well-intentioned, it can cause offence if they are modeled with an explicit suspicion of criminality.

Option (c) raises concerns of alluding to malign intent. It misrepresents exceptional behavior as a formal process and assigns malicious motive to what could be an accidental action. Although analysis of insider threats is unavoidable, we should be conscious of these sensibilities in our models. Techniques for presenting these scenarios in a more acceptable form include:

- **Compartmentalization:** It is better to contain analysis of misplaced trust and abuse to specific risk views that can be restricted from the main published model
- **De-personalization:** Ensure that sensitive threat-related elements are modeled with a degree of separation from (i.e., not directly attached to) Actors and Roles, as shown in Figure 36

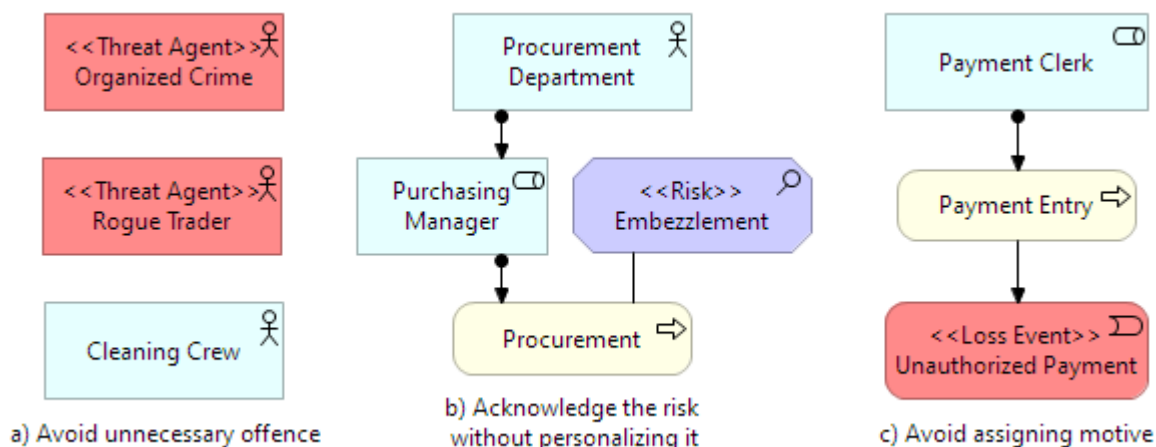
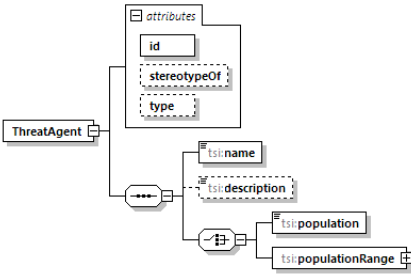


Figure 36: Sensitivity in the Representation of Threats

- **Alternative Scenario:** In some types of analysis; e.g., impact assessment, it may be possible to substitute an insider threat with a functionally-similar (from a security perspective) scenario predicated on a compromised device or account
- **Zero Trust:** Modeling within a security architecture framework that embraces zero trust principles that consistently performs verification, will normalize an approach in which the challenging of trust is commonplace without a strong inference of implicit suspicion

The profile for the Threat Agent element is shown in Table 27.

Table 27: Threat Agent Properties

Relationship	Properties	Schema
	Threat Agent is a stereotype of Business Actor and contains the same set of properties, except that: It cannot be designated as a Data Subject Its type property can take the value “TECHNICAL” to represent computer viruses and other types of malware.	

6.5 Business Geography

Business geography is easily modeled using the ArchiMate Location element unadorned. Location is intended to represent both physical and conceptual space. A means of marking a Location as a security domain (conceptual space) is discussed in Section 7.5.

6.6 Business Time Dependencies

The SABSA Time cell is concerned with the delivery schedule of goals and responding to events.

Target dates in the ArchiMate Specification are intrinsic to the definition of a Goal (a desired state to be reached by a defined point in time) and can be made explicit through Implementation and Migration views.

The Business Event element can be used unadorned. A means of using a Business Event as a Security Event (conceptual), as used in the example of Figure 35, is discussed in Section 7.6.

7 Modeling the Conceptual Security Architecture

The Conceptual Architecture provides the architect's view of the System of Interest. It is concerned with bridging real-world entities from the contextual layer into abstractions for the Logical layer. The Security Architect must also devise a strategy to protect these assets in the virtual world, to assure that ICT systems reliably deliver the key capabilities supporting the mission despite threats and uncertainty.

The rows corresponding to the Conceptual Architecture's artifacts and activities are defined in the 2018 revision of the SABSA Matrices [[SABSA R101](#)], reproduced in Table 28.

Table 28: SABSA Conceptual Architecture

		Assets (What)	Motivation (Why)	Process (How)	People (Who)	Location (Where)	Time (When)
Conceptual	Artifacts	Business Value & Knowledge Strategy	Risk Management, Strategy & Objectives	Strategies for Process Assurance	Security & Risk Governance; Trust Framework	Domain Framework	Time Management Framework
		Business Attributes Taxonomy & Profile	Enablement & Control Objectives; Policy Architecture; Risk Categories; Risk Management	Operational Process Inventory; Process Mapping Framework	Owners Custodians & Users; Trust Modeling Framework	Security Domain Concepts & Framework	Through-Life Risk Mgmt Framework; Attribute Perf. Targets
	Activities	Proxy Asset Development	Developing Risk Objectives	Delivery Planning	Role Management	Business Portfolio Management	Service Level Definition
		Defining Business Attributes Profile: Performance Criteria, Key Performance Indicators (KPIs) & Key Risk Indicators (KRIs)	Maintaining the Risk Modeling Framework; Risk Analysis on Business Attributes Profile	SLA Planning; BCP; Financial Planning; Transition Planning; Services Catalog	Maintain Trust Modeling Framework; Liabilities; Define Roles & Responsibilities	Business Footprint; Points of Supply & Access	Manage Performance Criteria & Targets; Abstracting Attribute Performance Targets

This outline enables us to visualize the set of processes and activities necessary to create and maintain an Architectural Description of the Conceptual Architecture, expressed in the ArchiMate language in Figure 37.

Although the Conceptual layer has no ArchiMate equivalent, many of the asset (What) and motivation (Why) concepts (attribute profiles, metrics, risk, compliance, control objectives, and layering, etc.) have already been discussed as stereotypes of Motivation elements (Chapter 5).

This section discusses the practical use of these elements in risk management process assurance, identifies conceptual security services (How) and makes proposals for the remaining artifacts from the People (Who), Location (Where), and Time domain (When) models.

Finally, it considers how the conceptual layer can be inserted between the ArchiMate business application without breaking existing models or abusing the language (too much!).

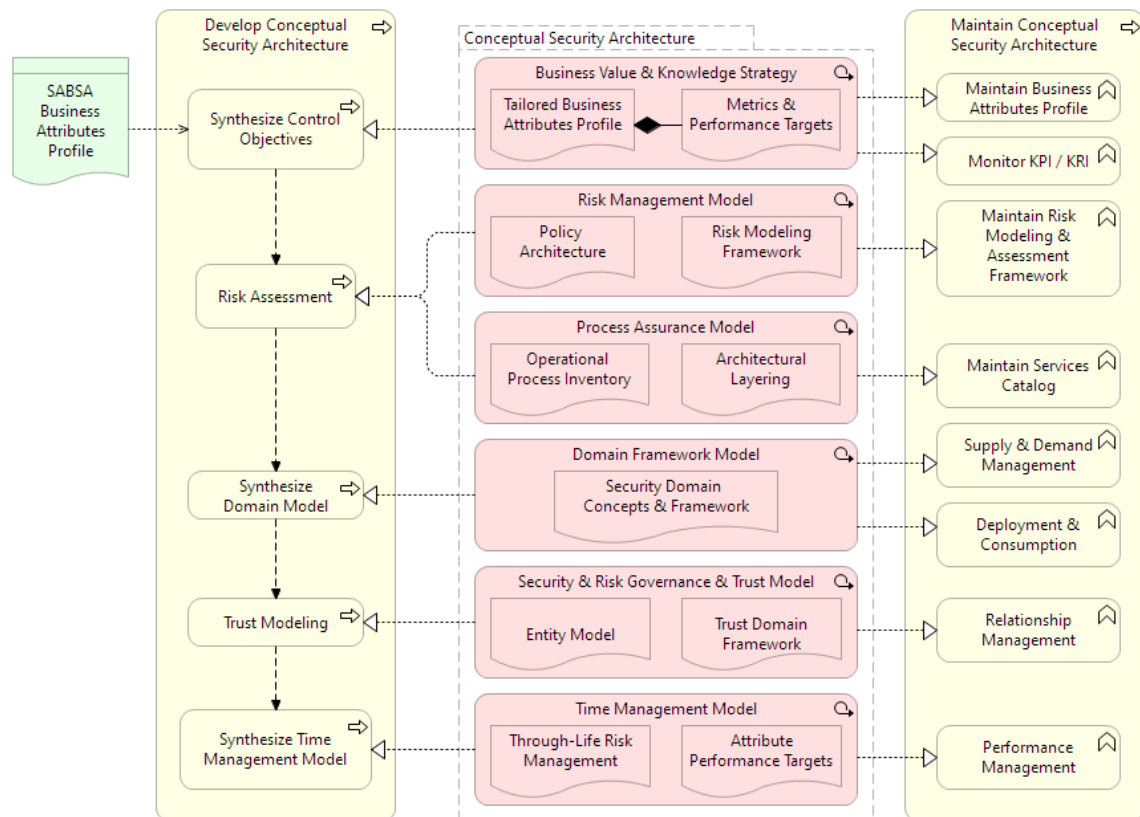


Figure 37: Developing the SABSA Conceptual Security Architecture

7.1 Attribute Profiling

Section 5.3 described how SABSA Attributes can be represented in the ArchiMate language. To recap the key points:

- SABSA Attributes are at the apex of the motivation pyramid – more abstract than Control Objectives
- They are global atomic singletons: existing at most once in the model and never appearing, even as copies, more than once in any ArchiMate diagram
- In models, Attribute profiles are constructed using influence relationships to form tree structures that broadly follow the layered architecture

- Each Attribute may influence/be influenced by zero or more other Attributes, but the tree structure must hold: these influence relationships should not be directed downwards from higher to lower layers, so circular paths should not occur in the structure
- Attributes should be associated to the elements to which they apply, and must be influenced by another Motivation element: typically, another Attribute and ultimately by a Goal or Requirement

This section describes the creation of such a profile through a worked example from GDPR Article 5 [GDPR], which sets out the principles relating to the processing of personal data. Article 5 §1 clause (a) states:

“Personal data shall be (a) processed lawfully, fairly and in a transparent manner in relation to the data subject (“lawfulness, fairness, and transparency”);”

By identifying key adjectives in the text, (marked in **bold**) we can select appropriate Attributes from the SABSA Attribute Taxonomy to create the initial Attribute Profile of Figure 38.

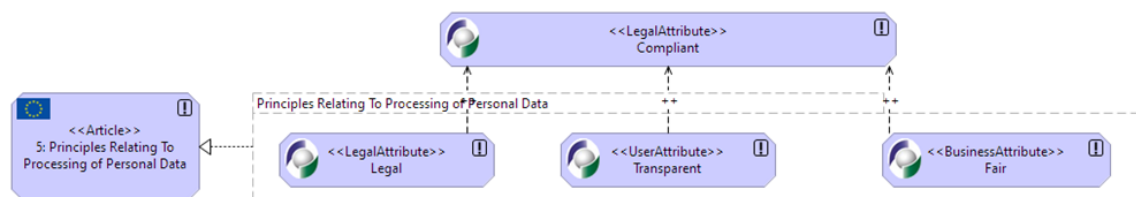


Figure 38: Attribute Profiling (i)

Had there been a legal definition of these terms in the regulation, it may have been possible to create Compliance Objectives at this point, but as there is none, analysis proceeds to clause (b).

“(b) collected for **specified**, **explicit**, and **legitimate** purposes and not further processed in a manner that is incompatible with those purposes; further processing for archiving purposes in the public interest, scientific or historical research purposes or statistical purposes shall, in accordance with Article 89(1), not be considered to be incompatible with the initial purposes (“purpose limitation”);”

If we accept “**legitimate**” as a synonym of “**legal**” and that anything “**specific**” is both, by definition, “**explicit**” and contributing to “**transparent**”, then we can refine the Profile to that of Figure 39. Note that the caveat to Clause (b) is a candidate <<Exception>> to any Compliance Objectives derived from this profile.

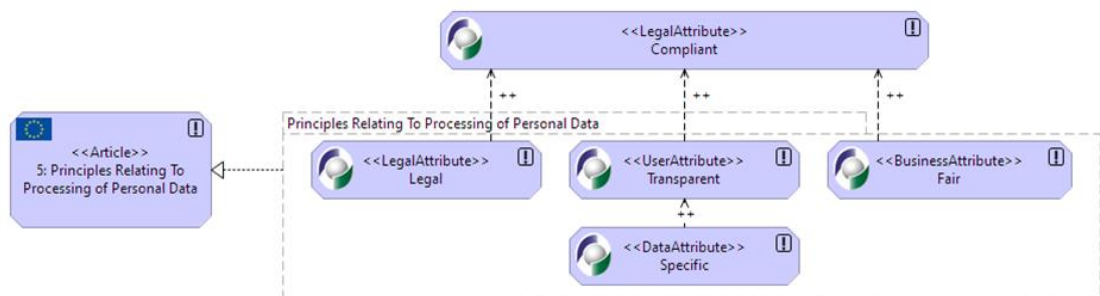


Figure 39: Attribute Profiling (ii)

Continuing this process with Clauses (c) and (d), the profile can now be elaborated to that of Figure 40.

“(c) **adequate, relevant and limited** to what is necessary in relation to the purposes for which they are processed (“data minimization”);

(d) **accurate** and, where necessary, **kept up to date**; every reasonable step must be taken to ensure that personal data that are inaccurate, having regard to the purposes for which they are processed, are erased or rectified without delay (“accuracy”);”

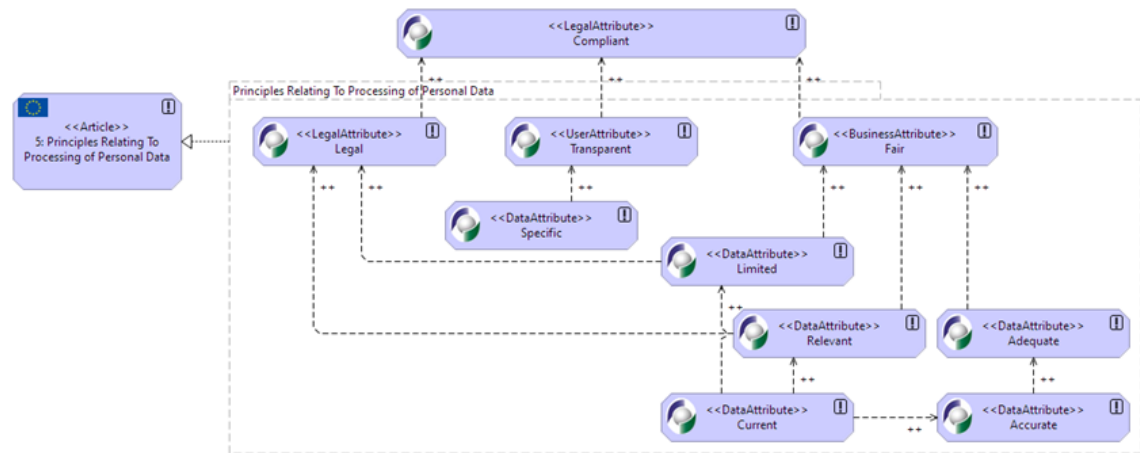


Figure 40: Attribute Profiling (iii)

By similarly analyzing clauses (e) and (f) and §2, we arrive at the completed profile shown in **Error! Reference source not found.**

“(e) kept in a form which permits **identification** of data subjects for no longer than is necessary for the purposes for which the personal data are processed;

(f) processed in a manner that ensures appropriate security of the personal data, including **protection** against unauthorized or unlawful processing and against **accidental loss, destruction or damage**, using appropriate technical or organizational measures (“**integrity and confidentiality**”).

2.The Controller shall be **responsible** (sic³⁰) for, and be able to demonstrate compliance with, §1 (“accountability”);”

³⁰ The Controller is actually *accountable* for demonstrating compliance using RACI terminology.

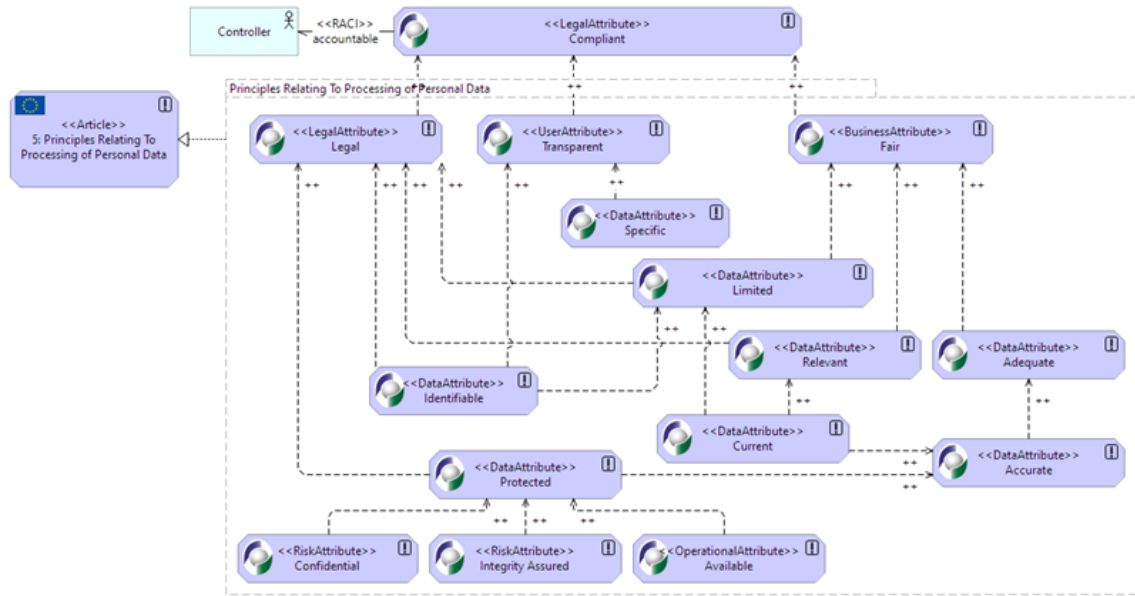


Figure 41: Completed Attribute Profile

The final Attribute Profile captures the essence of Article 5: the leaf nodes [Confidential, Integrity-Assured, Identifiable, Specific, Current, and Available] for which metrics and Compliance Objectives shall be defined.

7.2 Risk Management & Strategy

Risk and its constituent factors (threat, vulnerability and impact) are modeled as stereotypes of Assessment (adhering to the guidance in Section 4.3 and Section 5.5). The Security Overlay provides high-level representations of these concepts that can be adapted to suit an organization's preferred risk methodology. The schema for these elements has already been presented in Table 13: Risk Element Properties.

7.2.1 Risk Management

The Motivation aspect of the SABSA Matrix also addresses the process of risk management. Recalling the discussion of architectural planes in Section 4.2, a risk analysis process would be modeled in the secondary architecture (Management Processes) and look something like that shown in Figure 42.

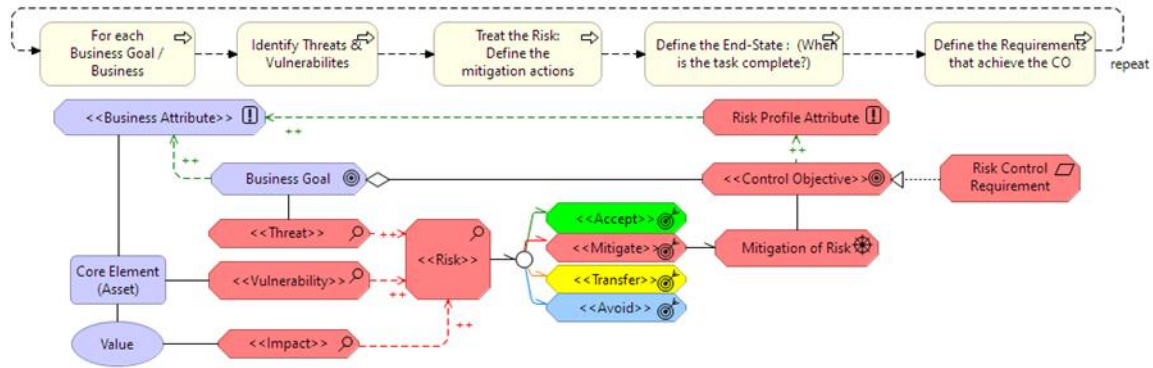


Figure 42: Business Risk Analysis Process

The first step identifies assets of significant value, and the SABSA Attributes in which that value is enshrined. For each Attribute, a set of concrete Business Goals are defined that describe the desired future state. The risk analysis process takes each Business Goal in turn and performs:

- Threat Analysis to identify threats and uncertainties affecting the Business Goal
- Vulnerability Analysis to identify weak points in attack vectors that could expose Attributes to this threat
- Impact Analysis to assess potential loss of Asset value, should the threat materialize

The risk assessment considers the severity of the threat, the exposure of the vulnerability, and the value of the Asset to determine the nature and scale of the risk. An influences relation is proposed here to reflect directionality and inherent subjectivity by which these factors influence risk analysis.

In the risk treatment phase, the Owner of a Business Goal must decide to accept, mitigate, transfer, or avoid the risk (modeled as stereotyped **Outcomes** of the Treatment process). The latter two options will most likely result in some redesign of the Contextual model, so the need to model them explicitly may be transient.

A <<Mitigate>> outcome creates a **Driver** for change. Ideally, a **triggers** relationship would be available here, but directional association must suffice as the specification forbids the participation of motivational elements in dynamic relationships. Mitigation is addressed by aggregating *Control Objectives* into the Business Goal. Aggregation best serves the many-to-many nature of the Goal-Control Objective relationship.

Of the alternative relations, composition implies that a Control Objective is inherent and exclusive to a single Business Goal. Realization presents each Control Objective as a choice, each sufficient to achieve the Goal.

7.2.2 Policy Architecture

The Policy Architecture is a framework of codified statements of regulatory compliance controls or those documented in the organization's policies and standards. As discussed in Section 5.8, policies, standards, and regulations are modeled as Representation stereotypes containing a structure of Groupings, Articles/Mandates, Compliance Objectives, and Requirements.

Controls mandated by policy are significantly different from those derived from risk:

- Policies are predicated on Statements of Applicability that define the policy’s scope and the conditions under which its controls apply – if the System of Interest is within scope and meets the criteria, the control statements must be applied, regardless of the organization’s risk appetite.
- Emphasis is on the traceability/audit of legal, compliance, or certification requirements to controls

To help manage the organization’s compliance posture, the Security Overlay provides elements that support the architectural pattern shown in the metamodel of Figure 43.

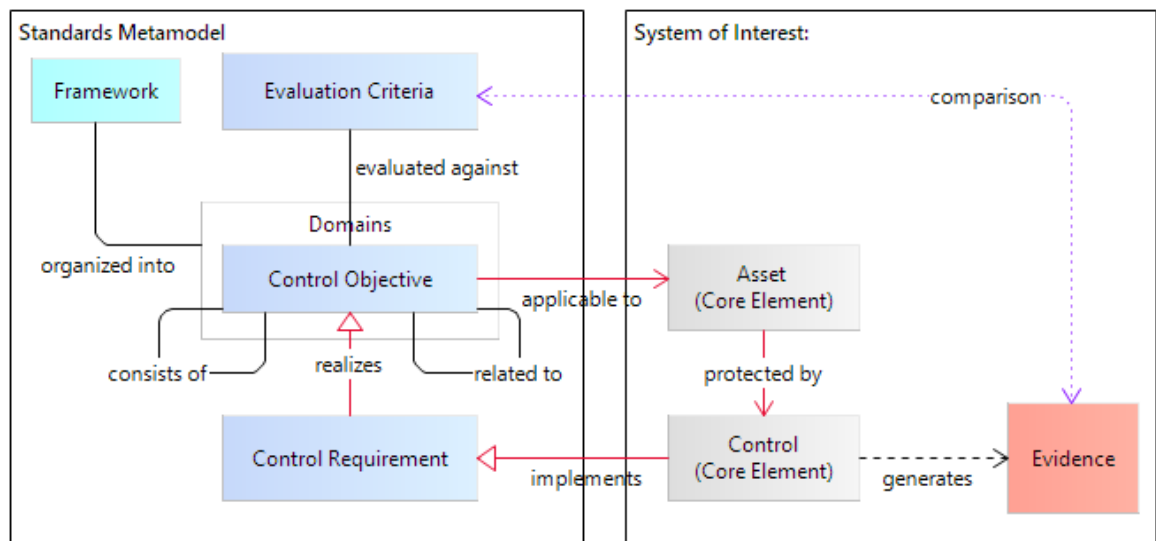


Figure 43: Compliance Metamodel

The structure of the compliance document is shown on the left, the Target Architecture on the right. Two kinds of relationships link them: one denoting the applicability (applicable to) of Control Objectives to Assets (core layer elements) and the other implementation (implements) of Control Requirements (and Constraints) by real-world Controls.

A state of compliance posture is demonstrated if, and only if, the following can be established:

- Each Asset to which an Objective applies is protected by a Control that implements each of the Requirements necessary to realize that Objective; i.e., there exists the closed path, marked in red
- All Controls on these paths produce Evidence of effectiveness that meet minimum criteria

The former, necessary but not sufficient, may be established by path analysis of the model.

The extent to which the latter can be automated varies on a control-by-control basis. In any event, such analysis would be performed outside the architectural model (unless you are contemplating a daily import of Production log files into your Enterprise Architecture tools!). The model should at least reference the locations where the necessary evidence can be found.

Figure 44Error! Reference source not found. shows how this might be applied to model compliance with NIST SP800-53r5 Objective, AC-1 [NIST SP 800-53].

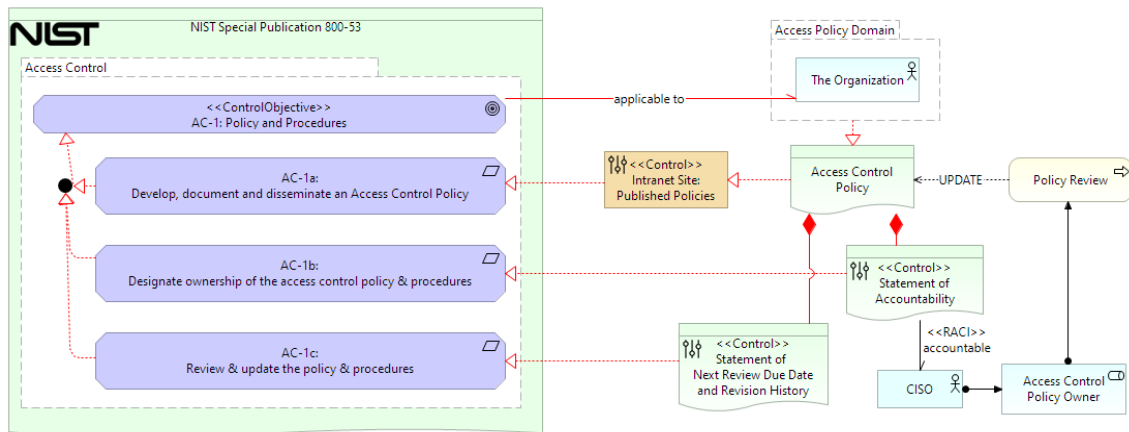


Figure 44: An Example Compliance Model

Given their sheer quantity, the ability to trace policy statements through to the realization of controls is perhaps one of the most valuable of model validation capabilities.

7.2.3 Multi-Regulatory Compliance

A typical policy, standard, or regulation contains several hundred compliance statements and requirements. The task of tracing them to effective control measures via manual methods presents a significant challenge. Unfortunately, most organizations operate in a multi-regulatory environment in which they are subject to multiple, simultaneous control frameworks (imposed by regulators, service providers, key clients, or market demand). Moreover, the digital economy demands compliance with ever more standards, with increasing rigor, more frequently, and held to ever high levels of scrutiny. A strategy of addressing each framework in isolation would not only require a huge commitment of resources, but would also be wasteful, given the overlap and duplication in their scope and objectives. Consequently, an aim of modeling should be to refactor these controls, drawn from disparate risk and compliance sources, into a minimal, consolidated control set.

The modeling approach outlined in this document can help achieve this consolidation. This section explains how.

We have seen in Section 5.7 and Section 7.1 that we can build an Attribute profile from a multi-tiered risk analysis or compliance mandate to produce a structure that resembles either the right or left side of Figure 45.

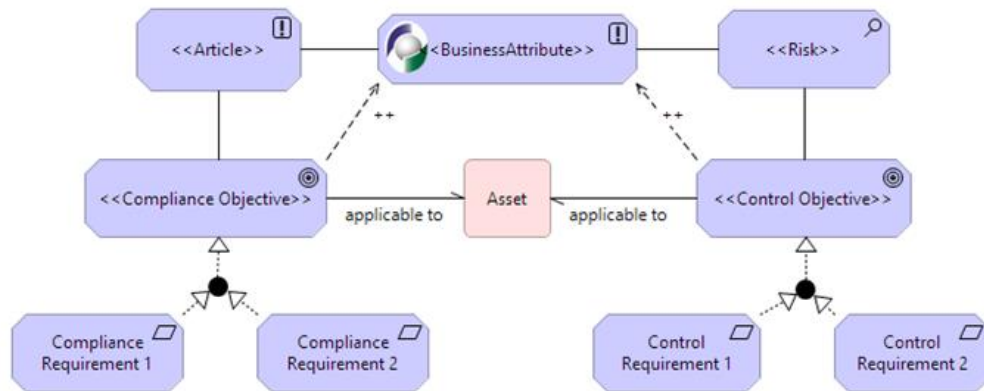


Figure 45: Control Consolidation

After investing a significant effort in profiling all assets in the architecture, a model may easily contain thousands of control elements (Compliance & Control Objectives, and Requirements). Most modeling tools provide a means to generate a view from the perspective of any element, so it is relatively easy to enumerate all the control statements that apply to any given asset.

Unfortunately, because these control statements have been derived from different sources at different times, their titles and descriptions are expressed in free-form text, making it difficult to detect potential duplicates.

However, because these statements have been linked to an Attributes Profile and because SABSA Attributes are atomic singletons, we can be fairly confident that control statements that influence the same SABSA Attributes are good candidates for consolidation. By re-wording the control statement title and description, combining similar statements into one is often possible. Even for control statements too complex to merge easily, the fact that they influence the same Attribute means that rationalization should be possible at the sub-goal or requirement level.

Figure 46 shows this in practice using a generated view of the SABSA Attribute: “Available”. It shows this Attribute being influenced by four Objectives: two from GDPR [GDPR] and two from ISO/IEC 27001 [ISO/IEC 27001].

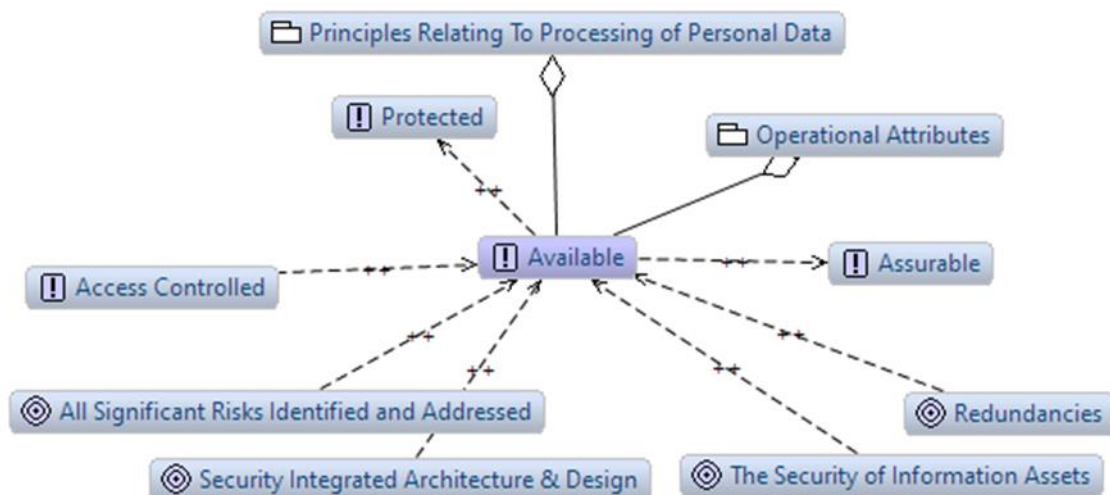


Figure 46: Possible Duplicate Objectives, Coupled Through Attribute Profile

7.3 Conceptual Security Services

The Process cell of the SABSA Matrix considers the “How” of Conceptual Layer Services.

Section 4.3 describes how Control Requirements must eventually be implemented by concrete measures in the core layers, shown in Figure 8 as Security Services and later extended in Section 5.10 by the Security Overlay’s <<Control>> stereotype for non-service-oriented mechanisms. However, two important security services must be represented at the conceptual level because they mediate the use of Logical layer resources by Business Layer Actors. We speak, of course, about Authentication and Access Control.

So how can we represent these services in our models? Figure 47 shows some possible options.

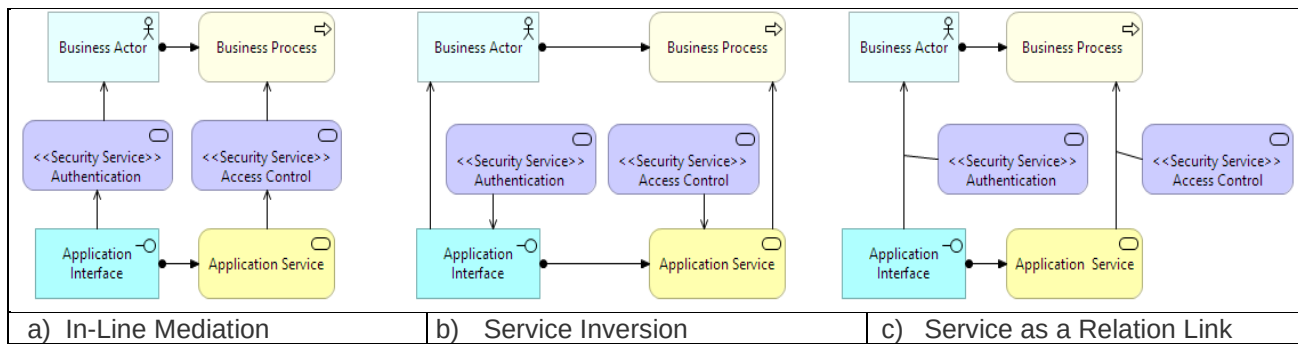


Figure 47: Security Services in the Conceptual Layer

There are two important things to note:

- Conceptual services are Singletons: The idea of an Authentication Service, for example, is unique, no matter how many ways it might be implemented
- A convention on service placement: Access control mediates access to services; Actors authenticate to interfaces

In Figure 47(a), conceptual services are inserted in-line between Business and Application Layer elements. Visually, the role of mediation is clear. However, shared services (e.g., single sign-on) become a Singularity: a hub in the underlying model, linking all Actors and Interfaces, all Processes to all Services, and losing the analytical ability to determine which Business nodes use which Applications.

Figure 47(b) restores the direct serving relationships from Application to Business Layer but loses the sense of security services acting as mediator. The pattern also runs counter to an architectural design principle that services offered should not be invoked from a lower layer. The pattern is not particularly accurate either: authentication, in particular, is often intercepted by application containers; i.e., at the technology level, rather than invoked explicitly by the applications themselves.

Figure 47(c) provides a pattern that resolves these issues: the direct link between business process and application is maintained, the security service is visually represented as guarding the service call, and there is no inversion of the directionality of service invocation between layers. It also retains the flexibility to show where the service implementations are deployed invoked in the Application or Technology Layers.

At a practical level, the Security Overlay proposes Security Services as Singleton elements, created as stereotypes of Application Services, but rendered the same visual style as Motivation elements.

7.4 Identity and Trust

The “People” cell of the SABSA Conceptual Layer considers how business entities (Organizational Actors and Roles) can be mapped to their Logical counterparts (Accounts and Application Roles); an important topic, given the emphasis placed on Logical Access Management (LAM) in security governance.

Similarly, the trust that exists between real-world actors needs to be made explicit in our models if it is to be adequately protected. This section proposes methods for this to be achieved.

7.4.1 Identity and Access Rights

LAM is a Logical layer process that provides Contextual entities (Actors in Business Roles) with suitably provisioned accounts, principally to access applications and technology. A few Conceptual elements (Principal, Access Rights, Credentials) are required to establish this mapping from Contextual to Logical elements. Currently, none of the concepts mentioned, nor any representation of Accounts and Application Roles, exist in the ArchiMate Specification.

What the Security Overlay would like to express is shown in Figure 48. Business Actors are represented by one or more Principals in the Conceptual layer.³¹ Each Principal is issued with Credentials and granted a set of organization-wide access rights (Authorizations), reflecting the activities of their business role.

Note that not all Principals have to represent Business Actors. The concept can also represent machines: robots, processes, applications, servers, or logical entities that participate in authenticated transactions.

At the Logical layer, Credentials are verified by an Authentication Service to bind Principals to Accounts. The LAM system will have pre-provisioned each account with sets of application-specific access rights that map to Application Roles defined in and recognized by Application Components and System Software.

This pattern forms part of an “Administration time” process (secondary architecture) and is supported by Management Matrix services such as identification, enrolment, authorization, provisioning, and credential management. Credentials and Application Role Claims are presented to Authentication and Access Control.

³¹ This construct is often used to ensure that Single Sign-On does not enable an Actor to move seamlessly between Production and non-Production environments, or that an individual who is both an employee and a customer of an organization is issued with accounts that are logically isolated.

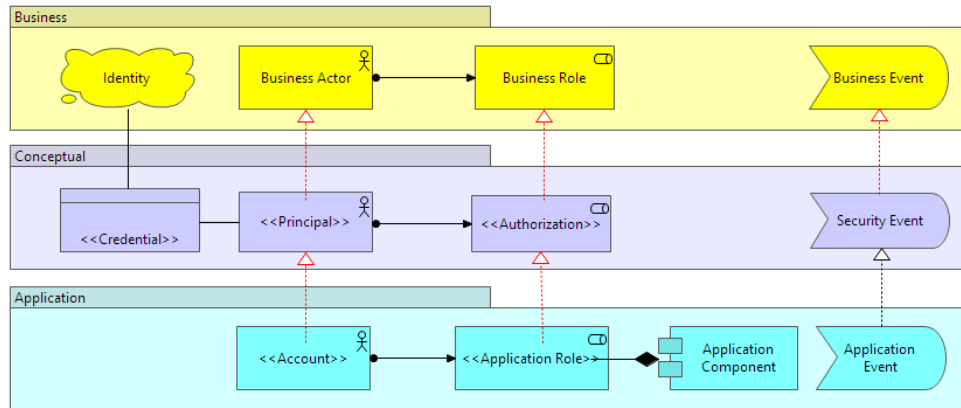


Figure 48: What the Security Overlay Would Like to Express

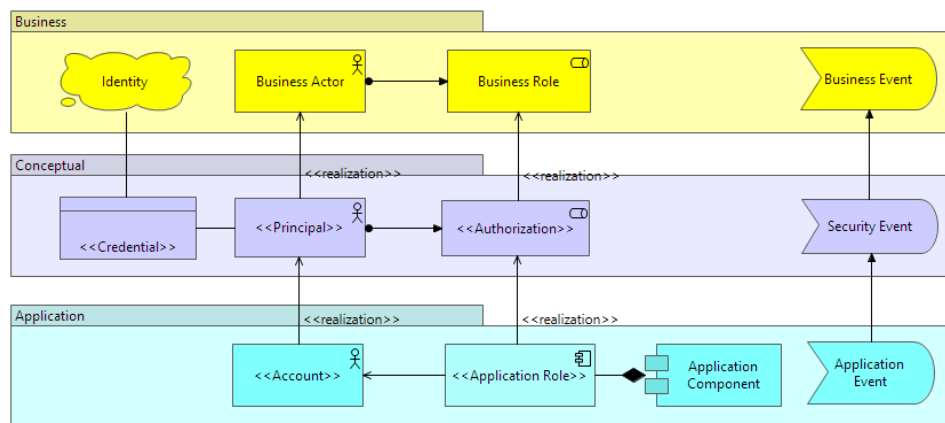


Figure 49: What the ArchiMate Specification Supports

As the ArchiMate Specification lacks native elements to represent these security concepts, the Security Overlay uses stereotypes of Business Layer elements. Ideally, the Security Overlay would have kept the precedent of using the **realization** of similar entities across layers to achieve a consistent structure from Business Actor (*Contextual*), via Principal (*Conceptual*) to Account (*Logical*) and from a Business Role (*Contextual*), via Authorization (*Conceptual*) to an Application Role (*Logical*). It would also have been intuitive to use composition from Application Component to Application Role as the definition and implementation of role-based access is coded into software.

Unfortunately, these cases extend the grammar of the ArchiMate language beyond that for it was designed: **realization** between elements of the same type and Role **composition** into Application are not legal. The closest approximation supported by the language specification is shown in Figure 49, with an improvised *Realization* relationship (created from a stereotyped Serving) used to derive Conceptual layer elements from Business ones and Logical layer entities from Conceptual ones.

Application Roles can only be modeled as composites of Application Components by modeling them as Components. Visually, this can be offset if your tool supports custom iconography. The significant upside of this is the ease of assigning Application Roles to Application Functions.

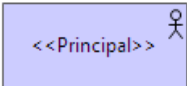
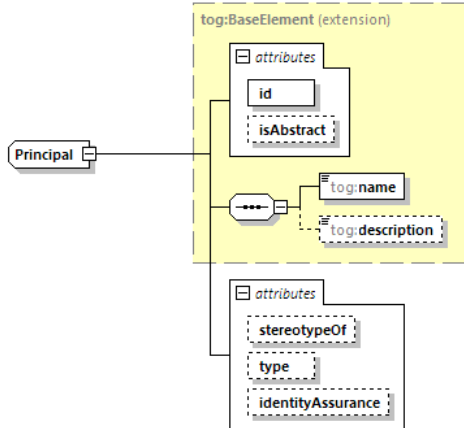
Linking these elements to the Logical layer, Account represents an authenticated Principal on the target system from which access to a restricted set of resources (e.g., the individual's own email) can be controlled. Application Role represents roles defined in the application code to support, for example, Role-Based Access Control (RBAC) mechanisms. These elements will be discussed further in the next section.

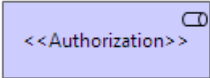
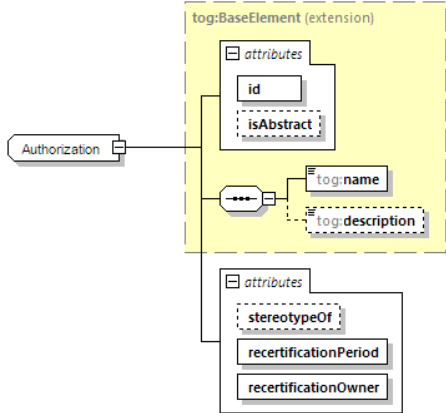
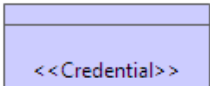
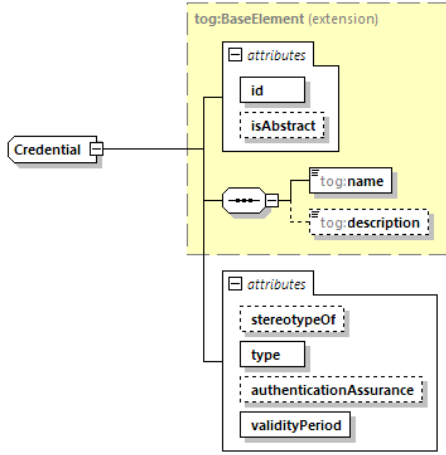
By modeling this structure, the following examples of security validation become possible:

- Transparency in onboarding: the ability to detect a single individual enrolled in the system under multiple Principals, thereby potentially by-passing segregation of duty controls
- To assure that human Principals are not assigned machine accounts and cannot access interfaces and services intended for machine accounts (and *vice-versa*)
- Highlighting areas of risk caused by sensitive functions being accessed by an Active Directory group that has no identified Owner or has not been recertified within the period required by policy
- Every instance of a Credential appearing outside the LAM domain (or the domain of its creation) should trigger requirements for secure distribution, storage, access control, etc.

Table 29 summarizes the new Conceptual layer elements. Logical layer elements are addressed in Chapter 8.

Table 29: Elements used in Logical Access Management

Element	Properties	Schema
	Principal is the conceptual representation of an Entity capable of holding an Account in the target system, including TECHNICAL entities. Principals are created through an identification process with a given level of identityAssurance.	

Element	Properties	Schema
 <pre> classDiagram class Authorization </pre>	Authorization is a conceptual representation of actions that a Principal is authorized to perform in a Business Role. As it decouples Business Roles from Application Roles, it models the type of mappings used in central access management systems; e.g., Active Directory Groups. Its properties capture the period and responsibility for the recertification cycle.	 <pre> <?xml version="1.0" encoding="UTF-8" standalone="no" ?> <xsd:extension base="to:BaseElement" name="Authorization" /> <xsd:element name="Authorization"> <xsd:complexType base="to:BaseElement" /> <xsd:attribute name="id" type="xsd:string" /> <xsd:attribute name="isAbstract" type="xsd:boolean" /> <xsd:attribute name="name" type="xsd:string" /> <xsd:attribute name="description" type="xsd:string" /> <xsd:attribute name="stereotypeOf" type="xsd:string" /> <xsd:attribute name="recertificationPeriod" type="xsd:string" /> <xsd:attribute name="recertificationOwner" type="xsd:string" /> </xsd:element> </pre>
 <pre> classDiagram class Credential </pre>	Credential is the conceptual representation of a proof of identity that enables a Principal to authenticate to a given application or authentication service. The type property is used to enumerate common types of credential: PIN, PASSWORD, SOFTTOKEN, BIOMETRIC, etc. The authenticationAssurance property denotes the associated authentication strength – taking into account the technology, processes of generation, distribution, expiry, etc. The validityPeriod property is used to express the credential's expiry lifetime.	 <pre> <?xml version="1.0" encoding="UTF-8" standalone="no" ?> <xsd:extension base="to:BaseElement" name="Credential" /> <xsd:element name="Credential"> <xsd:complexType base="to:BaseElement" /> <xsd:attribute name="id" type="xsd:string" /> <xsd:attribute name="isAbstract" type="xsd:boolean" /> <xsd:attribute name="name" type="xsd:string" /> <xsd:attribute name="description" type="xsd:string" /> <xsd:attribute name="stereotypeOf" type="xsd:string" /> <xsd:attribute name="type" type="xsd:string" /> <xsd:attribute name="authenticationAssurance" type="xsd:string" /> <xsd:attribute name="validityPeriod" type="xsd:string" /> </xsd:element> </pre>

7.4.2 Roles and Responsibilities

The SABSA Conceptual People cell addresses the concepts of role and responsibility. At the Business Layer, roles are defined in terms of mediating an Actor's engagement in behavior (e.g., processes). The task of the Conceptual model is not just to map these role assignments from the physical to the virtual realm but to do so in a way that is manageable, transparent, efficient, maintainable, scalable, and auditable. This section establishes this conceptual model.

Figure 50 shows how the concepts established earlier can be used to illustrate the relationship between Business Layer entities (An employee engaging in Business Process Y in the Role X). To perform Process Y, the employee needs to perform three functions (A, B, and C) using Application Z.

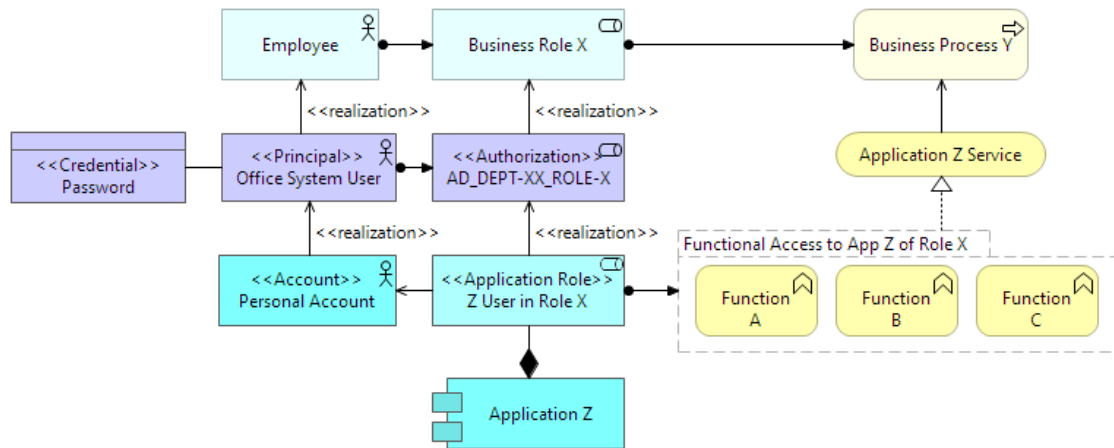


Figure 50: Modeling Identity and Role Concepts

In the Conceptual layer, the employee is instantiated as a Principal with an Office System User profile, assigned to an Authorization profile appropriate for staff of Department XX, in the Business Role X.

This Authorization profile (sometimes called a Group Profile) is an organization-wide set of authorizations, defining an employee's full access rights to the office network domain, multiple applications, infrastructure such as remote log-in, badge-entry systems, etc. The employee is also issued with a credential – a conceptual proof of authentication – that enables the person to claim the digital identity of the Principal.

When authenticated, the Principal is presented to a given target application, here, in Figure 50, Application Z, the application binds the Principal, by its identifier, to a local account that has been assigned a specific Application Role, whose profile includes access to Functions A, B, and C.

Key to this model is the recognition that an Application has no awareness of its external context. It can differentiate between different accounts, knows which accounts have been assigned to which local roles, and has been programmed to allow certain roles to access certain functionality. That is all.

At the Conceptual layer, the User Directory manages the enrollment of employees as Principals, allocated as members of various Groups. It also maintains a mapping from each Group to the Application-Roles that Group Members must perform in their business role. It sees nothing of the functional capabilities that lie behind those Application-Role titles.

7.4.3 Trust

The final aspect of the Conceptual People cell is that of the trust that exists between Business Actors. Trust is essential to business transactions yet rarely made explicit in conventional Architectural Descriptions.

Trust causes business partners to accept risks that would otherwise be declined, based on intangibles such as reputation, personal history, or the expectation of a long-term relationship. It is essentially a human quality that, though it cannot be reproduced outside the context of the business relationship, must be protected as soon as any part of that transaction is delegated to technology (e.g., an Agent, a process, or an IT system).

The analysis of signals crossing a security domain boundary provides a means to identify any implicit trust in the context of the interaction. Trust must be represented explicitly in the model to derive the necessary protection requirements. Examples of inter-domain signals are shown in Figure 51.

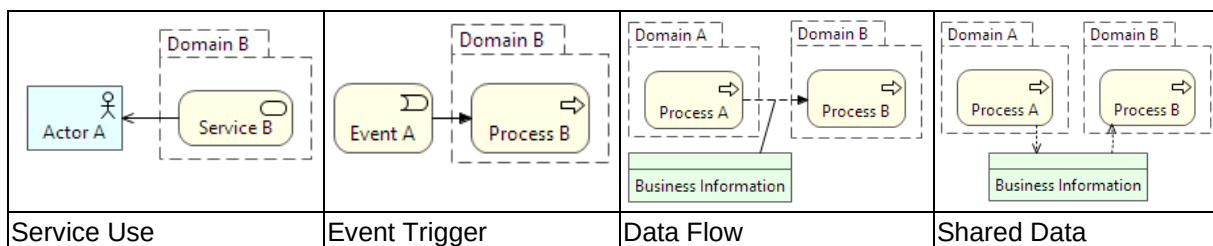


Figure 51: Common Examples of Signals Crossing Domain Boundaries

While active structure and behavior elements should be placed in the domain where an activity occurs (both visually and via structural relationships), the modeling guidance for passive structure³² elements is more nuanced. Information is generally copied and distributed in the course of processing and comes to exist in multiple domains.

A modeler may opt to assign an Object to all domains in which it is used or “free” it by not assigning it at all. The choice will often be determined by the sensitivity of the information: objects subject to regulation, for example, should be assigned to a security domain if access from outside that domain is a concern.

Another characteristic of a valid trust model is that, while any element may be a “*trusted*” entity, only an active structure (or behavior assigned to an active structure) can be a “*trusting*” entity. In other words, information may be the object of a trust relationship (be it trusted or untrusted) but is incapable of offering trust itself.

To model trust in ArchiMate language, a stereotyped flow relationship indicating that “A trusts B” seems most intuitive.³³ It can also convey a direction of trust that is independent of that of the underlying signal or show opposing flows to indicate bi-directional trust.

Like SABSA Business Attributes, Trust is modeled as a stereotype of Principle that, like Attributes, should only be *influenced* in the model through the realization of Control Objectives. A simple example of trust concepts expressed using ArchiMate language in this way is shown in Figure 52.

³² Except Artifacts and Materials (in technical and physical layers) which represent physical objects.

³³ While deliberately avoiding the question of what actually flows from one entity to the other.

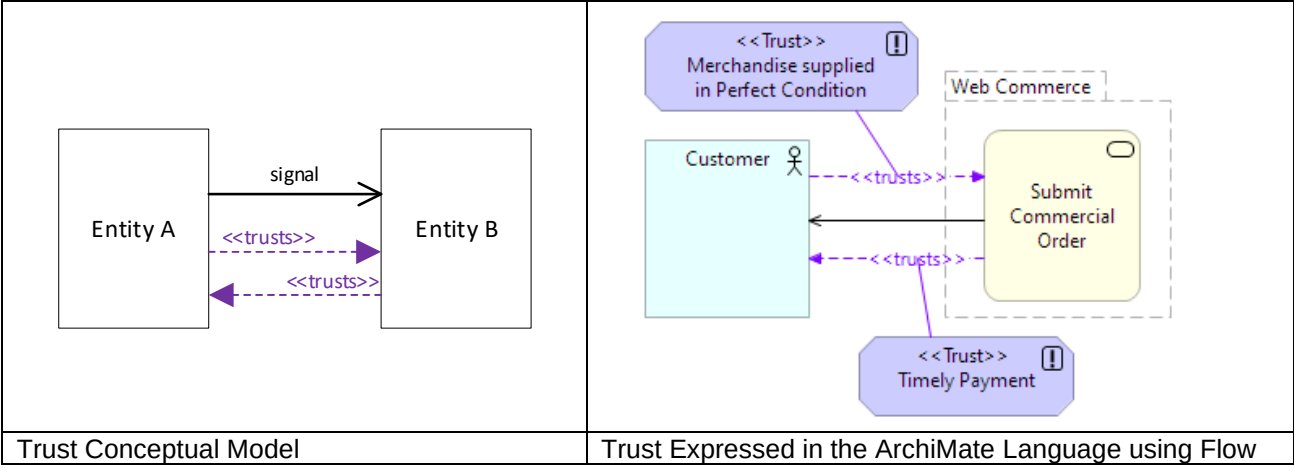


Figure 52: Simple Trust Relationships using Flow

This approach offers simplicity but, despite being a good first attempt, it has a couple of issues:

Firstly, the trust relationships are not linked to the underlying signal. Whenever more than one signal is drawn between a pair of entities, analysis cannot contextualize which trust belongs with which signal.³⁴

Second, the ArchiMate grammar prevents flow relationships from connecting to a passive structure. Trust in an Object, as in “*I trust every email that comes from my bank*”, cannot be expressed using this approach.

To overcome these limitations, the Security Overlay offers an alternative representation using Trust stereotypes alone. Figure 53 illustrates how this is used to apply trust expressions to the patterns introduced in Figure 51.

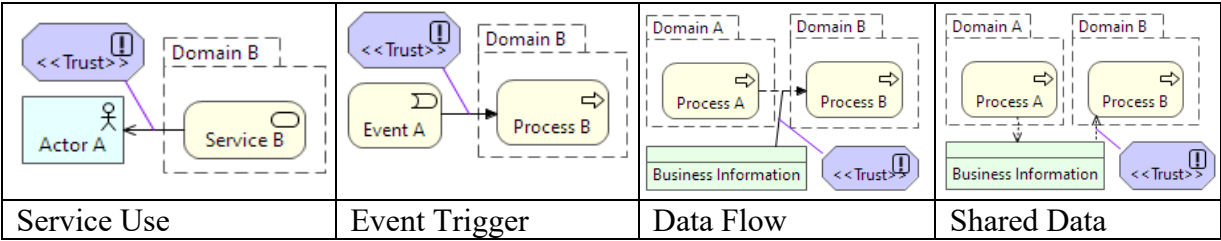
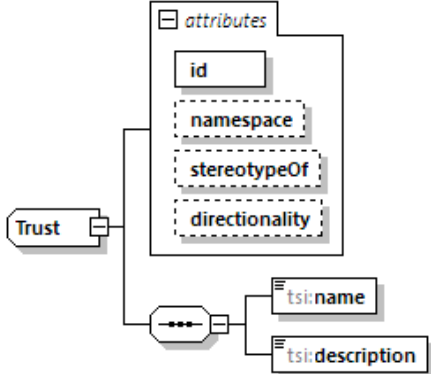
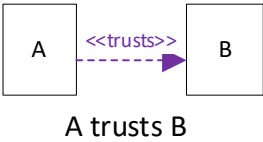
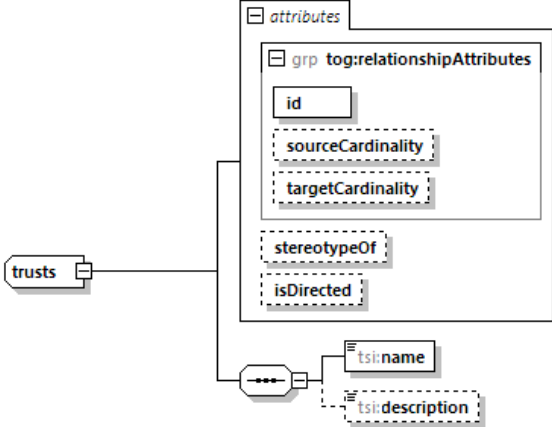


Figure 53: Trust Attributes Associated with Inter-Domain Signals

This alternative representation lacks a visual representation of the direction of trust, which is not necessarily the same as that of the signal, that can be partly offset by good naming of the Trust element and elaborated by its textual description. For automated analysis, the Overlay provides a **directionality** property that sets the direction of the trust relative to the associated signal.

³⁴ Although associations can be shown visually via selective diagrams ArchiMateSE aims to enable automated analysis.

Table 30: Elements and Relationships used in Trust Modeling

Element	Properties	Schema
	Trust is modeled as a stereotype of the Principle element that can be associated with any relationship to make any trust underpinning the transaction explicit. A directionality property defaulted to FORWARD declares the direction of the trust.	
Relationship		
	Simple trust between entities that arise from the general context rather than a specific transaction is modeled by a stereotyped data flow: the arrow indicating the direction of trust and its nature elaborated using a Trust element.	

The use of Security Domains and Trust Models support the following validations:

- Interactions that cross-security domain boundaries are identifiable as prime candidates for trust analysis

Traditional architectural descriptions focused primarily on technical aspects overlook the significance that trust plays in the business's risk appetite. Making this trust explicit in the model is a critical success factor for establishing a complete set of system protection requirements.

- A model declaring a serving relationship that crosses a trust boundary should also have a data or control flow (trigger) defined to be considered complete

This rule will identify any sensitive data (confidentiality, integrity, authenticity, etc.) crossing the boundary in the context of that service.

- Many patterns, such as access to sensitive or private data from outside the domain, can be matched as a policy violation if the Trust element is not protected through Control Objectives realized by Controls

7.5 Domain Framework Model

The Conceptual Domain framework denotes security domains in the System of Interest. The Security Overlay models them using a <<SecurityDomain>> stereotype of Grouping. These domains are an essential basis for creating Trust Models: the analysis of security concerns that arise from signals (service calls, system accesses, event triggers, and data flows) that are enacted across domain boundaries.

Where a SecurityDomain perfectly aligns with an existing bounded space (physical, logical, organizational, jurisdictional, network zone) defined by **Grouping** or **Location**, the Security Overlay offers an **isSecurityDomain** property, set FALSE by default, as a means of marking them as security domains without duplication.

The relevant ArchiMate elements for domain modeling are shown in Table 31. The ArchiMate syntax allows elements to be placed into Groupings using *composition*, *aggregation*, or *assignment*. Composition is generally too strong due to its implied exclusivity. In practice, elements are often subject to several domains simultaneously. *Aggregation* is usually the best choice for Grouping domains, *assignment* for Location.

Table 31: Security Domain Mapping

Element	Properties	Schema
	Any Location or Grouping element can be designated as a Security Domain by setting its isSecurityDomain property (FALSE by default) to TRUE.	
	SecurityDomain is a stereotype of grouping that makes the security domain property visually explicit. The isSecurityDomain property is fixed to TRUE.	

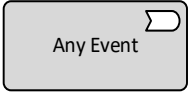
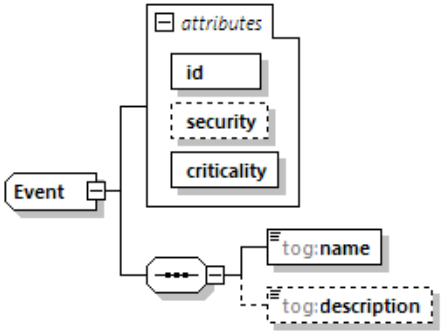
By assigning elements to security domains, analysis can determine whether a signal traverses a domain boundary: a condition that prompts the need for Trust Modeling.

7.6 Security Events

Security events are expressed using a similar approach to that outlined above.

The Security Overlay defines a security flag property for all Event elements, set FALSE by default. Marking an existing event as a security event is just a matter of setting this flag TRUE. A <<SecurityEvent>> stereotype can be created from any Business, Logical, Technology, or Implementation Event, as appropriate, to introduce a new (conceptual layer) security event.

Table 32: Security Events

Element	Properties	Schema
 Any Event  <<Security Event>>	Security-significant conditions occurring in the system context, triggering or triggered by active structure and behavior elements. Modeled by a stereotyped Security Event or by annotating an existing regular Event by setting the security property TRUE. The criticality property indicates the significance of the event.	

8 Modeling the Logical Security Architecture

The Logical Architecture documents the Solution Architect's view of the system. In this section, conceptual elements described in the previous section are applied to and guide the Application and Data Architecture.

It also proposes security properties for the ArchiMate Specification's Logical layer elements that support security analysis.

Table 33: SABSA Logical Architecture

		Assets (What)	Motivation (Why)	Process (How)	People (Who)	Location (Where)	Time (When)
Conceptual	Artifacts	Information Assets	Risk Management Policies	Process Maps & Services	Trust Relationships	Domain Maps	Calendar & Timetable
		Information Asset Register Business Information Model	Risk Models; Domain Policies; Assurance Criteria & Framework	Information Flows; Functional Transformations; Service-Oriented Architecture; Services Catalog; Application Functionality & Services	Domain Authorities; Entity Schema; Privilege Profiles; Trust Relationship Models	Domain Definition; Inter-Domain Associations & Interactions	Start Times, Lifetimes & Deadlines
	Activities	Logical Asset Management	Policy Management	Delivery Management	Enterprise-Wide User Management	Service Catalog Management	Evaluation Management
		Knowledge Management; Release & Deployment Management	Risk Modeling; Management of Policy Development & Maintenance; Policy Publication Compliance Management	SLA Management; Supply Chain Management; BCM; Financial Management; Transition Management	Trust Modeling; Identity & Access Management; Account Administration & Provisioning	Configuration Management; Capacity Planning; Availability Management	Monitoring & Reporting KPIs & KRIs

This outline enables us to visualize the set of artifacts and activities of the Logical Architecture, expressed in ArchiMate language in Figure 54.

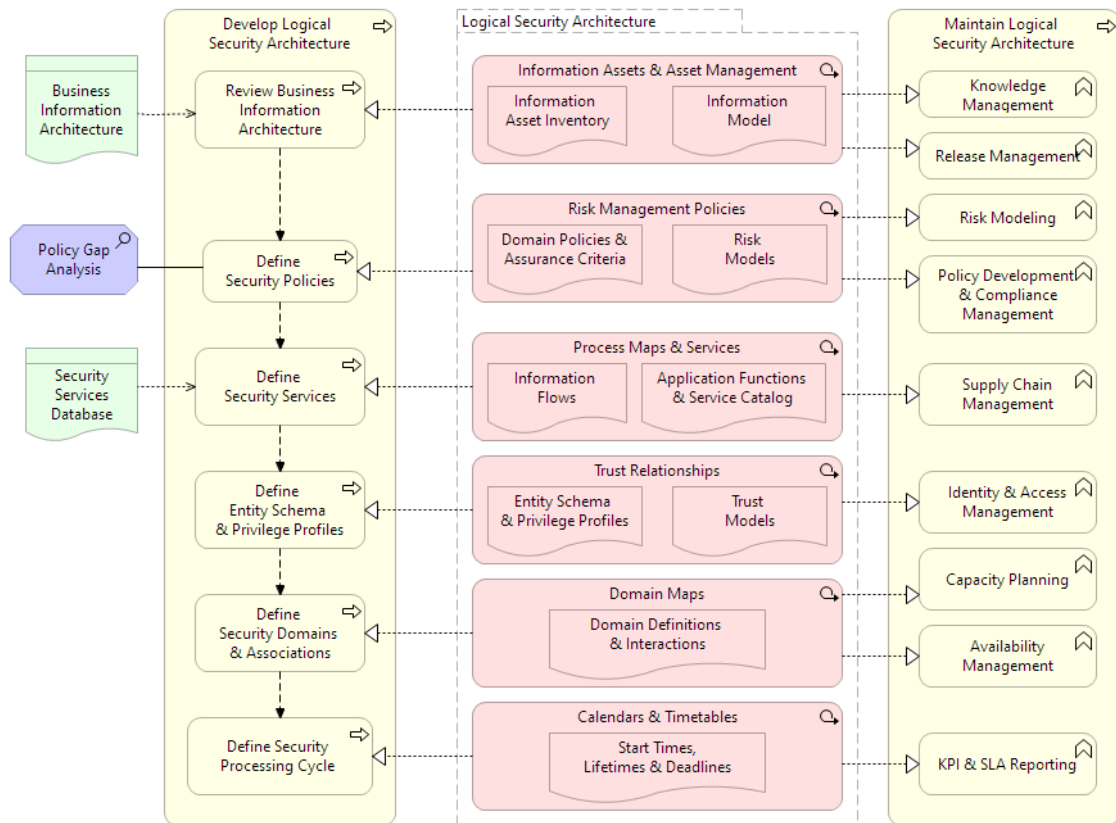


Figure 54: Developing the SABSA Logical Security Architecture

8.1 Information Assets

The principal information assets in this layer are software assets (applications) and data.

8.1.1 Application Components

Most organizations maintain a register of the applications deployed in their IT environment. The primary drivers for this register are operational: patch management, vendor support, license management, etc. Because these concerns are focused on software products rather than logical building blocks, discussion is deferred to Chapter 9.

The kind of inventory relevant at this layer is the identification of *critical* applications: the software upon which the organization is disproportionately reliant for achieving its primary mission. Organizations perform a regular “Critical Application Review” to identify these dependencies. Applications on the critical list are then prioritized for risk assessment, penetration testing, monitoring, business continuity planning, audit, etc.

But how should critical applications be identified? The most common approach is simply to request a rating recertification from the Application Owner. This approach has a few issues:

- In a Service-Oriented Architecture (SOA), the use of application services can be highly dynamic

Even if the Application Owner is aware of who is using the application (a big ask), they may not know for what – neither at the time that access was granted, how it is currently used, or will be used in the future.

- A tendency for subjective bias

Applications are disproportionately important to their owners. The application’s availability, for example, might be a factor in the measurement of their performance.

A better solution can be found using architectural models, as shown in Figure 55.

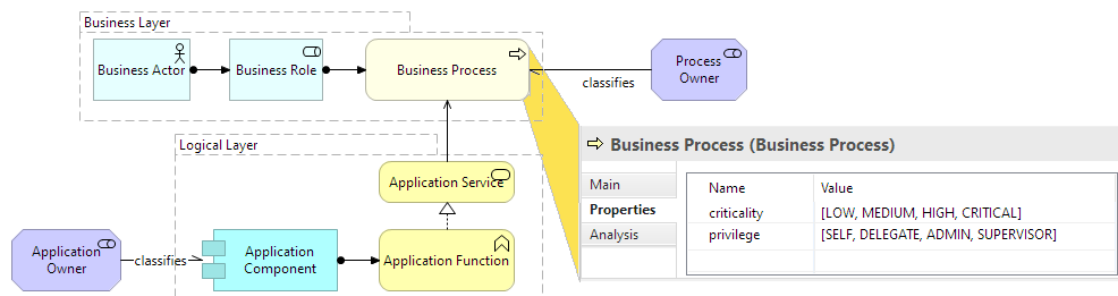


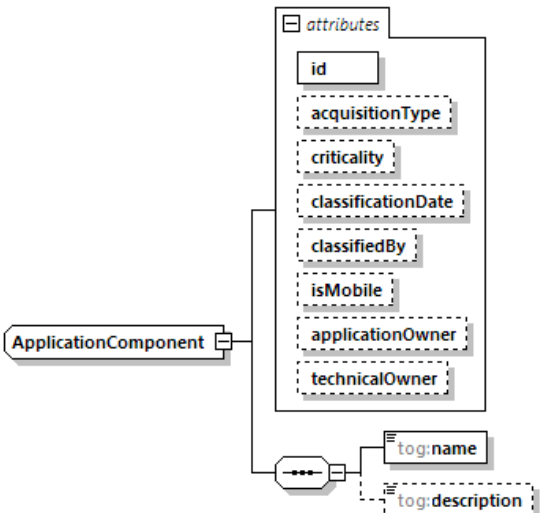
Figure 55: Critical Applications

A better definition of a critical application is “one that supports mission-critical capabilities and value streams”. Analysis performed at the contextual level has enabled the model to trace this value to business processes. The Security Overlay provides a criticality property to identify critical business behavior elements.

Figure 55 can be analyzed to identify Application Components that provide functionality to Business Processes rated “CRITICAL”. All such Applications are candidates for the critical list. Their candidature can be confirmed by inspecting the criticality property of the serving relationship.

The criticality of an application can be determined as the highest common denominator of critical process-centric analysis. Any Application that provides “HIGH” or “CRITICAL” services to a “CRITICAL” process should rank highly in any Critical Application Register.

Table 34: Application Component Properties

Element	Properties	Schema
	acquisitionType identifies the means of acquisition: [UNCLASSIFIED, CUSTOM, OPENSOURCE, COTS, GOTS]. Its value can indicate intent at development time or derived from an Artifact post-deployment; The properties criticality, classificationDate and classifiedBy capture the certification lifecycle; isMobile is a flag that identifies mobile code, defaulting to FALSE; applicationOwner is responsible for authorizing access and performing periodic recertification; technicalOwner is responsible for defining a secure configuration.	

8.1.2 Security Configuration

Security Configuration is a stereotype of Data Object that declares any configuration settings that differ from the baseline (least common functionality) configuration. It is appropriate at the Logical layer because different instances of the same building block may have different configurations in different contexts. Properties match those of the baseline configuration but with values toggled between ENABLED and DISABLED.

8.1.3 Software Defects and Malware

As Defects are associated with specific software releases, discussion is deferred to the Technical Layer. Nevertheless, being modeled as a <<Vulnerability>> (Motivation layer elements can be placed anywhere), it is often useful to associate functional defects with application behavior elements directly.

The Security Overlay includes a Malware stereotype of Application Component that can represent non-human agents or malware threat. It provides a type property that enumerates a controlled vocabulary of threat types with a default set of values drawn from the STIX™ taxonomy.³⁵

8.1.4 Data Assets

A means of tracking critical data assets is a prerequisite for their effective protection. When those assets are subject to regulation, as is the case with personal or medical data, it is also mandated by compliance.

As with applications, the criticality of data stems from the criticality of information in the contextual layer. As seen in Section 6.1.2, the Security Overlay enables Business Objects to be annotated with properties that convey their security classifications (confidentiality, privacy,

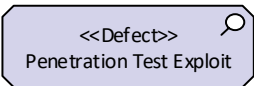
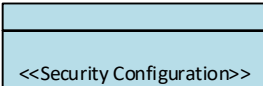
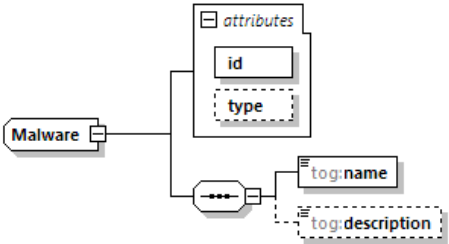
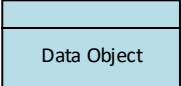
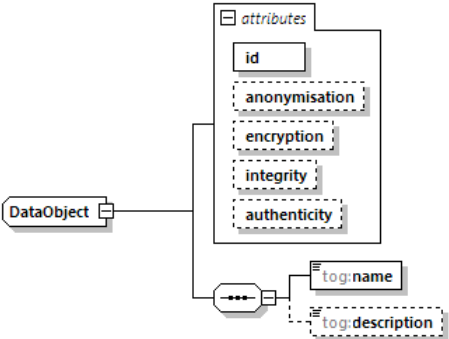
³⁵ Structured Threat Information Expression (STIX) is a language used to express cyber threat intelligence.

integrity, authenticity, and retention period), enabling all logical representations of that information to inherit these protective markings.

Modeling provides a means to bind data objects to their business significance and show its provenance (where it comes from), how it is processed (by which application functions having what read/write/modify capability), where it goes (flow relationships, sometimes traversing domain boundaries), and who has access to it (the business actors and roles to whom it is served). By tracing down to the Technology Layer, as will be shown in Section 9.1.1.2, we can also determine its storage location.

The security properties that enable a Data Object to contribute to the protection of information include the ability to implement data-level controls for confidentiality, privacy and integrity, using various techniques of encryption, anonymization (tokenization, redaction) and integrity protection (checksum, HMAC, signature).

Table 35: Data Object Properties

Element	Properties	Schema
	Vulnerability elements represent defects published in CVE™ lists or discovered in the software development lifecycle.	Discussed in Technology Layer
	Security Configuration declares functionality that has been enabled in addition to the baseline (least common functionality) configuration. Properties match those of the baseline configuration but with values toggled between ENABLED and DISABLED.	An open schema in which to declare the security configuration of the application as deployed in a specific application context.
	Malware is used to model the threat of malicious code. The type property is based on an open vocabulary that includes values such as: ADWARE, BOT™, DOWNLOADER, KEYLOGGER, RANSOMWARE, ROOTKIT, SPYWARE, TROJAN, VIRUS, and WORM	
	Object properties declare controls applied at data level: anonymization for privacy protection: [NONE, MASK, REDACT, TOKEN, HASH, OBSFUCATE]; encryption for confidentiality protection: [NONE, 3DES, AES, AES256, RSA, ECC]; integrity for tamper detection: [NONE, CHECKSUM, MD5, HMAC, SHA1, SHA2, SIGNATURE]; and authenticity: [NONE, SELF, ASSERTION, SHARED_SECRET, SIGNED].	

8.2 Risk Modeling

8.2.1 Risk Modeling

Sections 4.3, 5.5, and 7.2 described the Security Overlay’s risk elements and risk management strategy without being prescriptive about any particular methodology that an organization may choose to adopt.

These high-level definitions can be adapted to suit an organization’s chosen risk methodology. To illustrate the application of these concepts, this section models a scenario described in the *Open FAIR™ Risk Analysis Example Guide* [G21A], which contains both qualitative and quantitative analyses.

8.2.2 The Scenario

The example scenario considers a situation in which an HR Executive commits the security sin of posting log-in credentials on a sticky note attached to their workstation. The Executive’s office is accessible to five groups of people: constituencies of differing size, goals, motivation, access opportunity, capability, and competence.

Figure 56 shows how this scenario might be modeled in ArchiMate language.

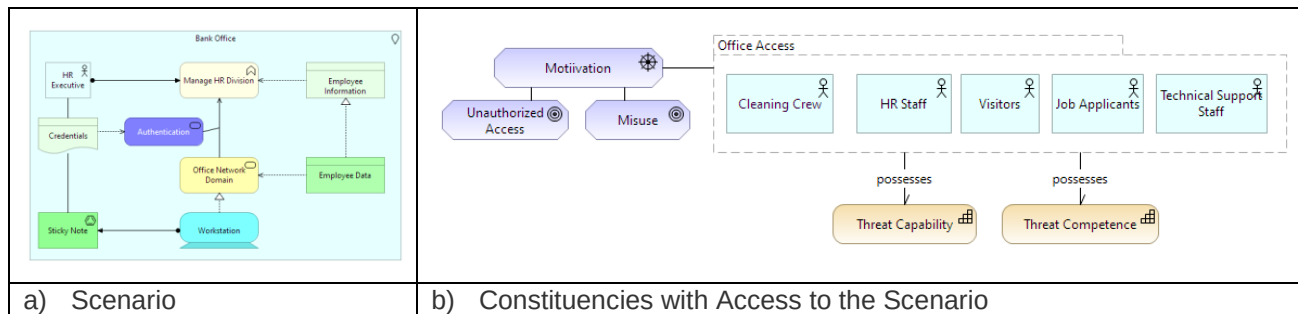


Figure 56: Open FAIR™ Example Scenario

The analysis described in the guide focuses on the risk³⁶ posed by one of these groups: the Cleaning Crew.

The Open FAIR Risk Taxonomy [C20B] defines a structure, reproduced in Figure 57, to analyze this risk. The assessment of any factor in the taxonomy is determined by a qualitative (risk matrix) or quantitative (mathematical) combination of its constituent factors.

³⁶ As noted earlier, the Open FAIR Taxonomy’s definition of risk as “the probable frequency and probable magnitude of future loss” (i.e., foreboding only negative potential) differs from that of the TOGAF Standard and SABSA.

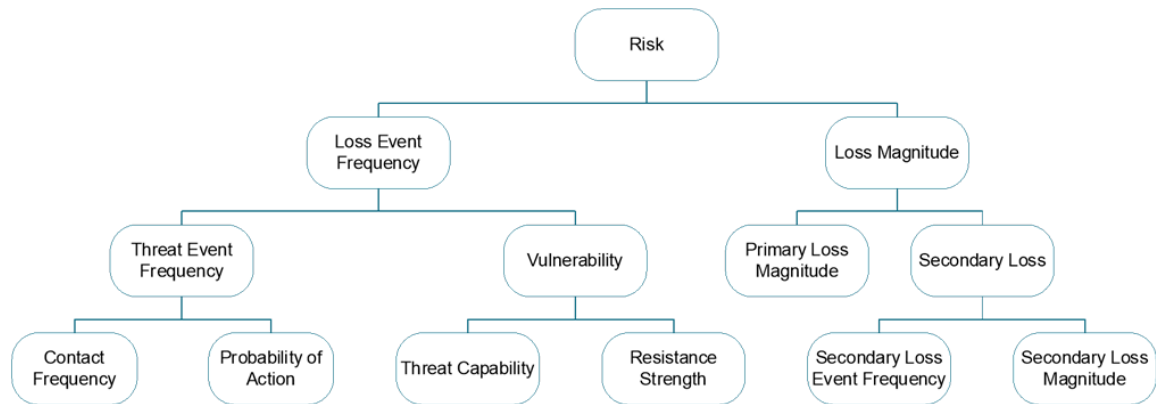


Figure 57: The Open Fair Risk Taxonomy

An equivalent ArchiMate representation of this model is shown in modeled in Figure 58.

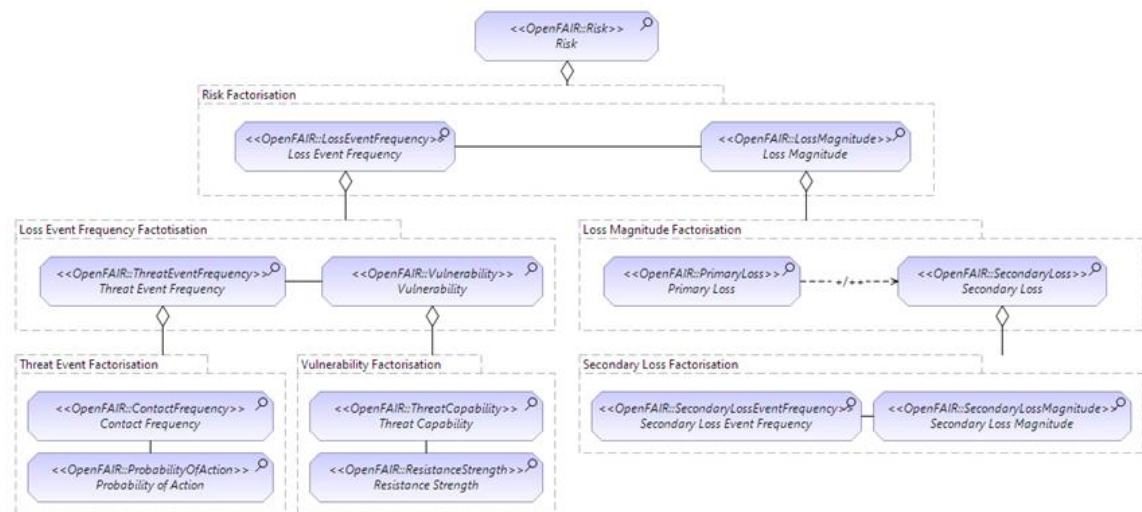


Figure 58: The Open FAIR Risk Taxonomy Modeled in the ArchiMate Language

The following pages show the qualitative analysis of this example scenario (Figure 59) and the quantitative analysis in Figure 60.

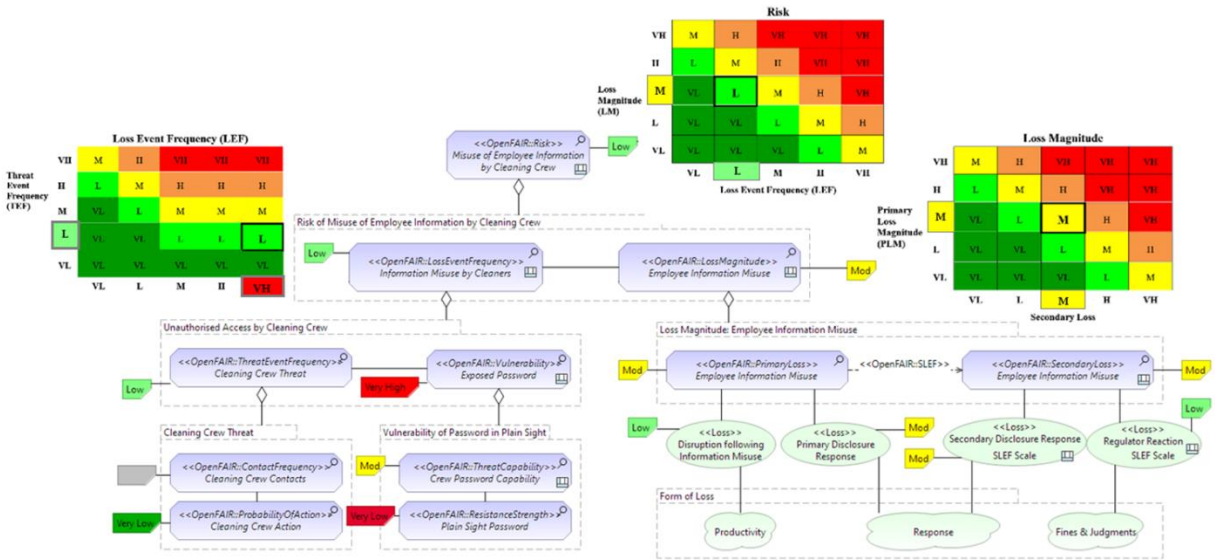


Figure 59: Qualitative Example from the Open FAIR Risk Analysis Example Guide

Figure 59 shows risk matrices combining factors using qualitative scales and Security Overlay elements, specialized and extended for the Open FAIR taxonomy.

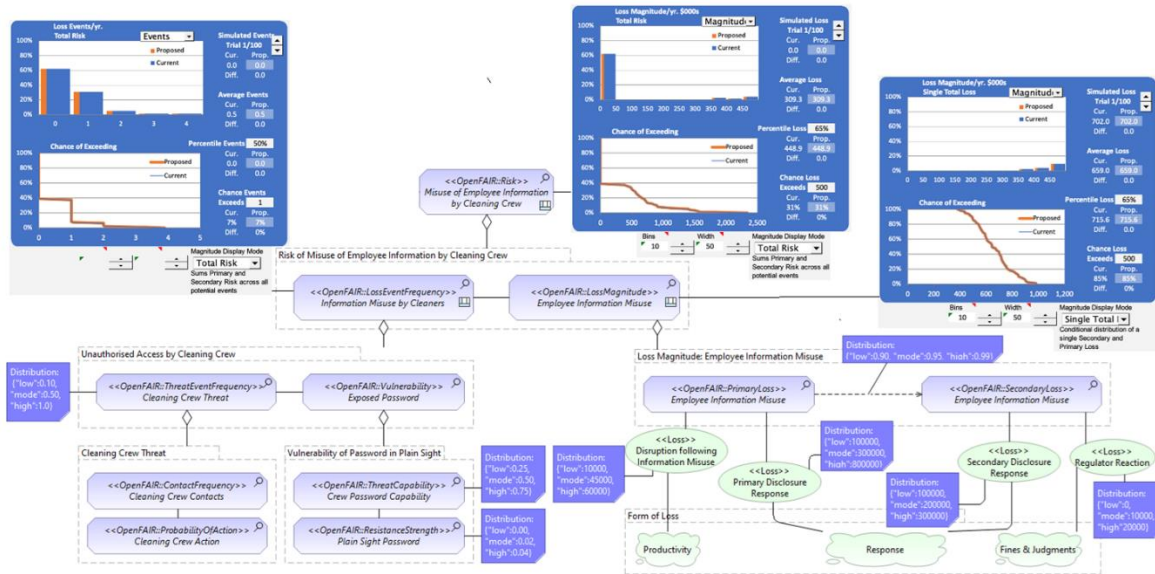


Figure 60: Quantitative Example from the Open FAIR Risk Analysis Example Guide

Figure 60 shows the analysis of the same scenario using quantitative methods in which estimates are made for the leaf nodes, based on a 90% confidence interval (low, high) and a most likely value (mode). Although the expression of qualitative or quantitative algorithms is beyond the current capability of the ArchiMate language, many tools offer programming and graphical support that can implement model-driven risk analysis based on input parameters structured in this way.

8.3 Application Functionality and Services

The Process cell of the SABSA Matrix Logical architecture addresses application functionality and services. Services are key to both availability and access control. The focus of this section is placed on availability. Access control will be discussed in the next section, the “People” cell, which continues the discussion of the design and implementation of LAM.

The previous sections enabled the model to trace the value of assets and mitigate risks from the information specified in the contextual layer. The model also provides a means of ensuring that Logical layer services can provide the necessary availability of and delivery to committed SLAs.

In terms of what we might call “supply-side availability”, the Security Overlay is based on a simple premise that the most important characteristic of internal behavior elements (processes, functions, and interactions) is their contribution to fulfilling the business mission. As discussed earlier, criticality can be determined through traceability to Value Streams, or by an enumerated criticality property with qualitative range of values.

For services, it is their availability: human-oriented services being presented as having a service window; technical services by an enumerated availability ranges (e.g., HIGH = 99.99 - 99.999%), supported by recovery targets, the Recovery Time and Recovery Point Objectives (RTO/RPOs).

On the “demand side”, availability is also affected by the resources available to handle the rate of incoming requests. Recall the Security Overlay’s annotation of properties for Actor population size, Business Service execution frequency, SLA reaction and response times, the cardinality on servicing relationships.

When traced from the Business Layer to the Application Layer, these factors provide information that can help determine whether Application functions and services are capable of meeting peak demand within the terms of the SLA. Diagrams like the example of Figure 61 might be used to identify critical business services reliant on applications not operated in high availability configurations.

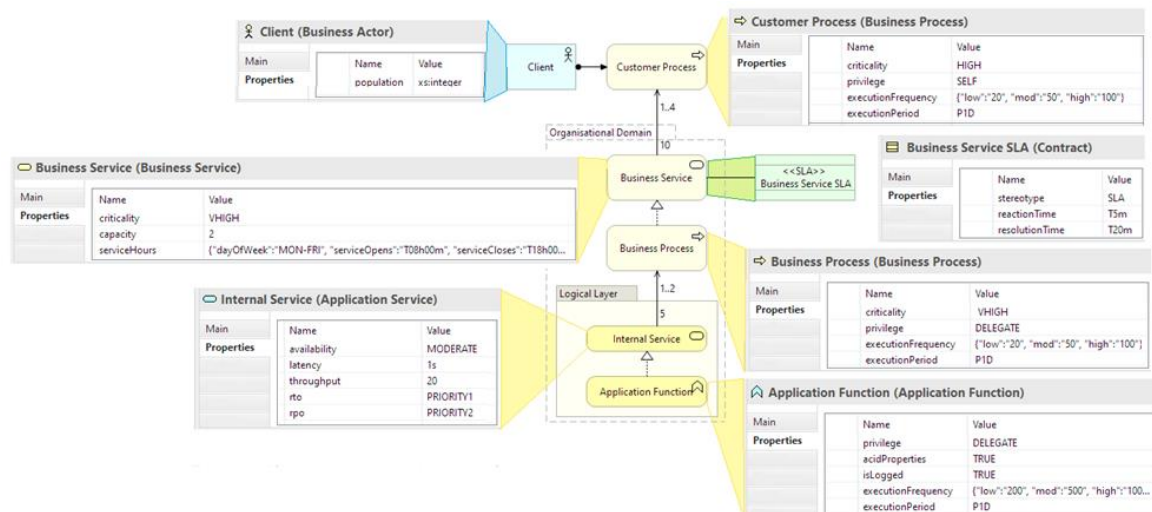
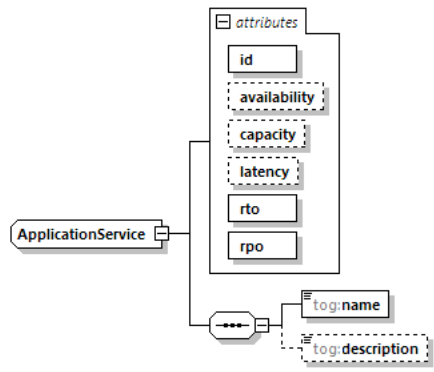
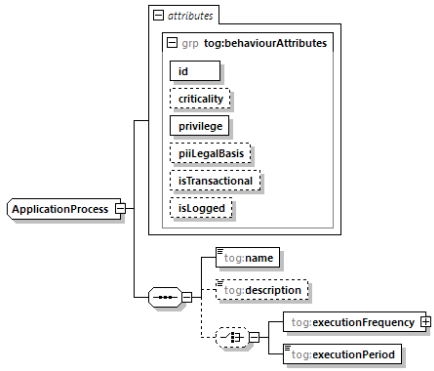


Figure 61: Application Services

The properties of the Application Behavior elements are shown in Table 36. Because service elements play a key role in both availability and access control, only the availability properties are explained in this section.

Table 36: Application Behavior Availability Properties

Element	Properties	Schema
 Application Service	Application Services offer functionality and availability. The availability properties include: - An availability requirement: enumerated ranges based on % up-time: [NA, UNCLASSIFIED, VLOW, LOW, MODERATE, HIGH, VHIGH] Capacity indicates the maximum supported limit for in-flight service requests - A latency target for maximum acceptable response time - Recovery Time & Point Objectives (rto and rpo) for recovery targets following any outage	
 Application Process  Application Function	Application behavior element properties most relevant to availability are: criticality captures the highest common denominator of business activity supported: [NA, UNCLASSIFIED, VLOW, LOW, MODERATE, HIGH, VHIGH] isTransactional flags whether the activity exhibits ACID³⁷ qualities isLogged flags whether the activity is logged executionFrequency and executionPeriod are used to express, as a confidence range, how often the process is performed Remaining properties are discussed in Section 8.4.4.	

³⁷ Referring to the Atomicity, Consistency, Isolation, and Durability properties of transactional database design.

8.4 Logical Access Management

A key security issue for Application Services is the ability to enforce access control, predicated in turn on the Service's ability to establish a minimum level of assurance of the digital identity presented by the user. On the premise that on the Internet, “*nobody knows you're a dog*”,³⁸ the Security Overlay must be able to express that while “*being a dog*” might not be a concern for some low-risk services, others will demand a degree of confidence that the Principal accessing the service is a legitimate proxy of the identified, real-world Actor.

To model this, the Security Overlay proposes properties that reflect the 3-level assurance criteria identified in NIST's Digital Identity Guidelines [NIST SP800-63-3]: The default schema proposes as a 3-tier classification, STANDARD, ENHANCED, ASSURED, that maps easily to the assurance levels described in NIST, but this can be adapted to other schemes such as ISO/IEC 29115's 4-level scheme [ISO/IEC 29115] or a proprietary scheme (e.g., numeric range), defined by the implementing organization.

Access control is increasingly implemented via external services (policy administration, information, and decision points), but enforcement is predominantly local and predicated on the Principal having authenticated to an account that has been provisioned with a set of access rights.

Credentials generally materialize at the logical level as some form of data (e.g., a passcode, dactylogram, or retina scan) that must be compared against the credential on record. These may or may not be associated with the target system account, depending on whether authentication is performed locally or represented by a trusted token from a centralized Authentication Service, as shown in Figure 62.

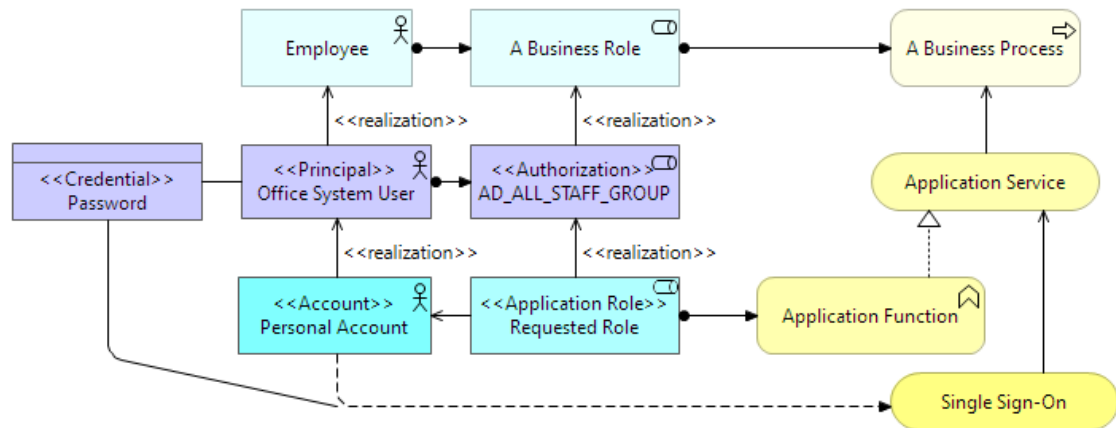


Figure 62: Modeling Authentication in the Primary Architecture

In the example of Figure 62, a Service Provider initiated Single Sign-on Authentication has been modeled as an Infrastructure service because that is how it looks from the Application's perspective: the paradigm of the externally-visible service redirecting an unauthenticated session to an authentication service.

The foremost reference on the good style and practical usage of the ArchiMate language [Wierda, 2021] advocates the structuring of models in three architectural planes: a primary architecture (describing run-time use), a secondary architecture (system development, administration, and

³⁸ Peter Steiner: The New Yorker, July 5, 1993

operations), and a tertiary architecture (ownership and governance). While authentication is a run-time process in the primary architecture, the service is best modeled as part of a LAM capability in the secondary architecture, as shown in Figure 63.

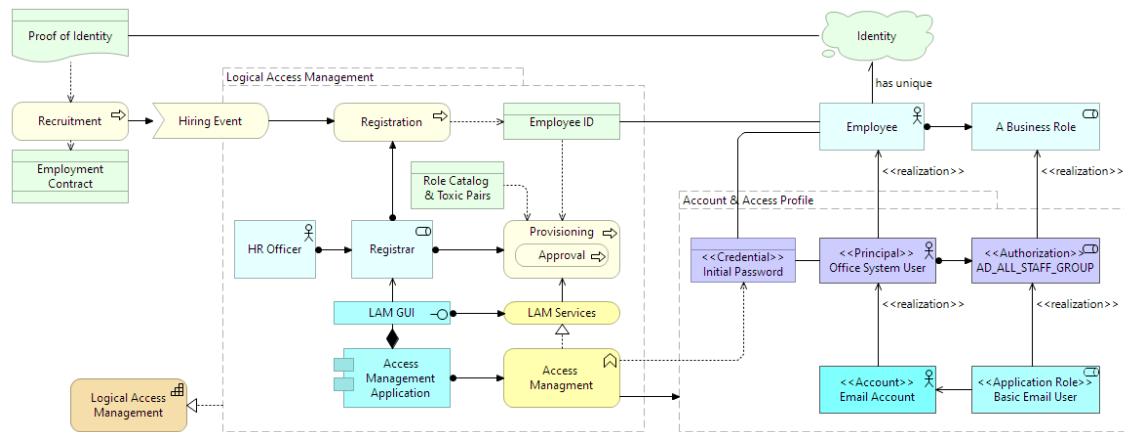


Figure 63: Example of Registration and Provisioning in the Secondary Architecture

8.4.1 Account

Account is the Logical layer representation of a Principal from the perspective of an application or target system. A Principal may hold multiple Accounts on multiple systems.

The principal security properties of an Account are:

- **type:** Indicates whether the account is intended for use by human individuals (NOMINAL), shared among a team (FUNCTIONAL) or machines (TECHNICAL) or a temporary disposable account used for testing, guest WiFi, etc. (DISPOSABLE)
- **domain:** Identifies the policy domain that defines account namespaces, naming conventions, the assignment or right to claim a particular account name, etc.

8.4.2 Application Role

Application Role represents the roles (and their associated set of access rights) defined in the application code or configurable on the target system. In the Security Overlay, Application Roles are fashioned from a stereotype of Application Component. They serve Accounts and can be assigned to Application functions, as shown in Figure 60 and Figure 61.

8.4.3 Application Service

Application Services were discussed earlier in relation to SLAs and Availability. In terms of their ability to assert access control, three additional properties are provided:

- **identityProof:** Enrolment and credential issuance processes bind the applicant to a real identity
- **authenticationStrength:** The assurance strength of the authentication mechanism itself
- **authenticatorTrust:** Assurance in the credential being presented, considering aspects such as bearer-type, one-time use, freshness, challenge/response, trust in the credential issuer, etc.

8.4.4 Application Process and Function

The key security-related property of an Application Function (and other internal behavior elements) is the **authorizationContext** in which the function is executed. It is important to declare when the application is performing the function using its own access rights or is acting as a proxy for the caller, under some form of impersonation or constrained delegation. The former indicates a highly privileged technical account that merits high assurance controls.

8.4.5 Application Interface

Application Interfaces bind Application Services to an endpoint or channel through which they may be consumed. Interfaces can be categorized into two types: human (e.g., command line, Graphical User Interface (GUI); voice-operated) and machine interfaces (e.g., remote procedure call, message-driven, Representational State Transfer (REST) or Simple Object Access Protocol (SOAP)), each of which is likely to be subject to different constraints or policies.

For example, an organization may decide that all machine-to-machine authentication must use certificates rather than passwords or that machine credentials should not be valid when proffered to human interfaces.

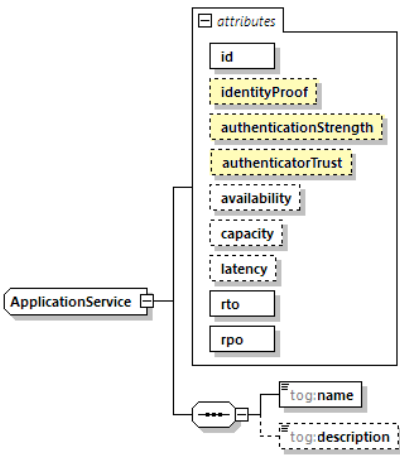
The Security Overlay provides a `type` property to indicate interfaces intended for human or machine use.

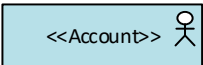
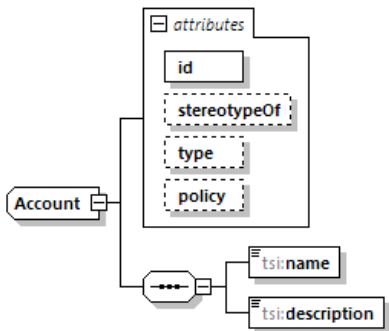
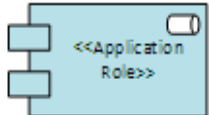
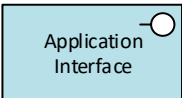
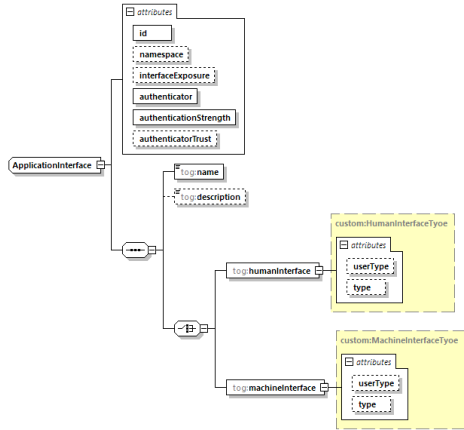
Each interface via which a service is exposed must satisfy the minimum **authenticationStrength** and **authenticatorTrust** requirements defined by the Application Service using a type-appropriate mechanism.

An **authentication** property declares the authentication mechanism supported by the interface, set to one from a list of predefined values: password, password hash, ticket, One-Time Password (OTP), public key, certificate, Security Assertion Markup Language (SAML), etc.

Table 37 presents a summary of overloaded elements discussed so far.

Table 37: Security Properties of Elements used in the Logical Layer

Element	Properties	Schema
	Application Service availability properties were discussed in Section 8.3 The access control properties, identityProof, authenticationStrength & authenticatorTrust are enumerated types that denote the required assurance.	

Element	Properties	Schema
	Account is the Logical layer representation of a Principal from the perspective of an application or target system type: indicates whether the account is intended for use by human individuals (NOMINAL), shared among a team (FUNCTIONAL) or machines (TECHNICAL) or a temporary disposable account used for testing, guest WiFi, etc. (DISPOSABLE) policy: identifies the policy domain that defines account namespaces, naming conventions, the assignment or right to claim a particular account name, etc.	
	An Application Role is modeled as a stereotype of Application Component. It defines a set of access rights on a single target system.	An open schema in which to declare a user-defined set of access rights.
	The interfaceType property distinguishes interfaces used by humans from those intended for machines. Each option has its own set of valid enumerated values The interfaceExposure property marks the interface as being exposed locally or remotely. The authenticator property declares the mechanism implemented at the interface; e.g., PASSWORD or BIOMETRIC and its associated assurance strength and trust (a reflection of the rigor of the user registration process)	

Element	Properties	Schema
Application Process Application Function	Application behavior element properties include: criticality captures the highest common denominator of business activity supported: [NA, UNCLASSIFIED, VLOW, LOW, MODERATE, HIGH, VHIGH] privilege indicates the access profile used in execution; e.g., [SELF, USER, ADMIN, ROOT] piiLegalBasis declares the justification for any personal data processed isTransactional flags whether the activity exhibits ACID qualities isLogged flags whether the activity is logged executionFrequency and executionPeriod express as a confidence range how often the process is performed	

8.5 Logical Domains

Logical domains can represent notional boundaries in the Logical layer, such as schema belonging to a particular namespace, web applications deployed under an Internet domain name, or the authorized usage (functions) covered by a Public Key Infrastructure (PKI) Signing Certificate.

As with other core layers, logical domains can be expressed by Grouping elements (Location being intended to represent only physical or conceptual space).

Pure logical space is modeled as a Grouping unadorned. A means of marking a Logical Domain as a security domain (conceptual space) is discussed in Section 7.5.

8.6 Timing and Events

8.6.1 Application Security Events

These elements can be used to model security-significant errors and exceptions thrown by application behavior; e.g., an exception encountered while validating a digital signature. The schema is the same as that presented in Section 7.6.

9 Modeling the Physical Security Architecture

The Physical Security Architecture provides a security perspective of the Builder's view of the target system. The entities considered in this layer include the physical forms of persisted information (software and data), physical infrastructure (hardware, system software, and network components) and human-machine interfaces.

This layer of the SABSA Matrix is concerned with how these components are structured and managed holistically to provide an operational environment of technological platforms and physical space. The security aspects of each component taken individually are addressed in the SABSA Component Architecture.

When it comes to modeling, it is difficult to show the organization of components without the components themselves. We must also contend with the fact that the ArchiMate Specification has no direct equivalent of the Component Architecture, its Physical layer representing the world of industrial robots, IoT, facilities, and distribution networks. For these reasons, this section discusses the Physical and Component layers together. The artifacts of the SABSA Physical and Component layers are shown in Table 38 and Table 39.

Table 38: SABSA Physical Architecture

		Assets (What)	Motivation (Why)	Process (How)	People (Who)	Location (Where)	Time (When)
Physical	Artifacts	Data Assets	Risk Management Practices	Process Mechanisms	Human Interface	Infrastructure	Processing Schedule
		Data Dictionary & Data Storage Devices Inventory	Risk Management Rules & Procedures; Risk Metadata	Working Procedures; Application Software; Middleware; Systems; Security Mechanisms Process Control Points	User Interface to Business Systems; Identity & Access Control Systems	Workspaces; Host Platforms, Layout of Devices & Networks	Timing & Sequencing of Processes & Sessions
	Activities	Physical Asset Management	Risk Data Management	Operations Management	User Support	Resources Management	Performance Data Collection
		Change Management; Platform & Data Storage Management	Risk Procedure Management; Risk Metadata Management	Job, Incident, Event & Disaster Recovery Management	Service Desk, Problem & Request Management	Physical & Environmental Security Management; Real Estate & Facilities Mgmt	Business Systems Monitoring; Procedure Management

Table 39: SABSA Component Architecture

		Assets (What)	Motivation (Why)	Process (How)	People (Who)	Location (Where)	Time (When)
		Components and Standards					
Component	Artifacts	Component Assets	Risk Management	Process	Human Entities:	Locator	Step Timing Sequencing
		Products and Tools, including Data Repositories & Processors	Risk Analysis Tools; Risk Registers; Risk Monitoring & Reporting Tools	Tools & Protocols for Process Delivery; Application Products;	Identities, Job Descriptions; Roles; Functions; Actions & Access Control Lists	Nodes, Addresses & other Locators; Component Configuration	Time Schedules; Clocks, Timers & Interrupts
	Activities	Delivery and Continuity Management	Operational Risk Management	Process Delivery Management	Governance Relationship & Personnel Management	Environment Management	Time & Performance Management
		Assurance of Operational Excellence & Continuity	Risk Assessment; Risk Monitoring & Reporting; Risk Treatment	Management & Support of Systems, Applications & Services	Management & Support of Enterprise-wide & Extended Enterprise Relationships	Management of Buildings, Sites, Platforms & Networks	Management of Calendar & Timetable

This outline enables us to visualize the set of artifacts and activities that create and maintain the Physical Architecture, expressed in ArchiMate language in Figure 64.

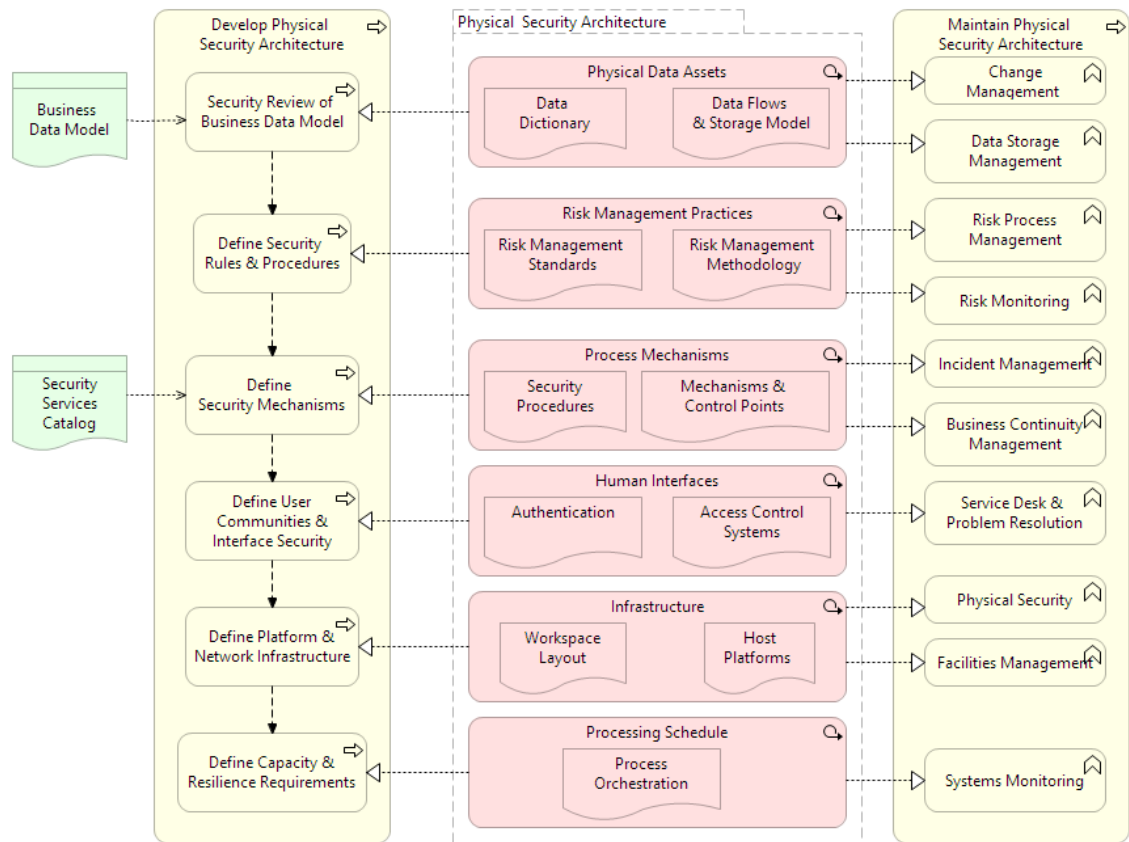


Figure 64: Developing the SABSA Physical Security Architecture

9.1 Data and Technology Assets

The principal Assets in this cell are the persisted formats of information/knowledge (applications and data) and IT infrastructure (hardware, system software, and network components). The “bricks & mortar” type of physical Assets are discussed in Section 9.5.

9.1.1 Artifact

In standard ArchiMate notation, Artifact serves as the universal passive structure element of the Technology Layer. It is overloaded to represent any kind of data object in the file system: executables, scripts, data and configuration files, databases, documents, specifications – everything in fact, except System Software.

The Security Overlay refines Artifact into sub-types that better reflect its different purposes and provides appropriate properties; see Figure 65. These stereotypes are Executable, Data, and Configuration. Having distinct stereotypes facilitates the creation and maintenance of separate software and data inventories.

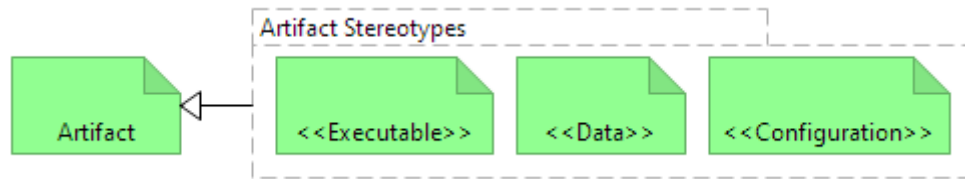


Figure 65: Stereotypes of Artifact

The consequence of this in the model is that an Artifact element's **isAbstract** property is TRUE by default, and turned FALSE only when one of these stereotypes is selected.

9.1.1.1 Executable

Executables are annotated with the same version control and file integrity properties as System Software. Commercial software can also be associated with Common Vulnerabilities and Exposures (CVE) vulnerabilities (Software Defects), whereas bespoke software vulnerabilities may be associated with internally-managed defect reports arising from static analysis, penetration testing, bug reports, etc.

9.1.1.2 Data

Data Artifacts realize Data objects from the Logical layer. Their security properties also reflect those of the Data Object, but here **encryption** and **integrity** refer to controls applied at the file/database level rather than those managed by the application. A **type** property is used to distinguish between artifacts stored in the file system and those that are part of a database.

9.1.1.3 Configuration

A configuration file associated with an Executable or System Software element in the Technology Layer represents a Baseline Configuration that should include all relevant security settings (secure by default). Different instances of this component in the Logical layer may opt to enable or relax these settings to achieve a Least Privilege configuration in a specific application context. Figure 66 illustrates this principle.

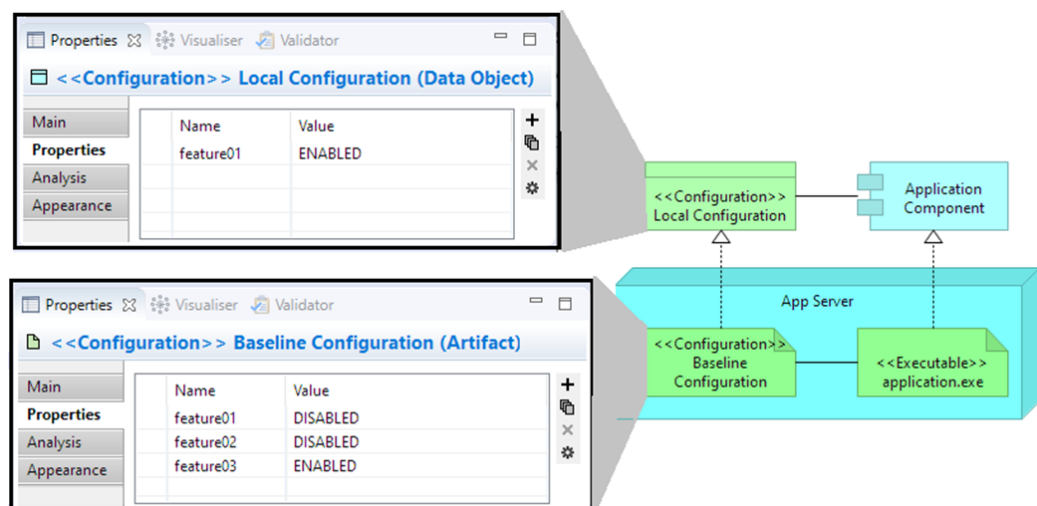


Figure 66: Configuration Files

9.1.2 Device and Node

Nodes represent a computational or physical resource that hosts, manipulates or interacts with other computational or physical resources. Devices model physical IT resources (i.e., hardware) upon which system software and Artifacts may be stored or deployed for execution.

When modeling security infrastructure, it is sometimes useful to model nodes and devices having a specific security purpose as distinct stereotypes so that role-specific properties can be assigned.

For example, Figure 67 shows a Hardware Security Module (HSM) being modeled as a stereotyped Device with properties that reflect FIPS-140-2 assurance levels and a VPN Gateway as a stereotyped Node with properties that reflect the Common Criteria Evaluation Assurance Levels (EAL) for the protection profile of a type of product.

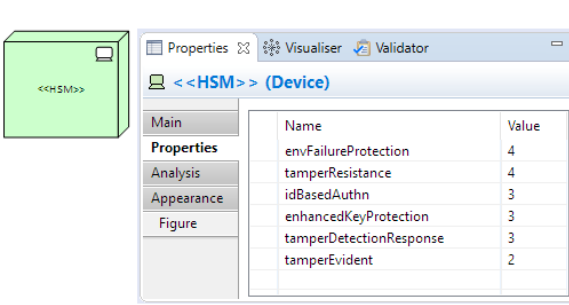
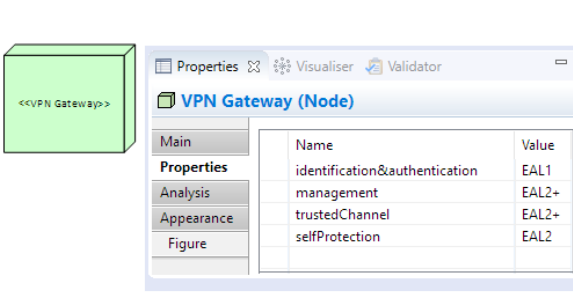
	
HSM with FIPS 140-2 Assurance Level Properties	VPN Gateway with Common Criteria EAL Properties

Figure 67: Stereotyping Security Nodes and Devices

9.2 Risk Management Practices

9.2.1 Defect

Software Defects are either published in product threat advisories, CVE lists, or discovered in custom code. Because architectural models are mostly concerned with deployed code (either in the current or some future state), defects are more likely to be associated with Production code than those still in Development.

The main exception to this is an organization’s process for approving deviations. Often there is legitimate business pressure for software to be deployed into Production despite defects: a planned security control may not have been completed in time, or a penetration test has uncovered a problem at the eleventh hour. In such circumstances, a decision must be made on risk. Models can be analyzed as “what if” scenarios to inform these risk decisions. Figure 68 illustrates how highly relevant risk information is readily at hand.

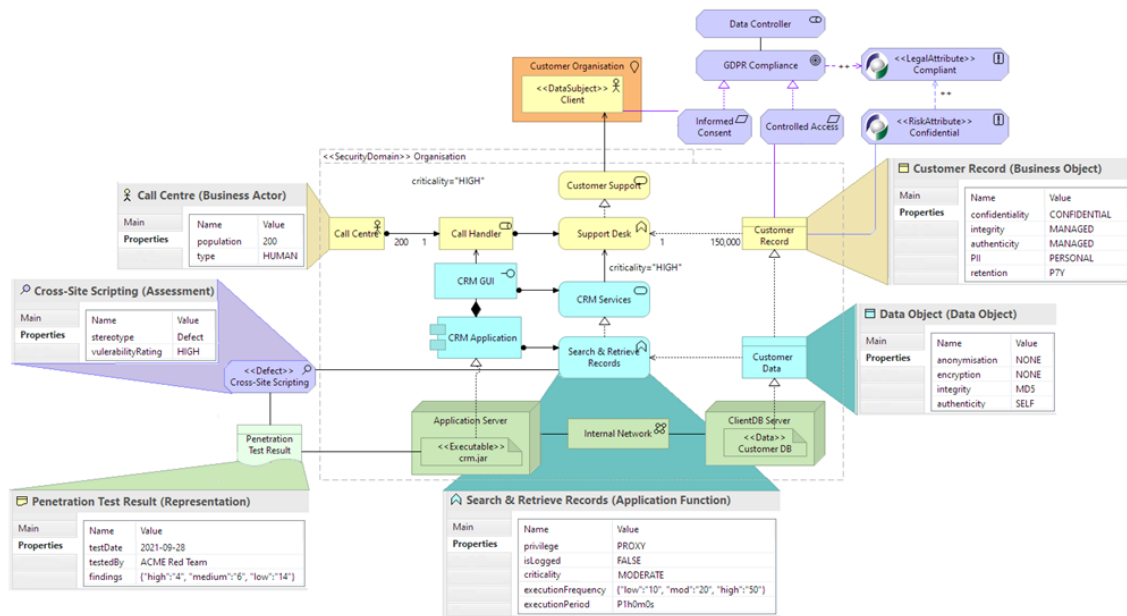


Figure 68: Risk Management Practices

In the Security Overlay, defects are modeled as specializations of Vulnerability. The Cross-Site Scripting (XSS) Vulnerability in Figure 66 evaluates the severity of the technical findings of a penetration test recently performed on a CRM application. The report identifies 24 defects, four of which are rated HIGH. One of these is manifest in the application’s “Search & Retrieve” function. This function has read access to Customer Data, information classified as CONFIDENTIAL PERSONAL and openly identifiable (no anonymization). The “Search & Retrieve” function is considered of MODERATE criticality because the Customer Relationship Management (CRM) Support Desk (itself a moderately critical Business Process) is highly reliant on it. The function runs as a proxy with the access rights of the End User. As Search and Retrieve actions are not logged, there is no audit trail of records that have been accessed. Finally, we can see that the vulnerable function is used by 200 Call Center accounts, each with access to 150,000 Customer records. Any breach would be a compliance issue, requiring the Controller to notify the GDPR Supervisory Authority.

9.3 Process Mechanisms

From a security perspective, Technology Services fall into two main categories: those that realize the security services identified in the conceptual layer, and those that provide general platform services to Applications.

Figure 69 illustrates the pattern by which a conceptual security service, authentication, is realized by two distinct Technology Services: one which provides a single sign-on capability for the organization domain, and the other, the local authentication mechanism of a cloud application.

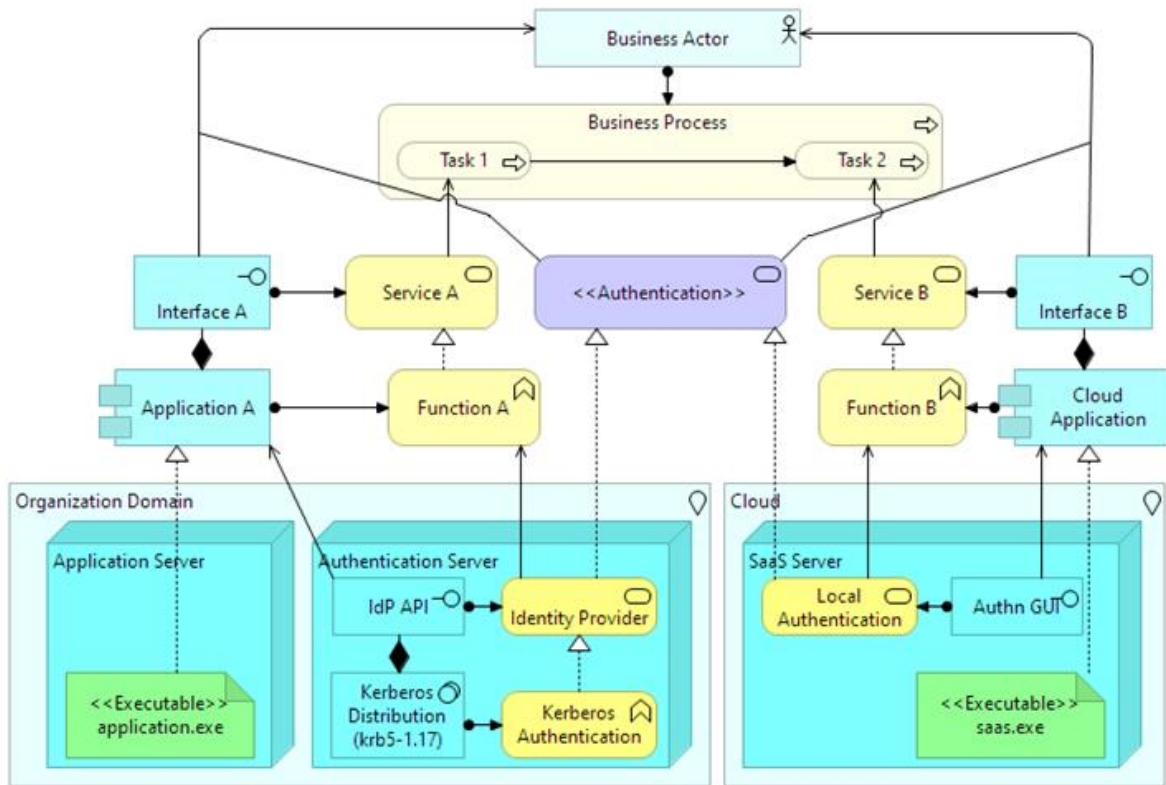


Figure 69: Realization of Security Services

Similar patterns can be deployed for other security services provided by the infrastructure: Security Incident Event Management (SIEM), Privileged Access Management, Virus Scan, Data Leak Prevention, and so on.

The closed loops, including the invocation of the conceptual Authentication service, its technical implementations and the services being protected, means that the model can not only verify that assurance strength requirements required by each Application Service are met but also trace any Compliance requirements by identifying the closed loops described in Section 7.2.2.

Turning to Technology Services in general, while the ability to enforce access control is as important as the earlier analysis of Application Services, it is often mitigated by being co-located on the same Node, local network, or network domain. Priority should focus, therefore, on service invocations that originate outside trust domain boundaries.

9.3.1 Technology Functions and Services

Technology functions and services are modeled in much the same way as their counterparts at the Application level (Section 8.3). They share the same security concerns of availability and access control, so it should come as no surprise that their Security Overlay properties are the same.

9.3.2 System Software

The most important property of system software is its release version. This information is key to determining its patch status and any CVEs³⁹ (Software Defects) to which it may be vulnerable.

The Security Overlay proposes two properties for this purpose: **version**, which declares the release version and a derived property, and a derived property, **patchStatus**, which makes explicit whether the release version is current. If the executable code carries some form of integrity protection; e.g., a hash or code signing, this can also be denoted using the **integrity** property.

9.4 Human-Machine Interfaces

9.4.1 Technology Interface

Technology Interfaces represent endpoints that specify how Technology Services can be accessed by other active structure elements in the environment. These may be other Technology Nodes, Application Components, or Actors/Roles (e.g., relevant to a Management Interface).

The properties of the Technology Interface replicate those of the Application Interface, but in practice, are more likely to be technical Application Programming Interfaces (APIs) serving Application Components. GUIs tend to be used when Operators require intervention.

9.5 Physical Environment

The Location element is perfectly suited to representing physical workspaces or the place where IT equipment is located (e.g., Data Centers). Network zones and other logical or policy-governed boundaries are better represented by Groupings, marked with the **security** property set TRUE when coinciding with a security domain, as described in Section 7.5.

9.6 Timing and Interrupts

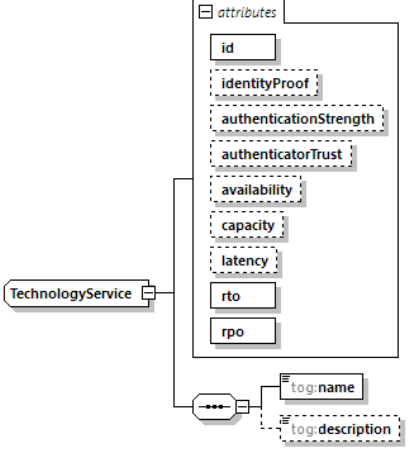
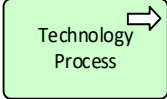
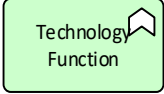
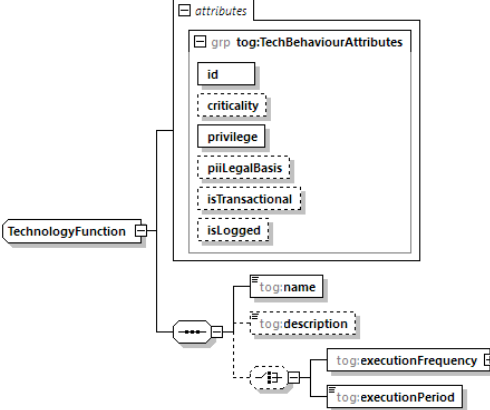
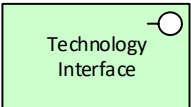
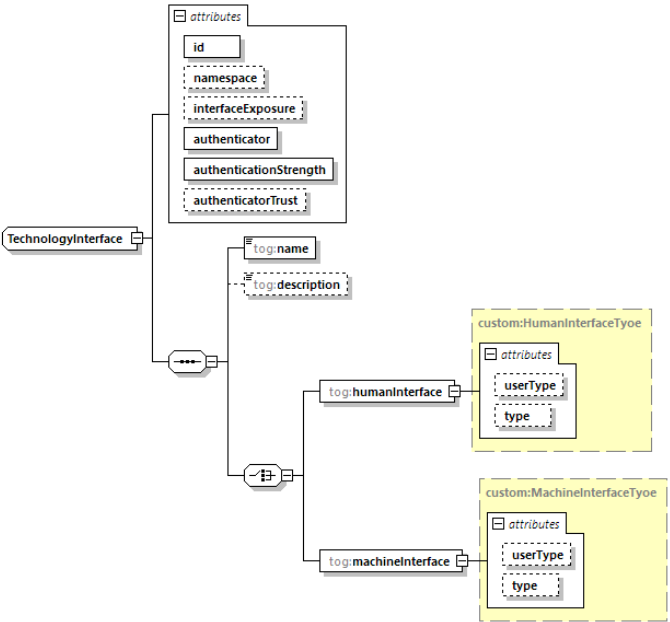
9.6.1 Technology Security Events

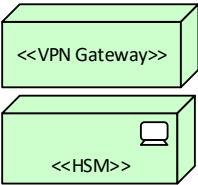
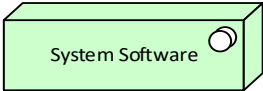
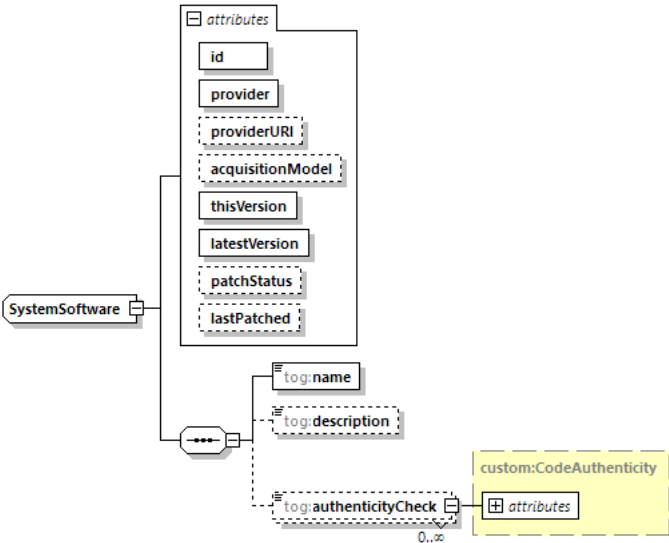
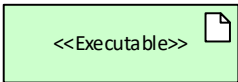
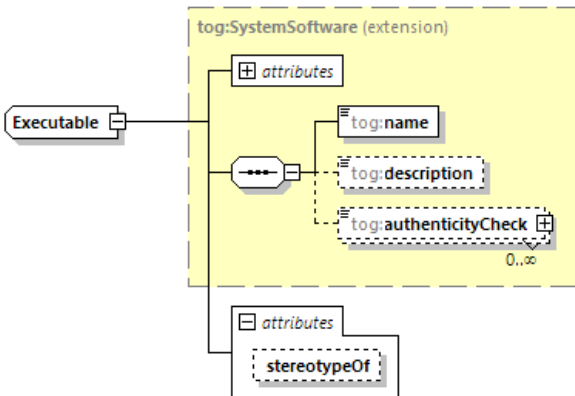
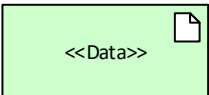
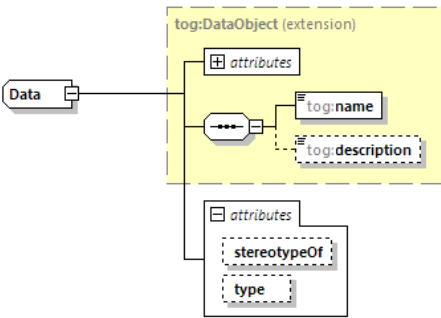
These elements can be used to model security-significant errors and exceptions thrown at the infrastructure level; e.g., an alert thrown by an Intrusion Detection System or Data Leak Prevention. The schema is the same as that presented in Section 7.6.

Table 40 presents a summary of Technology elements.

³⁹ A list of Common Vulnerabilities & Exposures; refer to: <https://cve.mitre.org>.

Table 40: Security Properties of Elements used in the Technical Layer

Element	Properties	Schema
 Technology Service	Infrastructure Service properties mirror those defined for Application Services.	
 Technology Process  Technology Function	Technology behavior properties mirror those defined for Application behavior.	
 Technology Interface	Technology Interface properties mirror those defined for Application Interface.	

Element	Properties	Schema
	Nodes and Devices with a specific security purpose can be modeled as distinct stereotypes so that role-specific properties can be assigned.	Element Specific
	System Software properties declare the patch release version, date and status (derived), and any code integrity protection. The authenticityCheck is used to verify signed code.	
	Executable is modeled as a stereotype of Artifact but with the same properties as System Software.	
	Executable is modeled as a stereotype of Artifact but with the same properties as DataObject plus a type property that indicates the storage medium; e.g., FILE, DB_TABLE.	

Element	Properties	Schema
<<Defect>> Code Review <<Defect>> Static Analysis <<Defect>> CVE-2019-wxyz	Defect is modeled as a specialization of Vulnerability, adding properties that reference an external document such as a CVE or Test Report.	
<<Security Configuration>>	Security Configuration declares functionality that has been enabled in the baseline (least common configuration). Properties are user-defined but set to either ENABLED or DISABLED.	Element Specific

10 Conclusion

This document describes a viable and versatile Security Overlay to support the SABSA approach with the ArchiMate Enterprise Architecture modeling language. The expected benefit of such an alignment is that Enterprise Security Architecture concepts can be incorporated into a unified Enterprise Architecture model that is shared with other architects and reflects the reality of the enterprise's single architecture. Security becomes an integral part of the design.

The ability to generate consistent views of a single model will benefit all who rely upon them: the more accurate, consistent, and complete the model, the better the analysis, decision-making, change planning, and execution is likely to be. Should a rudimentary alignment have proved impossible, the prospect of creating distinct Enterprise Architecture/Enterprise Security Architecture models, defined in different notations, by different tools, by different teams would be a death knell. Synchronization would likely be so complex and resource-intensive as to be unviable.

Equally important as what security brings to the Enterprise Architecture model is what modeling brings to security.

Firstly, visualization: security models are a complex mesh of concepts and relationships that are far better suited to representation by a simple yet expressive notation of symbolic nodes and lines than as checklists and matrices. Because diagrams are much more intuitive to create, understand, verify, and review, the result is more usable and of superior quality.

Secondly, the economics of modern IT drive projects to deliver more value earlier and repeatedly on ever shorter cycles. These “Agile” approaches shift emphasis heavily towards functionality, too often to the detriment of good structure and the planning, blueprints, and documentation overheads that go with it. In this world, architecture has been defined as the opposite of Agile: *“the decisions that need to be made correctly at the start of the project because they are difficult to change afterwards”* [Fowler, 2023].

SABSA teaches us that security architecture is not another architectural layer but a holistic concern, affecting all Enterprise Architecture layers. Risks arising from a de-emphasis on architecture and design are especially exposed by Agile. How can we perform an effective security design review when “the code is the design”?

It is not that Agile is implacably opposed to documentation – only that it should be an enabler and not a drag on the project. To this end, scaled Agile methodologies are turning towards Model-Based System Engineering (MBSE) as a means to keep the artifact set complete, up-to-date, consistent, and tailored to specific stakeholder views.

By generating these artifacts as views of a single underlying model, this overhead is vastly reduced. Reliable documentation can be maintained and published easily and regularly.

Re-use becomes possible on multiple levels: elements, patterns, and analysis. As these models expand beyond software design (UML) to full system models (ArchiMate Specification), this Security Overlay will enable SABSA to be lighter, faster, and better adapted to the short-cycle, Agile, minimum viable approach of modern projects.

Lastly, ArchiMate models are machine-readable because they are defined in a semi-formal notation that can be exported in a standardized XML Model Exchange File Format. The Security Overlay outlined in this document has maintained an eye on its potential use, not only for

documentation but also to query and analyze them via automated means. Should a security notation such as the one described here become a standardized extension to the language, the prospect of tools to support, for example, policy compliance checking or computer-assisted requirements generation, would become a realistic proposition.

In demonstrating this alignment, relatively few fundamental issues have been encountered. The most significant of these are the different concepts of an architectural Principle, the lack of a means of demonstrating the precise implementation of Controls and the absence of the elements necessary to populate the Conceptual layer. These challenges have been resolved by using the language's inbuilt extension mechanisms without breaking the grammar while retaining a degree of elegance.

Although there is still a significant amount to learn in this area, there is a basis here to work toward viable security patterns in the ArchiMate Specification that can lay the groundwork for a standardized security extension to the language in the future.

Acronyms and Abbreviations

ACID	Atomicity, Consistency, Isolation, and Durability
ADM	Architecture Development Method
API	Application Programming Interfaces
BCP	Business Continuity Plan
BPML	Business Process Modeling Language
CVE	Common Vulnerabilities and Exposures
DODAF	Department of Defense Architecture Framework
EAL	Evaluation Assurance Levels
EoI	Entity of Interest
FIPS	Federal Information Processing Standards
GDPR	General Data Protection Regulation
GUI	Graphical User Interface
HR	Human Resources
HSM	Hardware Security Module
IEC	International Electrotechnical Commission
IoT	Internet of Things
ISMS	Information Security Management System
ISO	International Organization for Standardization
ITIL	IT Infrastructure Library
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
KRI	Key Risk Indicator
LAM	Logical Access Management
MBSA	Model-Based System Engineering

MSA	Modeling SABSA ArchiMate
NIST	National Institute of Standards and Technology
O-ESA	Open Enterprise Security Architecture
OLA	Operational Level Agreement
OTP	One-Time Password
PII	Personally Identifiable Information
PKI	Public Key Infrastructure
RACI	Responsible/Accountable/Consulted/Informed
RBAC	Role-Based Access Control
REST	Representational State Transfer
RPO	Recovery Point Objectives
RTO	Recovery Time Objectives
SAML	Security Assertion Markup Language
SBB	Solution Building Block
SIEM	Security Incident Event Management
SLA	Service-Level Agreement
SME	Subject Matter Expert
SOA	Service-Oriented Architecture
SOAP	Simple Object Access Protocol
STIX	Structured Threat Information Expression
UML	Unified Modeling Language
XSS	Cross-Site Scripting

Index

Account	95	model-based engineering	5
ADM.....	20	model-driven approach	7
Agile	7	modeling.....	8
Application Layer.....	12	models	5
Application Role	95	Monte Carlo simulation	40
Application Services.....	95	Motivation elements.....	16
ArchiMate core elements.....	10	Motivation metamodel	29
ArchiMate elements	10	multilingual teams.....	6
Architectural Description	24	nodes	104
Architecture Development		non-directional association.....	26
Method	20	notation.....	6
articles	44	notational patterns	6
asset analysis	54	out-of-model specialization.....	19
Bayesian methods.....	40	Physical Security Architecture	100
Behavioral elements	55	Policy Architecture	69
bi-directional association.....	25	primary architecture	24
Business Attributes.....	32	realization.....	19
business geography	63	real-world Actor	94
Business Layer	12	real-world Controls	70
colors	15	regulations and standards	44
compartmentalization	62	relationship overloading.....	19
Compliance controls.....	43	review cycles.....	7
Conceptual Architecture.....	64	review feedback	7
Conceptual layer.....	65	review process.....	5
Conceptual layer elements.....	76	risk analysis.....	43
consistency	6	risk factors.....	68
control architecture.....	47	risk management	68
Control elements	42	risk methodology.....	68
control stereotype	41	SABSA Attribute	33
data artifacts	103	SABSA Attribute modeling	37
Data Owners	55	SABSA Component Architecture	100
de-personalization	62	SABSA Matrices.....	7
digitalization.....	5	SABSA methodology.....	7
documentation	5	SABSA Physical and Component	
executables	103	layers.....	100
Grouping elements	98	secondary architecture.....	24
human visual processing	6	security architecture	1
Implementation and Migration		security modeling	25
elements	17	Security Overlay	vi
information security	1	software defects.....	104
in-model specialization.....	18	Solution Building Blocks	5
legal texts.....	44	stakeholder feedback.....	5
Logical Architecture.....	84	Strategy elements	16, 31
Logical domains	98	Technology Layer	12
Miscellaneous elements.....	18	Technology Services	105

tertiary architecture.....	24
TOGAF Standard	1
Trust	48

Zachman Framework	1
zero trust principles.....	62